

GPT와 MCP로 구현하는 자율형 AI Agent 구축

강사 : 오영제

실습 환경 설정

Anaconda 설치

Download from <https://www.anaconda.com/download>

ANACONDA. Products Solutions Resources Partners Company

Free Download Sign Up

Distribution

Free Download*

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- ✓ Easily search and install thousands of data science, machine learning, and AI packages
- ✓ Manage packages and environments from a desktop application or work from the command line
- ✓ Deploy across hardware and software platforms
- ✓ Distribution installation on Windows, MacOS, or Linux

*Use of Anaconda's Offerings at an organization of more than 200 employees requires a Business or Enterprise license. [See Pricing](#)

Provide email to download Distribution

Email Address:

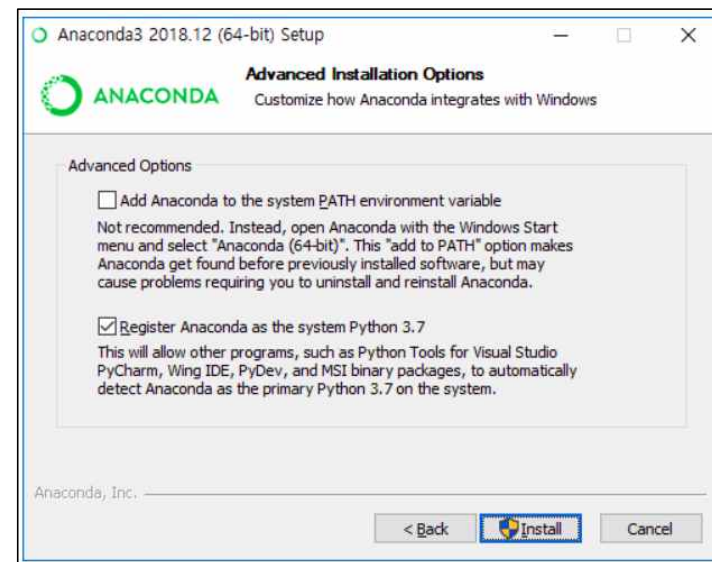
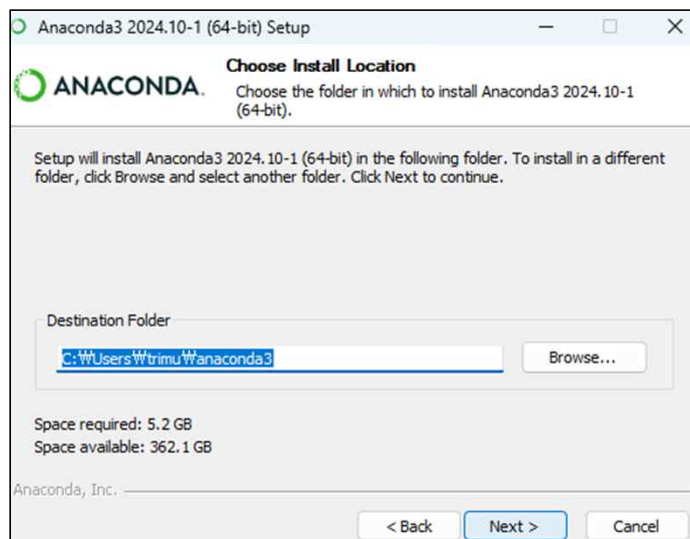
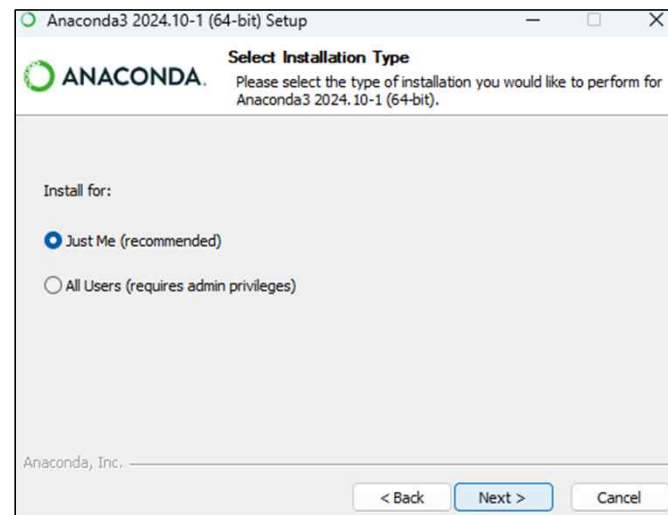
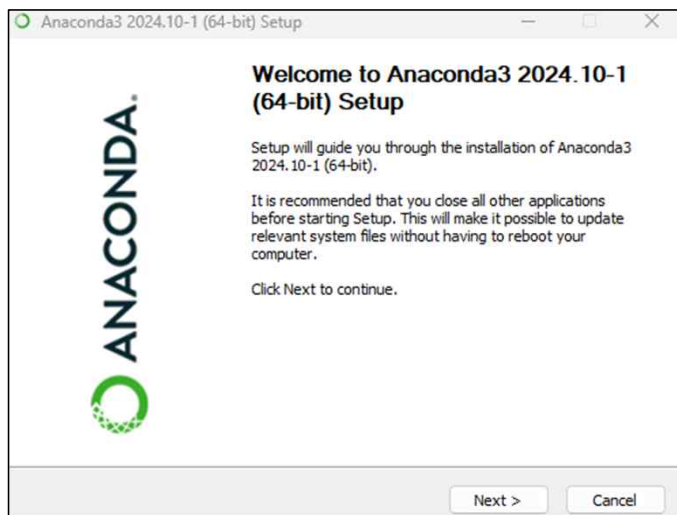
☐ Agree to receive communication from Anaconda regarding relevant content, products, and services. I understand that I can revoke this consent [here](#) at any time.

By continuing, I agree to Anaconda's [Privacy Policy](#), and [Terms of Service](#).

Submit >

Skip registration

← skip registration



가상환경 생성 및 package 설치

```
conda create -n langchain python=3.12 -y
```

```
conda activate langchain
```

```
pip install -r requirements.txt
```

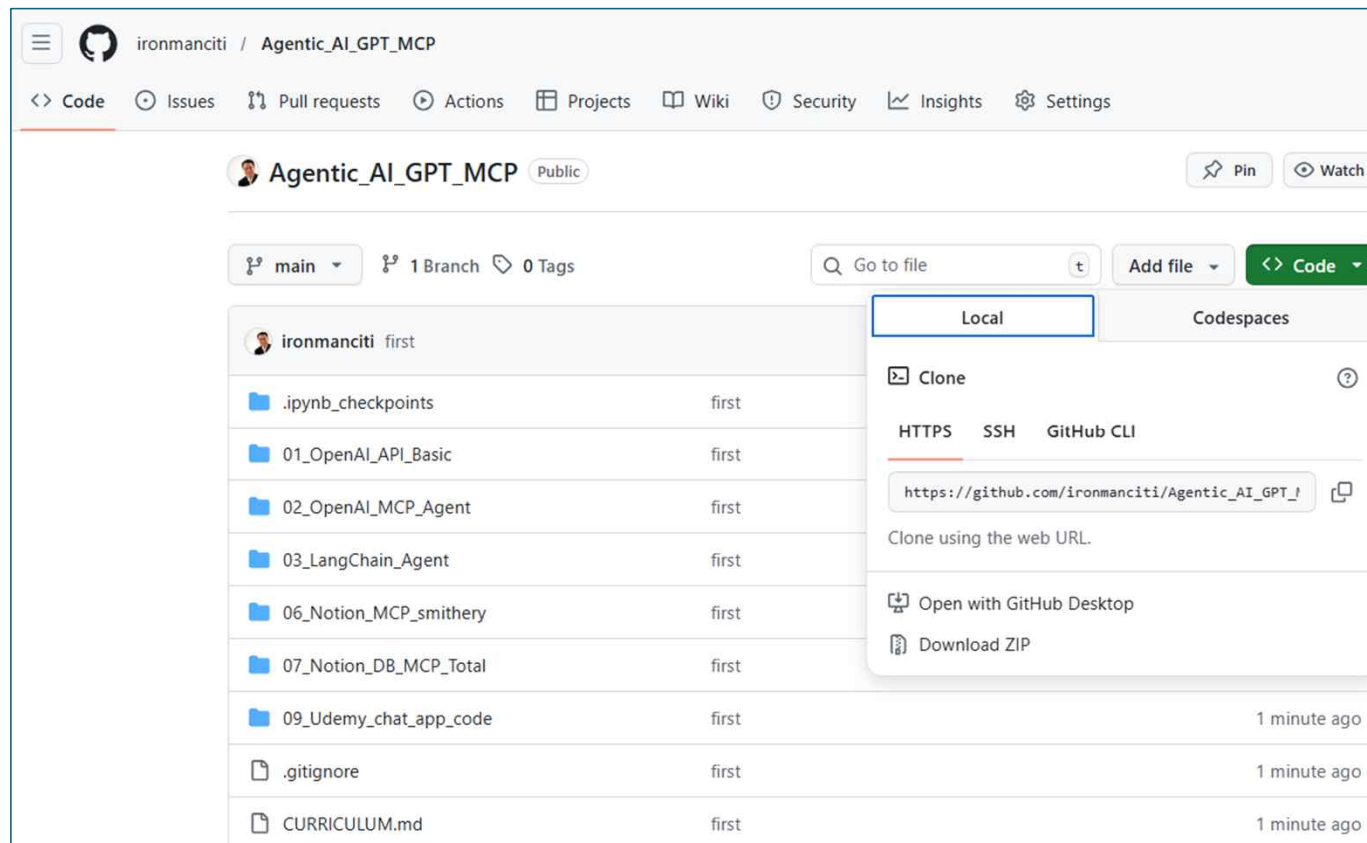
- package Install (수동 설치 경우)

```
pip install -U langchain
```

```
pip install -U langchain-openai
```

Github Repository 에서 실습 code download

- [github.com/ironmanciti/Agentic AI GPT MCP](https://github.com/ironmanciti/Agentic_AI_GPT_MCP)



vscode 설치 (<https://code.visualstudio.com/download>)

 Visual Studio Code

Docs Updates Blog API Extensions MCP FAQ

🔍 Search Docs Ctrl+Shift+P

Download

Version 1.108 is now available! Read about the new features and fixes from December.

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows
Windows 10, 11

User Installer [x64](#) [Arm64](#)
System Installer [x64](#) [Arm64](#)
.zip [x64](#) [Arm64](#)
CLI [x64](#) [Arm64](#)



↓ .deb
Debian, Ubuntu

↓ .rpm
Red Hat, Fedora, SUSE

.deb [x64](#) [Arm32](#) [Arm64](#)
.rpm [x64](#) [Arm32](#) [Arm64](#)
.tar.gz [x64](#) [Arm32](#) [Arm64](#)
Snap [Snap Store](#)
CLI [x64](#) [Arm32](#) [Arm64](#)

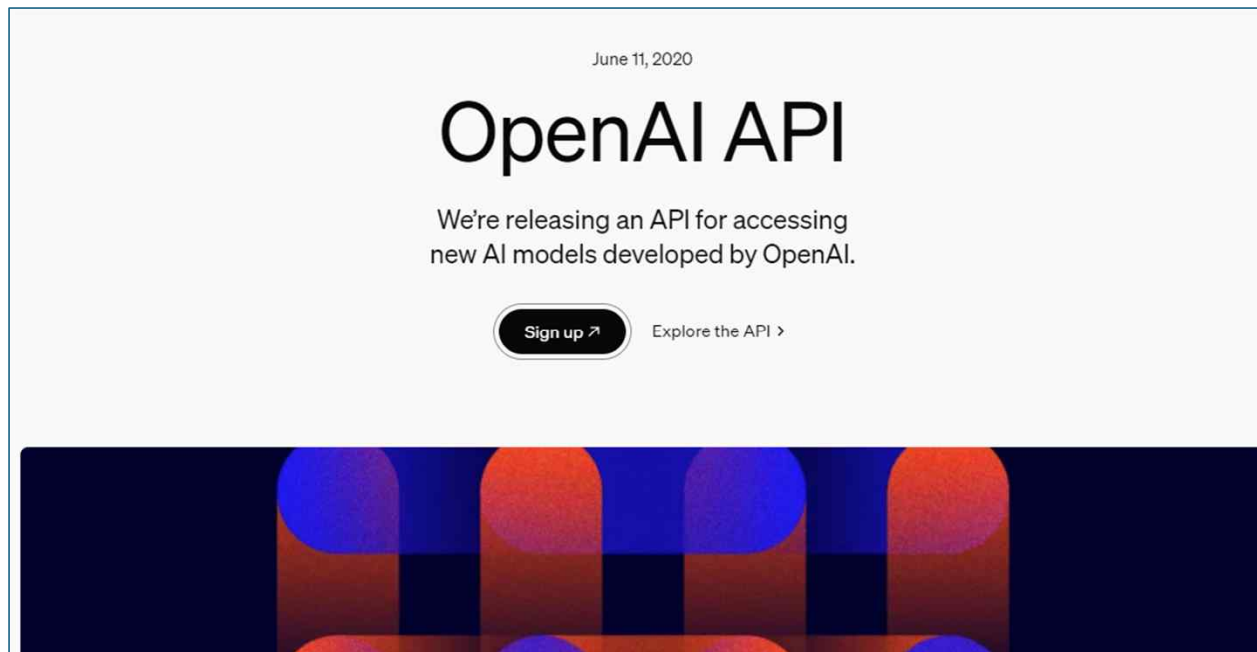


↓ Mac
macOS 11.0+

.zip [Intel chip](#) [Apple silicon](#) [Universal](#)
CLI [Intel chip](#) [Apple silicon](#)

OpenAI API Key 생성

<https://openai.com/index/openai-api/>



계정 만들기

이메일 주소*

[계속](#)

이미 계정이 있으신가요? [로그인](#)

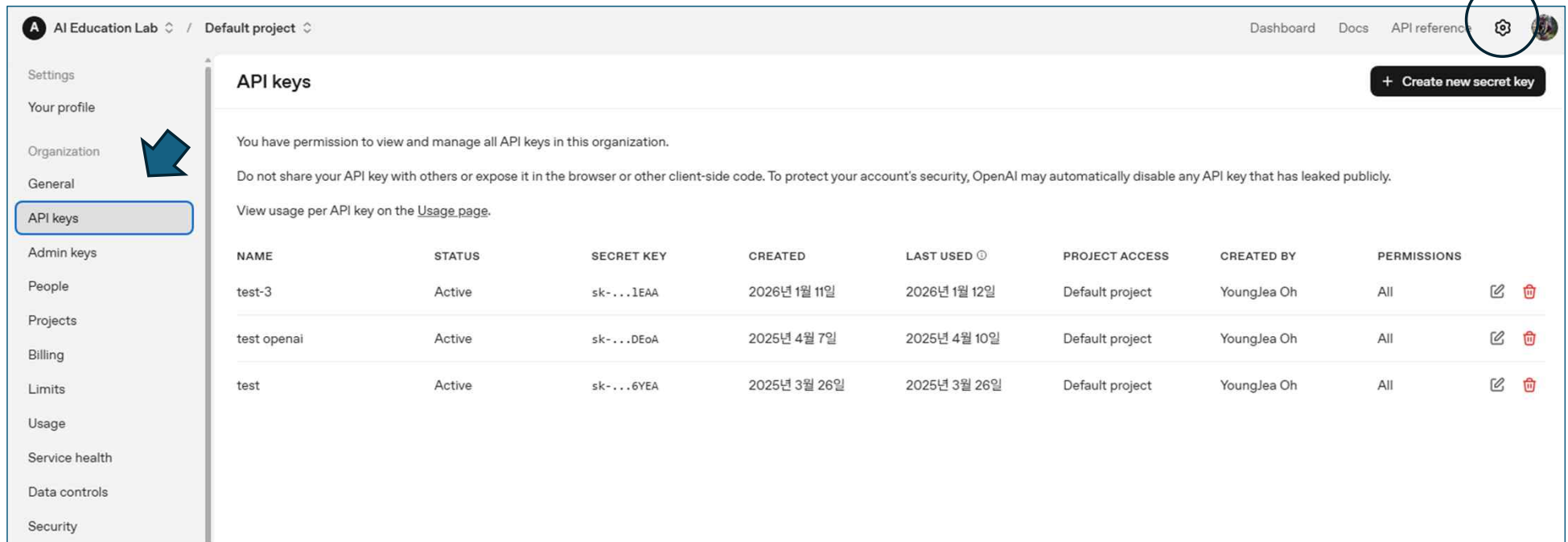
또는

 Google로 계속하기

 Microsoft 계정으로 계속하기

 Apple로 계속하기

Billing 정보 등록 (settings 에서 등록)



AI Education Lab / Default project

Dashboard Docs API reference

Settings
Your profile
Organization
General
API keys
Admin keys
People
Projects
Billing
Limits
Usage
Service health
Data controls
Security

API keys + Create new secret key

You have permission to view and manage all API keys in this organization.

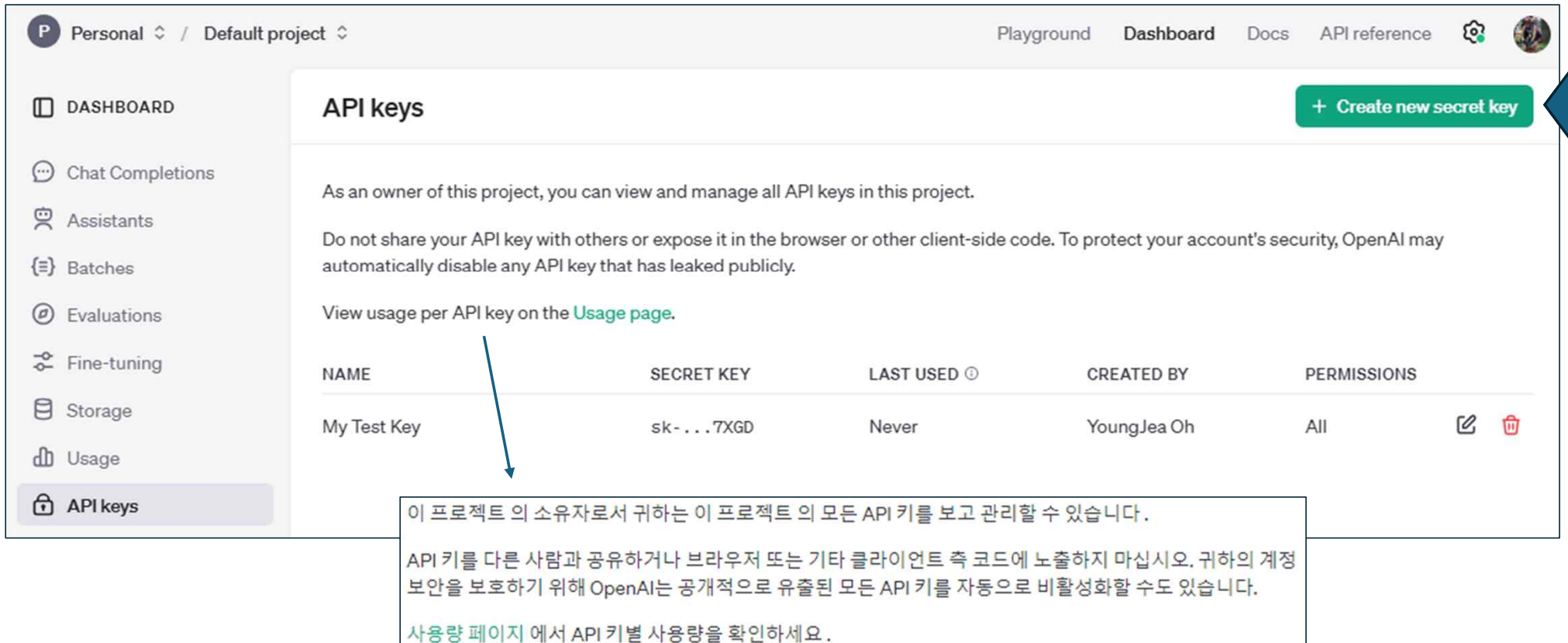
Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	STATUS	SECRET KEY	CREATED	LAST USED	PROJECT ACCESS	CREATED BY	PERMISSIONS
test-3	Active	sk-...1EAA	2026년 1월 11일	2026년 1월 12일	Default project	YoungJea Oh	All
test openai	Active	sk-...DEoA	2025년 4월 7일	2025년 4월 10일	Default project	YoungJea Oh	All
test	Active	sk-...6YEA	2025년 3월 26일	2025년 3월 26일	Default project	YoungJea Oh	All

API key 생성 (Dashboard → API keys)

<https://platform.openai.com/api-keys>



The screenshot shows the OpenAI API keys management interface. On the left is a sidebar with navigation links: DASHBOARD, Chat Completions, Assistants, Batches, Evaluations, Fine-tuning, Storage, Usage, and API keys (highlighted). The main content area is titled 'API keys' and includes a '+ Create new secret key' button (indicated by a blue arrow). Below this, there is instructional text about API key security and a link to the 'Usage page'. A table lists existing API keys. One key, 'My Test Key', is highlighted with a blue arrow pointing to a text box containing Korean instructions. The text box explains that as the project owner, the user can view and manage all API keys, warns against sharing keys, and directs the user to the 'Usage page' to check usage.

API keys + Create new secret key

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED ⓘ	CREATED BY	PERMISSIONS
My Test Key	sk-...7XGD	Never	YoungJea Oh	All

이 프로젝트의 소유자로서 귀하는 이 프로젝트의 모든 API 키를 보고 관리할 수 있습니다.

API 키를 다른 사람과 공유하거나 브라우저 또는 기타 클라이언트 측 코드에 노출하지 마십시오. 귀하의 계정 보안을 보호하기 위해 OpenAI는 공개적으로 유출된 모든 API 키를 자동으로 비활성화할 수도 있습니다.

[사용량 페이지](#) 에서 API 키별 사용량을 확인하세요.

Create new secret key

Owned by

☒ You ☐ Service account

이 API 키는 사용자와 연결되어 있으며 선택한 프로젝트에 대해 요청할 수 있습니다. 조직이 나 프로젝트에서 제거되면 이 키가 비활성화됩니다.

Name Optional

Project

Permissions

All Restricted Read Only

Cancel

Create secret key

Save your key

Please save this secret key somewhere safe and accessible. For security reasons, **you won't be able to view it again** through your OpenAI account. If you lose this secret key, you'll need to generate a new one.

BRuDSd7TQbDNViu39T3qT3B1bkFJEoTJIwcT8PhptPA

Copy

Permissions

Read and write API resources

Done

이 프로젝트의 소유자로서 귀하는 이 프로젝트의 모든 API 키를 보고 관리할 수 있습니다.

API 키를 다른 사람과 공유하거나 브라우저 또는 기타 클라이언트 측 코드에 노출하지 마십시오. 귀하의 계정 보안을 보호하기 위해 OpenAI는 공개적으로 유출된 모든 API 키를 자동으로 비활성화할 수도 있습니다.

사용량 페이지에서 API 키별 사용량을 확인하세요.

이름	비밀키	작성자	권한	
내 테스트 키	sk-...7XGD	오영재	모두	 

생성된 API Key 관리

- 타인 노출 안되도록 조심
- 노출되면 반드시 삭제 후 재 생성

API keys

+ Create new secret key

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED ⓘ	CREATED BY	PERMISSIONS
My Test Key	sk- . . . 7XGD	Never	YoungJea Oh	All

단일 프로젝트에 대해 API key 설정 방법

- .env – API 키가 포함된 로컬 파일 생성

```
# .env 파일  
OPENAI_API_KEY=sk-vvDtl*****XiilEpdjLhBJaH0f
```

- .gitignore 에 .env 파일 포함

```
# .env 파일을 git에서 무시  
.env
```

- Python code

```
pip install python-dotenv
```

```
from dotenv import load_dotenv
```

```
_ = load_dotenv() # local .env file을 읽어서 os.environ 에 OPENAI_API_KEY 추가
```

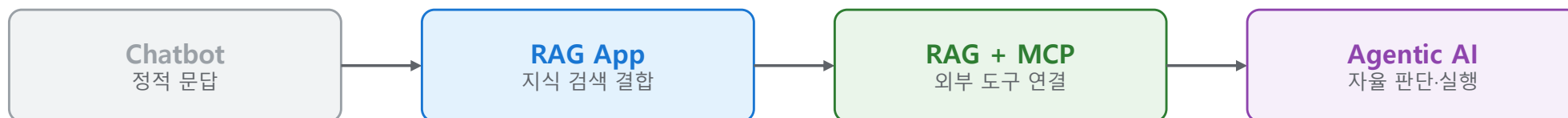
```
from openai import OpenAI
```

```
client = OpenAI() # os.environ.get("OPENAI_API_KEY")을 default 로 사용하여 API Key 이용
```

OpenAI Agents SDK

LLM 앱의 진화

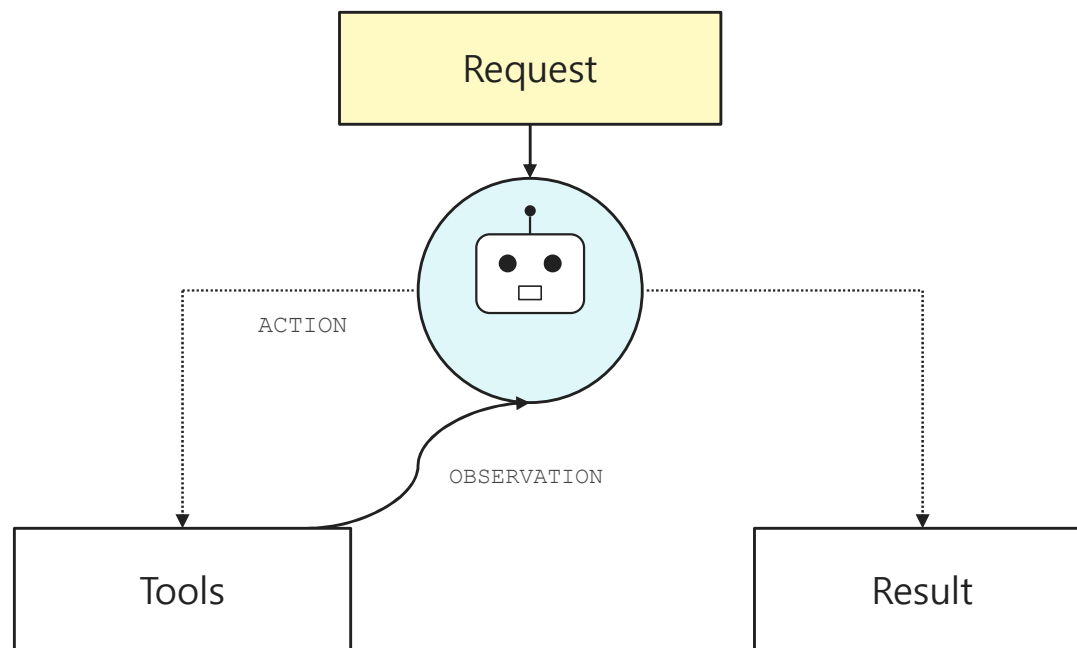
정적인 시스템에서 지능형 시스템으로 진화



LLM 단독으로는 불가능

- 실시간 데이터 접근
- 파일 열기·조작
- 개인·사내 문서 이해

해결: 도구(Tools)를 에이전트 루프로 호출



Agents: ReAct 루프의 핵심

Agent 란 ?

- LLM을 핵심 엔진으로 사용하여 주어진 목표를 달성하기 위해 독립적으로 작업(추론, 판단, 실행, 피드백)을 수행하는 인공지능 시스템
- 주로 LLM(대규모 언어 모델)의 능력을 기반으로 동작하며, 사용자의 명령을 이해하고, 판단하며, 실행
- 즉, LLM은 두뇌, Agent는 이 두뇌를 활용해 행동을 실행하는 작업자 역할
- 활용 사례
 - 고객 지원 챗봇
 - 연구 도우미
 - 개인 비서
 - 데이터 분석 에이전트
 - 교육용 튜터

1) Think



2) Call Tool



3) Observe



4) Answer

OpenAI Agents 구성 요소

OpenAI Agents SDK 핵심 구성 요소

- **모델 (Model)** : 고급 추론 능력과 에이전트 기능 내재 모델
- **에이전트 (Agents)** : LLM에 지시문과 도구를 장착한 실행 단위
- **핸드오프 (Handoffs)** : 특정 작업을 다른 에이전트에게 위임할 수 있도록 해 줌
- **가드레일 (Guardrails)** : 에이전트에게 전달되는 입력을 검증하는 장치
- **함수 기반 도구 (Function Tools)** : 에이전트가 호출할 수 있는 Python 함수 기반 도구
- **세션 (Session)** : 에이전트 실행 반복 과정(Agent Loop) 동안 필요한 작업 맥락을 유지해주는 영속적 메모리 레이어
- **추적 (Tracing)** : 에이전트 실행에 대한 내장 추적 기능

핸드오프 (Handoffs)

핸드오프 (Handoffs) 란?

- 에이전트가 다른 에이전트에게 작업을 위임하는 기능
- 각 에이전트가 서로 다른 전문 영역을 담당할 때 유용

예) 고객 지원 앱에서 **주문 조회**, **환불**, **FAQ** 등을 각각 전담하는 에이전트 구성

핵심 동작 원리

- Handoff는 LLM에게 **도구(Tool)**로 표현됨

예) "Refund Agent"로의 핸드오프 → LLM이 **transfer_to_refund_agent** 도구 호출

핸드오프 (Handoffs) 구현 방법

Handoff 생성 방법

■ 방법 1: handoffs 파라미터에 Agent 객체 직접 전달

- handoffs 파라미터에 위임 대상 **Agent** 객체를 전달
- **handoff_description** 설정 → LLM이 언제 이 핸드오프를 선택할지 판단하는 힌트

■ 방법 2: handoff() 함수 사용

- 위임 대상 에이전트를 지정하면서 세부 제어가 가능
- 추가 설정(**override**)이나 **입력 필터링** 같은 고급 기능 활용 가능

구조화된 출력 (Structured Output)

- 모델이 특정한 출력 구조를 따르도록 지시 받는 것
- 모델의 응답이 데이터베이스 스키마나 다른 정해진 형식에 부합하도록 보장
→ 모델 출력의 일관성 유지
- 구조화된 출력 구현 방법 → 출력 구조를 스키마로 정의
- 스키마는 JSON과 같은 단순한 형식으로 표현
- Pydantic과 같은 도구를 사용하여 타입 힌트와 유효성 검사를 포함한 스키마 정의

```
from pydantic import BaseModel
from agents import Agent, Runner

# 캘린더 이벤트 데이터 구조 정의
class CalendarEvent(BaseModel):
    name: str          # 이벤트 이름
    date: str           # 이벤트 날짜
    participants: list[str] # 참가자 목록

# 에이전트 정의: 텍스트에서 캘린더 이벤트 정보를 추출
agent = Agent(
    name="캘린더 추출기", # 에이전트 이름
    instructions="텍스트에서 캘린더 이벤트를 추출하세요. "
                 "이벤트 이름, 날짜, 참가자 정보를 구조화된 데이터로 반환하세요.",
    output_type=CalendarEvent, # 반환 데이터 형식
    model="gpt-5-mini",        # 사용할 LLM 모델
)
```

Tools & Tool Calling

LLM의 함수 호출 방법

LLM은 함수 호출(Function Calling) 기능을 통해 외부 도구 및 API와 안정적으로 통합될 수 있다.

JSON 기반 함수 호출:

- 모델은 함수 시그니처를 따르는 JSON 객체로 응답
- 개발자는 함수의 입력 파라미터를 JSON 스키마로 설명

자동 함수 호출 결정:

- 모델은 입력에 따라 적절한 함수를 호출할 시점을 감지

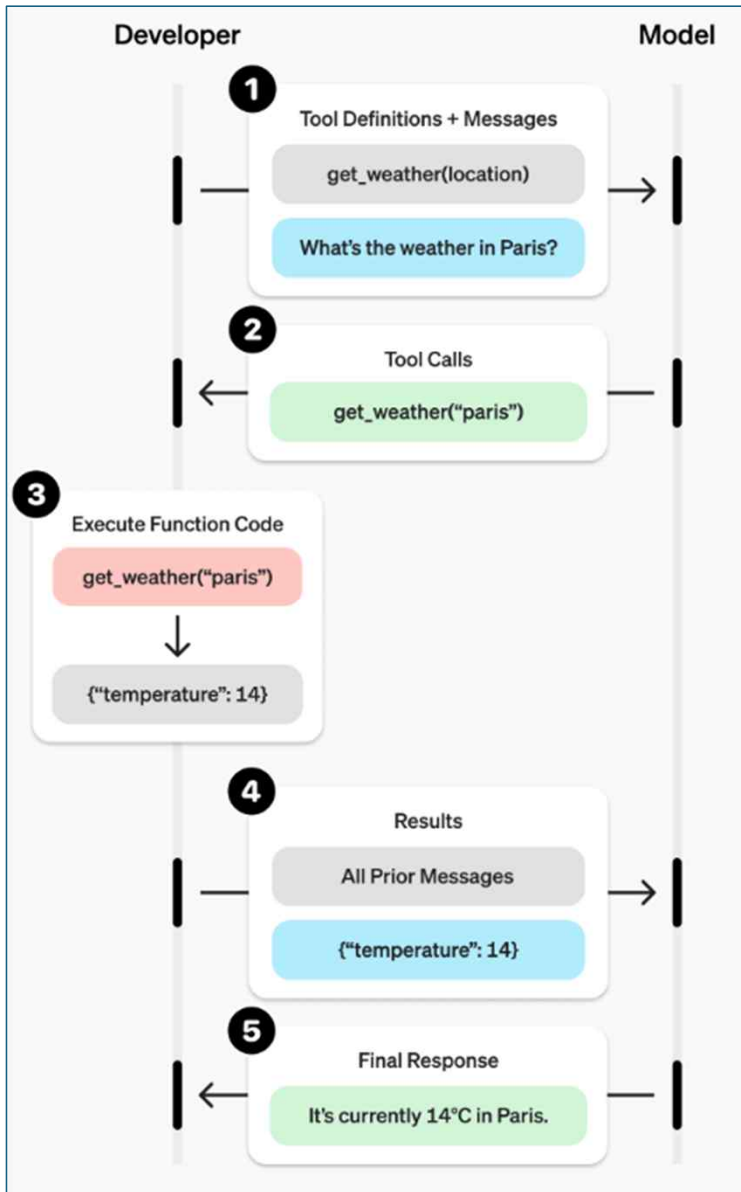
사용 사례:

- 외부 도구 호출: API, 플러그인 등과 연결해 작업 수행.

```
@function_tool
def multiply(x: float, y: float) -> float:
    """
    x 와 y 를 곱한다.
    """
    print("*** multiply 함수 실행 ***", x, y)
    return x * y

@function_tool
def get_weather(위도: float, 경도: float) -> str:
    print(위도, 경도)
    print(f"Weather 함수 실행 - 도시: {위도, 경도}")
    latitude=위도
    longitude=경도
    response = requests.get(f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current_weather=true")
    data = response.json()
    return data['current']['temperature_2m']

agent = Agent(
    name="Assistant",
    instructions=(
        "당신은 유용한 도우미 입니다. 가능하면 제공된 도구를 사용하세요."
        "당신은 기상 전문가 입니다. 도시의 경도와 위도를 주면 날씨를 알려줍니다."
        "사용자 질의에 답하기 위해 당신의 지식에만 너무 의존하지 말고 도구를 사용하세요."
    ),
    model="gpt-5-nano",
    tools=[multiply, get_weather]
)
```



• 함수 호출 단계

- (1) 함수 정보와 함께 LLM 모델 호출
- (2) 모델이 함수 호출을 결정 – 함수명과 argument 반환
- (3) 함수 호출 – 모델이 만들어준 argument 이용
- (4) 함수 호출 결과를 모델에 제공
- (5) 모델이 최종 응답 생성

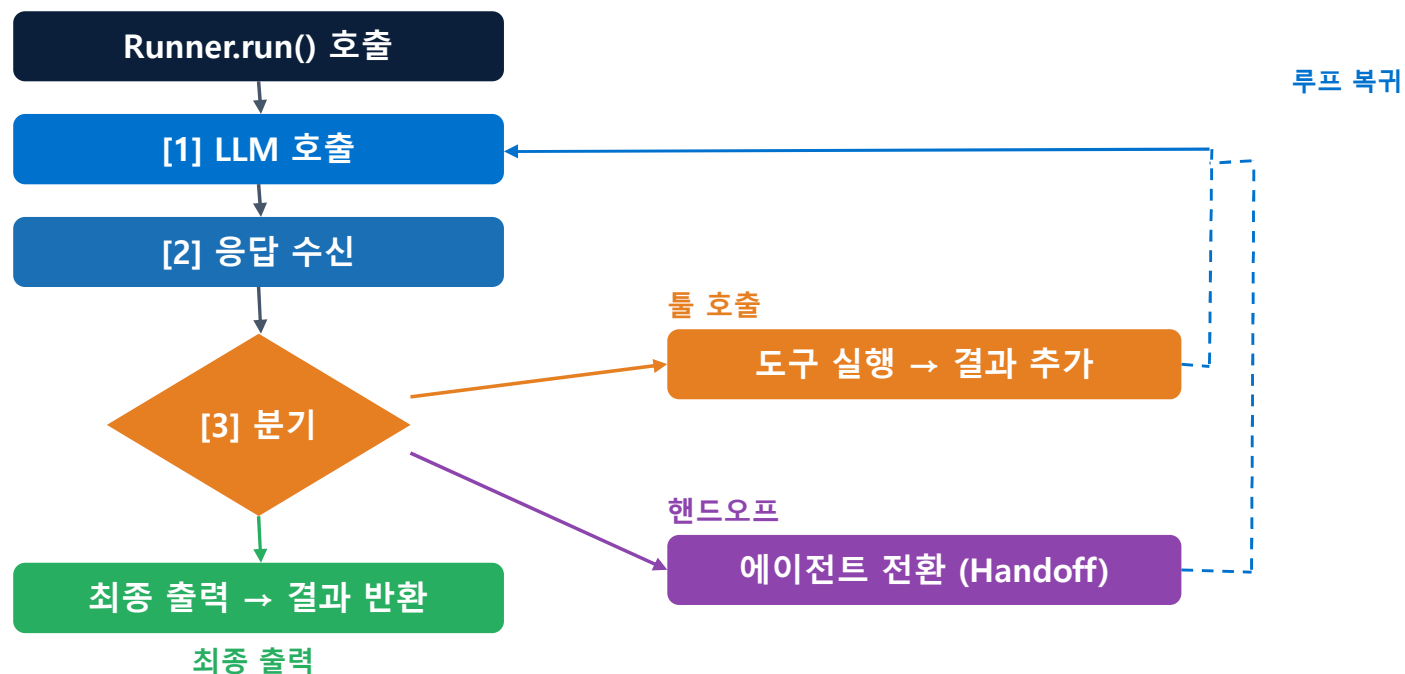
OpenAI Agents SDK 설치

OpenAI Agents SDK 설치

- `pip install openai-agents`
- <https://developers.openai.com/api/docs/guides/agents>
- <https://github.com/openai/openai-agents-python>

Agent Loop 흐름

에이전트 루프 동작 순서



툴 호출/핸드오프 → [1]로 돌아가 LLM 재호출, 최종 출력 시 루프 종료

■ Start ■ [1] LLM ■ [2] 응답 ■ 툴 호출 ■ 핸드오프 ■ 완료

output_type 3가지 케이스

output_type에 따른 최종 출력 결정

Case 1: output_type 미지정

- LLM이 자연어 문자열로 직접 응답 → result.final_output = 문자열

Case 2: output_type 지정 (구조화된 출력)

- Pydantic 모델 지정 → LLM 응답이 모델에 맞으면 구조화된 객체 반환

```
class WeatherResult(BaseModel):
```

```
    city: str; temperature: float; condition: str
```

```
agent = Agent(output_type=WeatherResult, tools=[get_weather])
```

Case 3: 핸드오프 + output_type

- 분류(Triage) 에이전트 → 전문 에이전트로 핸드오프
- 전문 에이전트에 output_type 설정 → 구조화된 결과 반환

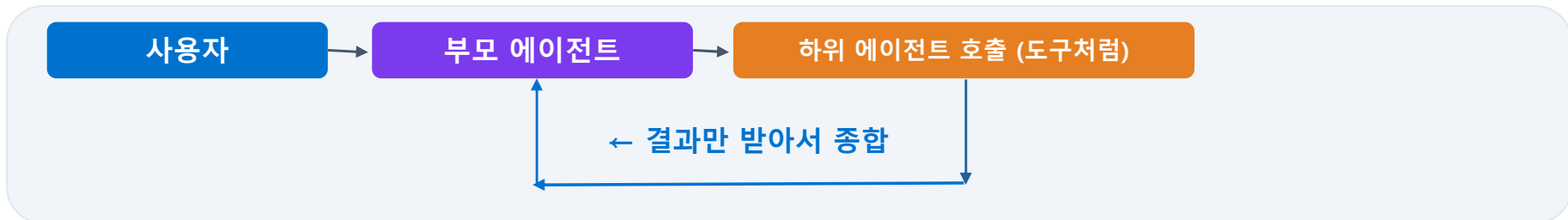
agent.as_tool() vs Handoff

Handoff - 제어권 완전 이전



예: 고객센터에서 환불 담당자에게 연결해주는 것

agent.as_tool() - 부모가 제어권 유지



예: 부모가 요약도 시키고 번역도 시킨 뒤 결과를 모아서 사용자에게 답변

실습 : 01_Agent_Overview

01. OpenAI Agent (Agents Overview)

OpenAI 플랫폼을 사용하여 사용자를 대신해 행동(예: 컴퓨터 제어 등)을 수행할 수 있는 에이전트(**Agent**)를 구축할 수 있습니다. Python용 **Agent SDK**를 사용하면 백엔드에서 이러한 에이전트의 오케스트레이션(조율) 로직을 만들 수 있습니다.

OpenAI Agents SDK는 에이전트 기반 AI 애플리케이션을 개발할 수 있게 해주는 도구입니다.

주요 기능 요약:

- **에이전트 루프 (Agent Loop):**
도구 실행 → 결과 전달 → LLM 호출 → 반복 실행 → 완료까지 자동 처리
- **핸드오프 (Handoffs):**
여러 에이전트 간의 협업과 위임을 유연하게 처리 가능
- **가드레일 (Guardrails):**
에이전트 입력을 사전 검사/검증하여, 조건을 만족하지 않으면 조기 종료 가능
- **함수 기반 도구 (Function Tools):**
Python 함수 하나를 자동으로 에이전트 도구로 변환,
Pydantic 기반 스키마 자동 생성 및 검증 포함

Runner 3종 비교

Runner 실행 방식 비교표

<code>Runner.run()</code>	await (비동기)	에이전트 루프 자동 실행, 도구/핸드오프 지원	Jupyter, FastAPI
<code>Runner.run_sync()</code>	동기 호출 (await 불필요)	내부적으로 이벤트 루프 실행	스크립트, 테스트
<code>Runner.run_streamed()</code>	await 없이 호출 + async for loop	토큰 단위 실시간 출력	채팅 UI

주의: `run_streamed()`는 코루틴이 아니므로 호출에 `await`를 붙이지 않음
— 반환된 `RunResultStreaming`의 `stream_events()`를 `async for`로 순회

코드 예시

```
# 비동기 실행
result = await Runner.run(starting_agent=agent, input="질문")

# 동기 실행
result = Runner.run_sync(agent, "질문")

# 스트리밍 실행 - 호출에는 await 없음
result = Runner.run_streamed(agent, "질문")
async for event in result.stream_events():
    print(event) # 토큰/이벤트 단위 실시간 처리
```

동기/비동기 실행 비교

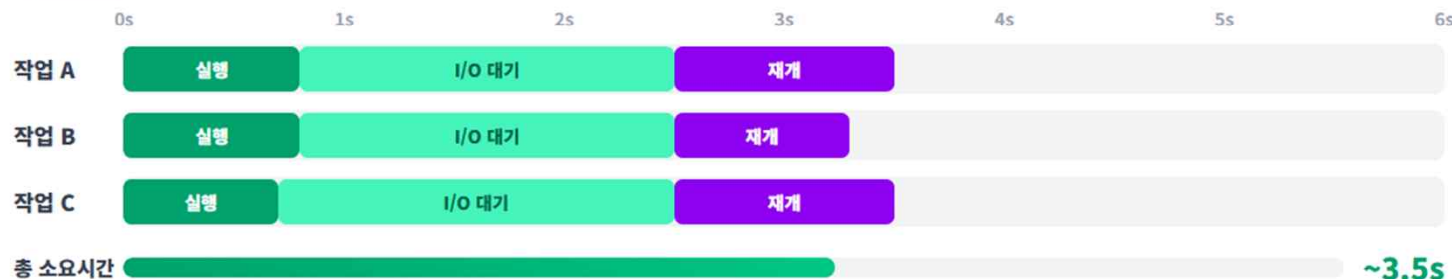
SYNC 동기 실행 — 순차적, 블로킹



● 동기 (run_sync)

- 작업이 끝날 때까지 다음 작업 시작 불가
- 코드가 단순하고 직관적
- 이벤트 루프 불필요
- 일반 .py 스크립트에서 사용

ASYNV 비동기 실행 — 동시, 논블로킹



● 비동기 (run)

- I/O 대기 중 다른 작업 처리 가능
- async/await 문법 필요
- 이벤트 루프 위에서 동작
- Jupyter / 웹앱 / 서버에서 사용

왜 Agents SDK 에서 비동기(async)가 필요한가

1 멀티에이전트 병렬 실행

여러 에이전트의 LLM 호출을 동시에 실행
asyncio.gather() → 3개 순차 6초를 ~2초로 단축

2 SDK 내부 동작이 이미 병렬

Input Guardrail이 본 실행과 동시에 검사 (tripwire)
한 턴의 여러 tool call도 병렬 실행 — 이벤트 루프 필수

3 스트리밍 (run_streamed)

async for event in result.stream_events()
토큰 단위 실시간 출력은 비동기 구조에서만 가능

4 실행 환경 (배포 시 필수)

FastAPI·웹앱은 이벤트 루프 위에서 동작 → await 필수
Jupyter에서 run_sync 사용 시 event loop 에러 발생

```
# 3개 에이전트 병렬 실행 — 총 소요시간: 가장 느린 호출 기준
res1, res2, res3 = await asyncio.gather(
    Runner.run(research_agent, q),
    Runner.run(translate_agent, q),
    Runner.run(summary_agent, q))
```

한 줄 정리

run_sync는 학습용 API,
run(async)는 실무용 API.
LLM 호출 = I/O 대기의 연속

02. Agent와 Runner

Runner는 에이전트 루프를 돌리는 실행기입니다.

- 동기: `Runner.run_sync(agent, input)` — 스크립트/테스트에 적합
- 비동기: `await Runner.run(agent, input)` — Jupyter/비동기 앱에 적합

에이전트 루프 동작:

1. LLM 호출 (에이전트 설정 + 메시지 히스토리)
2. 응답에 툴 호출이 있으면 실행 후 결과를 메시지에 추가하고 1번으로
3. 핸드오프가 있으면 대상 에이전트로 바꾸고 1번으로
4. **final_output**이 나오면 루프 종료 후 반환
5. `max_turns` 로 최대 턴 수 제한 가능

동기 vs 비동기 핸들러

SYNC

동기 핸들러 — 이벤트 루프 블로킹

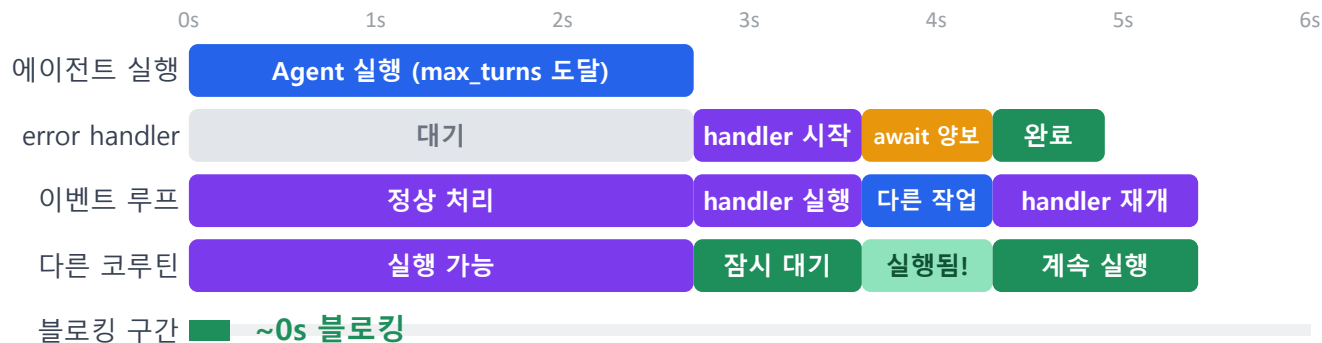


● 동기 핸들러 (def)

- 실행 중 이벤트 루프 블로킹
- 다른 코루틴 실행 불가
- DB 조회, HTTP 요청 등 I/O 작업에 부적합
- 단순 문자열 반환 정도만 적합

ASYNC

비동기 핸들러 — 이벤트 루프 논블로킹



● 비동기 핸들러 (async def)

- await 시 이벤트 루프에 제어권 양보
- 다른 코루틴 동시 실행 가능
- DB 조회, HTTP 요청, 로깅 등 I/O 작업 가능
- 실무에서 권장

※ ~0s인 이유: async 핸들러는 await 시점마다 제어권을 이벤트 루프에 반납 → 루프를 점유하는 것은 await 사이의 짧은 동기 코드 조각($\mu s \sim ms$)뿐.

실습 : 03_error_handling

03. Error Handling

에이전트 루프에서 `max_turns`를 초과하면 기본적으로 `MaxTurnsExceeded` 예외가 발생하여 프로그램이 중단됩니다. `error_handlers`를 사용하면 예외 대신 제어된 최종 메시지를 반환하도록 할 수 있습니다.

핵심 개념

- `error_handlers={"max_turns": handler}`: `Runner.run()` 호출 시 전달하여, 턴 초과 시 핸들러 함수가 실행되도록 설정
- `RunErrorHandlerResult`: 핸들러가 반환하는 결과 객체로, `final_output` (폴백 메시지)과 `include_in_history` 옵션을 포함
- `include_in_history=False`: 폴백 메시지를 대화 기록에 남기지 않음 (사용자 안내 전용)
- 동기(`def`) vs 비동기(`async def`) 핸들러: 동기 핸들러는 실행 중 이벤트 루프를 블로킹하고, 비동기 핸들러는 `await` 시 제어권을 양보 → I/O 작업에는 비동기 권장

학습 내용

1. `max_turns` 초과 시 `error_handlers` 로 예외 없이 안전하게 처리하기
2. 비동기(`async def`) / 동기(`def`) 핸들러 작성법
3. `include_in_history` 옵션의 역할과 활용
4. 동기 vs 비동기 핸들러의 이벤트 루프 블로킹 차이 관찰 실험 (틱 간격 측정)

on_handoff 콜백 (1/2) — 개념

핸드오프 발생 시 자동 실행되는 콜백

- **실행 시점**
 - LLM이 핸드오프 도구(transfer_to_<agent>)를 호출하는 순간, 제어권 이전 직전에 실행
- **용도 — 사이드 이펙트 처리**
 - 로깅, 알림 발송, 상태 업데이트, 다음 에이전트를 위한 데이터 준비



콜백은 ①과 ② 사이에서 실행 — 대상 에이전트가 응답을 만들기 전

핸드오프는 LLM에게 transfer_to_<agent_name> 이름의 도구로 노출되며, on_handoff는 그 도구가 호출되는 순간의 훅(hook)이다.

on_handoff 콜백 (2/2) — 코드

콜백 등록과 실행 확인

```
# 1. 콜백 정의 - 핸드오프 순간 자동 호출
def on_billing_handoff(ctx: RunContextWrapper[None]):
    print("[로그] Billing Agent로 핸드오프 발생")
def on_refund_handoff(ctx: RunContextWrapper[None]):
    print("[로그] Refund Agent로 핸드오프 발생")

# 2. handoff()의 on_handoff 파라미터로 등록
triage_agent = Agent(
    name="Triage_agent", ...,
    handoffs=[
        handoff(billing_agent, on_handoff=on_billing_handoff),
        handoff(refund_agent, on_handoff=on_refund_handoff)])

# 3. 실행 - 콜백 로그 → 응답 에이전트 확인
result = await Runner.run(triage_agent, "환불을 받고 싶습니다.")
print(result.last_agent.name)
```

실행 결과

[로그] Refund Agent로 핸드오프 발생

input_type — 핸드오프 입력 데이터

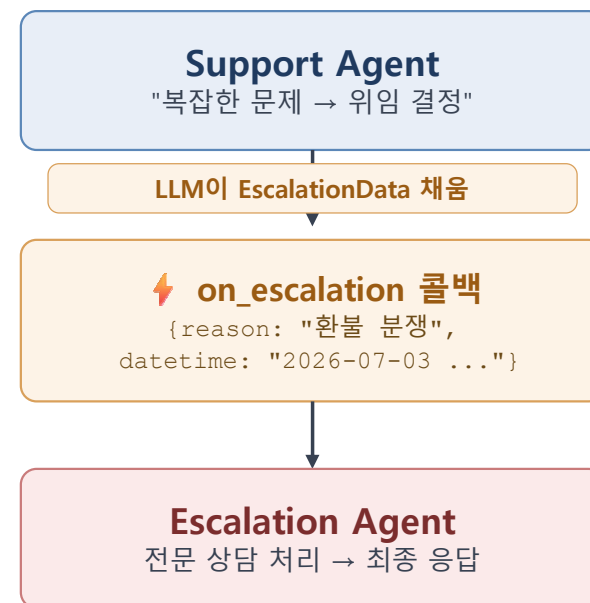
핸드오프 시 LLM이 구조화된 데이터를 함께 전달

input_type=Pydantic 모델을 지정하면, 핸드오프 도구의 인자 스키마가 강제됨 → LLM이 이유·시각 등 필수 정보를 채워서 위임 (예: 에스컬레이션 사유 로깅)

```
# 1. 전달받을 데이터 구조 정의 (Pydantic)
class EscalationData(BaseModel):
    reason: str      # 에스컬레이션 사유
    datetime: str    # 접수 시간

# 2. 콜백 — 두 번째 인자로 input_data 수신 (async def)
async def on_escalation(ctx, input_data: EscalationData):
    print(f"[에스컬레이션] {input_data.reason}, {input_data.datetime}")

# 3. handoff()에 콜백 + 데이터 타입 등록
support_agent = Agent(..., handoffs=[
    handoff(escalation_agent,
            on_handoff=on_escalation,
            input_type=EscalationData)])
```



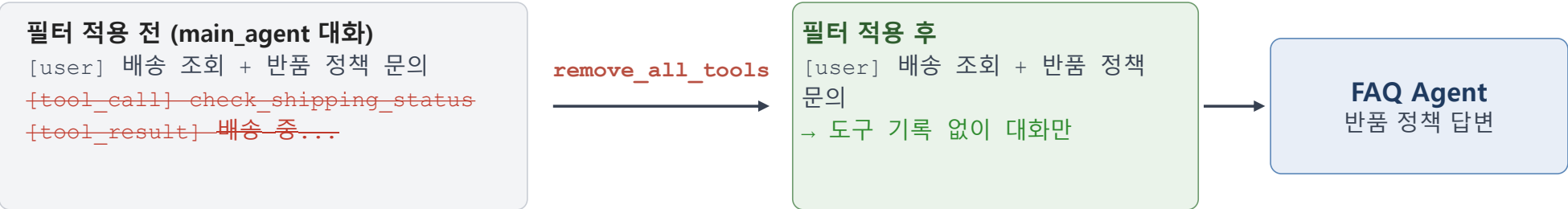
input_type을 쓰면 콜백은 async def + 두 번째 인자 input_data 필수 — 시그니처가 (ctx)에서 (ctx, input_data)로 바뀜

input_filter — 대화 기록 필터링

다음 에이전트에 전달되는 히스토리 제어

기본 동작: 핸드오프 시 새 에이전트는 이전 대화 기록 전체(도구 호출/결과 포함)를 그대로 이어받음 → input_filter로 가공 가능

내장 필터 (agents.extensions.handoff_filters)	동작	사용 예
remove_all_tools	도구 호출/결과 메시지를 모두 제거	FAQ 에이전트에 깔끔한 대화만 전달
nest_handoff_history	이전 대화 전체를 단일 assistant 메시지로 압축	요약 에이전트에 컨텍스트만 전달



```
handoff(faq_agent, input_filter=handoff_filters.remove_all_tools) # 함수 참조 전달 (괄호 없음)
```

RECOMMENDED_PROMPT_PREFIX — 권장 프롬프트

핸드오프 지시문을 프롬프트에 자동 삽입

문제 : LLM은 '핸드오프'라는 개념을 기본적으로 모름 → 핸드오프 도구가 있어도 소극적으로 사용

해결 : "너는 다른 에이전트에게 작업을 위임할 수 있다"는 시스템 컨텍스트를 instructions 앞에 삽입

방법 1: 문자열 상수를 프롬프트 앞에 연결

```
agent = Agent(  
    instructions=RECOMMENDED_PROMPT_PREFIX + "당신은 고객 지원...",  
    handoffs=[refund_agent, order_agent])
```

방법 2: 함수로 래핑 (기존 프롬프트에 자동 추가)

```
agent = Agent(  
    instructions=prompt_with_handoff_instructions("..."),  
    handoffs=[refund_agent, order_agent])
```

```
import from agents.extensions.handoff_prompt import RECOMMENDED_PROMPT_PREFIX,  
prompt_with_handoff_instructions
```

PREFIX에 담긴 내용

- "당신은 Agents SDK 멀티 에이전트 시스템의 일부"
- 핸드오프는 transfer_to_<agent> 함수 호출로 이루어짐
- 에이전트 간 전환을 사용자에게 드러내지 말 것

04. Handoffs (핸드오프 심화)

핸드오프(**Handoff**) 는 에이전트가 특정 작업을 다른 에이전트에게 위임하는 기능입니다.

핸드오프는 LLM에게 도구(**tool**) 로 표현됩니다.

예) Refund Agent 로의 핸드오프 → LLM 도구 이름: `transfer_to_refund_agent`

방식	예시	특징
Agent 직접 전달	<code>handoffs=[billing_agent]</code>	간단, 기본 동작
<code>handoff()</code> 함수	<code>handoffs=[handoff(billing_agent, on_handoff=...)]</code>	콜백, 이름 재정의, 입력 필터 등 고급 옵션

`handoff()` 함수의 주요 파라미터:

파라미터	설명
<code>agent</code>	위임할 대상 에이전트
<code>tool_name_override</code>	LLM에게 노출되는 도구 이름 재정의 (기본: <code>transfer_to_<agent_name></code>)
<code>tool_description_override</code>	도구 설명 재정의
<code>on_handoff</code>	핸드오프 호출 시 실행되는 콜백 함수
<code>input_type</code>	LLM이 핸드오프 시 함께 전달할 데이터 타입 (Pydantic 모델)
<code>input_filter</code>	다음 에이전트에게 전달되는 대화 기록 필터링 함수
<code>is_enabled</code>	핸드오프 활성화 여부 (bool 또는 런타임 함수)

Session — 개념과 필요성

멀티턴 대화 기록의 자동 관리

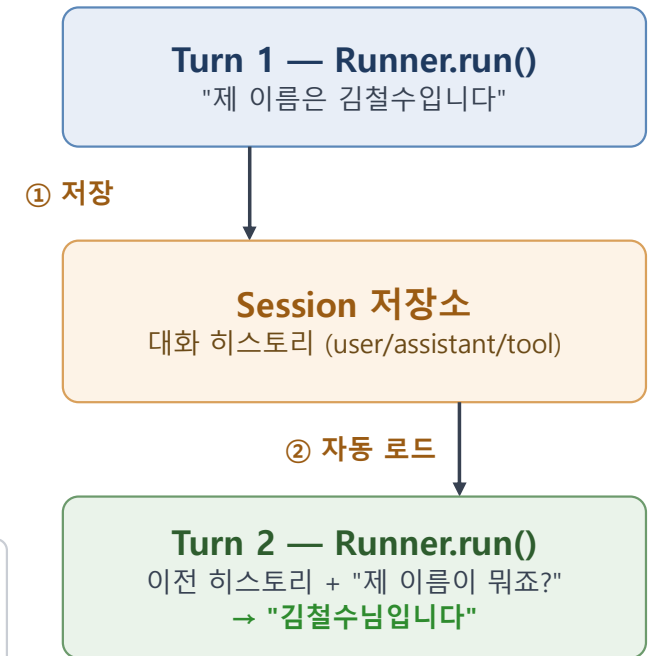
- **문제 — LLM 호출은 상태 비저장(stateless)**

- Runner.run()은 매번 독립 실행 → 이전 턴의 대화를 기억하지 못함
- 수동 해법: result.to_input_list()를 다음 입력에 직접 이어붙이기
→ 코드 번거로움, 누락·중복 위험

- **해결 — Session: 대화 기록 저장소**

- 실행 전: 저장된 히스토리를 자동 로드해 입력 앞에 결합
- 실행 후: 이번 턴의 user/assistant/tool 아이템을 자동 저장
- 같은 session_id로 실행하면 같은 대화를 계속 이어감

```
session = SQLiteSession("user_123")
await Runner.run(agent, "제 이름은 김철수입니다", session=session)
await Runner.run(agent, "제 이름이 뭐죠?", session=session)
# → "김철수님입니다" - 이전 턴을 기억
```



매 턴 반복: 로드 → 실행 → 저장

핵심: Session은 Runner 바깥에 있는 대화 메모리 — 어디에 저장하느냐에 따라 다음 슬라이드의 10종 구현으로 나뉜다

Session 유형 비교

SDK 제공 Session 10종

- **SQLiteSession** : 내장, 인메모리/파일 (개발/프로토타입)
- **AsyncSQLiteSession** : 비동기 전용 SQLite
- **AdvancedSQLiteSession** : 브랜치, 토큰 분석 (고급)
- **SQLAlchemySession** : PostgreSQL, MySQL (프로덕션)
- **RedisSession** : Redis 분산 저장 (대규모 서비스)
- **MongoDBSession** : MongoDB 문서 저장 (NoSQL 스택)
- **DaprSession** : Dapr state store 추상화 — 백엔드 교체 가능 (K8s/클라우드)
- **EncryptedSession** : 암호화 래퍼 (보안 필수)
- **OpenAIConversationsSession** : OpenAI 서버 저장
- **CustomSession** : 사용자 정의 (Django 등)

실습 : 05_Sessions

05. Sessions

Session은 여러 번의 `Runner.run` 호출에 걸쳐 대화 히스토리를 자동으로 유지하는 기능입니다.

- 세션을 쓰지 않으면 매 턴마다 이전 메시지 목록을 직접 구성해 넘겨야 합니다.
- `session` 을 넘기면 SDK가 대화를 저장소에 저장/불러와서, 같은 `session` 으로 실행할 때마다 맥락이 이어집니다.

활용: 채팅봇, 고객지원 에이전트 등 다회차 대화.

SDK 제공 Session 유형 (10종) ¶

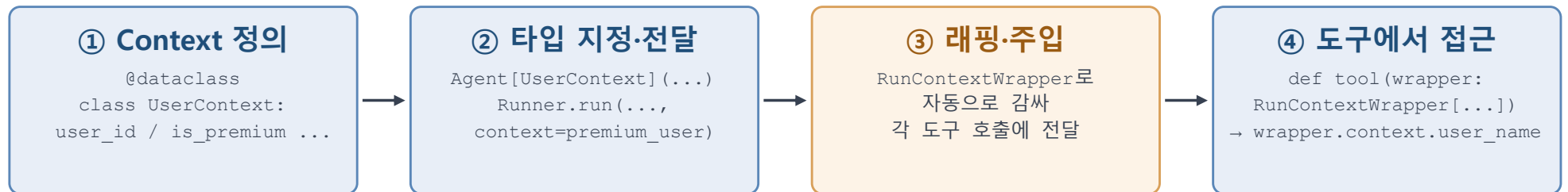
세션	위치	특징	용도	본 노트북 실습
<code>SQLiteSession</code>	<code>agents</code>	기본 내장, 인메모리/파일 저장	개발/프로토타입	✓
<code>AsyncSQLiteSession</code>	<code>agents.extensions.memory</code>	비동기 전용 SQLite	비동기 환경	✓
<code>AdvancedSQLiteSession</code>	<code>agents.extensions.memory</code>	브랜치, 토큰 사용량 분석	고급 분석 필요 시	✓
<code>SQLAlchemySession</code>	<code>agents.extensions.memory</code>	PostgreSQL, MySQL 등 지원	프로덕션 DB	✓
<code>EncryptedSession</code>	<code>agents.extensions.memory</code>	다른 세션을 암호화로 래핑	보안 필요 시	✓
<code>OpenAIConversationsSession</code>	<code>agents</code>	OpenAI 서버에 대화 저장	OpenAI 플랫폼 활용	✓
<code>CustomSession</code>	직접 구현	<code>Session</code> 프로토콜 구현	Django, MongoDB 등 연동	✓
<code>RedisSession</code>	<code>agents.extensions.memory</code>	Redis 기반 분산 저장	대규모 서비스	✗ (Redis 서버 필요)
<code>MongoDBSession</code>	<code>agents.extensions.memory</code>	MongoDB 문서 저장	NoSQL 스택	✗ (MongoDB 필요)
<code>DaprSession</code>	<code>agents.extensions.memory</code>	Dapr state store 추상화	K8s/클라우드	✗ (Dapr 사이드카 필요)

기본 컨텍스트 — RunContextWrapper

용도 — 도구 함수에 **사용자 정보(user_id·권한), DB 연결, API 키, 설정값 등 실행마다 달라지는 데이터**를 효율적으로 전달

의존성 주입 흐름

 사용자가 작성 SDK가 자동 처리



- 컨텍스트는 LLM에 전송되지 않는다(Local Context).
- 도구가 필요한 정보를 컨텍스트에서 직접 꺼내므로, API 키·DB 연결·개인정보가 프롬프트에 노출되지 않고 토큰도 소모하지 않는다.

활용: 사용자별 개인화(같은 에이전트 + 다른 컨텍스트) · DB/API 클라이언트 주입 · 요청별 설정 전달

ToolContext — 도구 메타데이터 접근

용도 - "지금 어떤 도구가, 어떤 인자로 호출됐는가"를 도구 안에서 알아야 할 때
— 로깅, 감사 추적(audit trail), 디버깅 등

```
from agents.tool_context import ToolContext

@function_tool
async def debug_tool(ctx: ToolContext[UserContext], query: str):
    print(ctx.tool_name)          # "debug_tool"
    print(ctx.tool_call_id)       # "call_abc123"
    print(ctx.tool_arguments)     # '{"query": ...}'
    print(ctx.context.user_name)  # 기존 접근 그대로

# 에이전트/실행은 동일 — 시그니처만 교체
debug_agent = Agent[UserContext](..., tools=[debug_tool])
await Runner.run(debug_agent, "...", context=premium_user)
```

상속 관계 :
ToolContext 는
RunContextWrapper 상속
+ 도구 메타데이터 3 가지 추가

- **추가로 얻는 정보**

- tool_name — 호출된 도구 이름
- tool_call_id — 이번 호출의 고유 ID
- tool_arguments — LLM이 넘긴 원본 인자(JSON)

활용: "누가.언제.어떤 인자로 도구를 호출했나" → 감사 로그 적재, 도구 공용 로깅 미들웨어, tool_call_id 기반 요청 추적

06. Context Management (컨텍스트 관리)

도구 함수에 사용자 정보(user_id·권한), DB 연결, API 키, 설정값 등 실행마다 달라지는 데이터를 효율적으로 전달하는 방법입니다. `RunContextWrapper` 를 사용한 의존성 주입(Dependency Injection) 방식입니다.

의존성 주입 흐름 (4단계)

단계	하는 일	코드	누가
① Context 정의	전달할 데이터 클래스 작성	<code>@dataclass class UserContext: ...</code>	사용자 작성
② 타입 지정·전달	에이전트에 타입 지정, 실행 시 전달	<code>Agent[UserContext](...), Runner.run(..., context=premium_user)</code>	사용자 작성
③ 래핑·주입	<code>RunContextWrapper</code> 로 감싸 각 도구 호출에 전달 (자동)		SDK가 자동 처리
④ 도구에서 접근	도구 첫 인자로 받아 꺼내 사용	<code>def tool(wrapper: RunContextWrapper[...]) → wrapper.context.user_name</code>	사용자 작성

중요: 컨텍스트는 LLM에 전송되지 않습니다! (Local Context)

- 도구가 필요한 정보를 컨텍스트에서 직접 꺼내므로, API 키·DB 연결·개인정보가 프롬프트에 노출되지 않고 토큰도 소모하지 않습니다.

가드레일 (Guardrails) — 개요

에이전트의 입력과 출력을 검증하는 안전장치

에이전트와 함께 실행되며, **입력(Input)**과 **출력(Output)**에 대해 검사·유효성 검증 수행조건 위반 시 **tripwire(즉시 중단 신호)** 발동

구분	Input Guardrail	Output Guardrail
검증 시점	에이전트 실행 전	에이전트 실행 후
검증 대상	사용자 입력	에이전트 출력
데코레이터	@input_guardrail	@output_guardrail
발동 예외	InputGuardrailTripwireTriggered	OutputGuardrailTripwireTriggered
실행 모드	병렬(Parallel) 또는 차단(Blocking)	항상 순차(Sequential) — 응답 생성 후 검사

Input Guardrail — 두 가지 구현 방식

공통 구조 — 모든 Input Guardrail은 같은 시그니처의 함수

```
@input_guardrail
async def guard(ctx, agent, input_data):  # input_data: str 또는 메시지 리스트
    # guardrail 검사 로직 (에이전트 실행 전)
    return GuardrailFunctionOutput(output_info=..., tripwire_triggered=bool)

trriage = Agent(..., input_guardrails=[guard])  # 메인 에이전트 실행 전 사용자 입력 검사
```

스타일 1 — 데코레이터 사용

`@input_guardrail` 만 붙이면 함수가 곧바로 `InputGuardrail` 인스턴스가 됨

`@input_guardrail(name=..., run_in_parallel=False)`
→ 데코레이터 인자로 옵션 지정 가능

- 코드가 짧고 간결 — 함수만 리스트에 전달
- 노트북 예제(homework_guardrail)에서 사용한 방식

스타일 2 — 수동 래핑

`InputGuardrail(guardrail_function=함수)` 로 직접 감싸서 `input_guardrails` 에 전달

- "가드레일 = 함수 래퍼" 구조가 코드에 명시적
- 내부적으로 두 방식은 완전히 동일하게 동작

Output Guardrail — 두 가지 구현 방식

공통 구조 — 모든 Output Guardrail은 같은 시그니처의 함수

```
@output_guardrail
async def guard(ctx, agent, output):
    # guardrail 검사 logic
    return GuardrailFunctionOutput(output_info=..., tripwire_triggered=bool)

assistant = Agent(..., output_guardrails=[guard]) # 응답 생성 후 항상 순차 실행
```

방식 1 — 정규식 기반

pii_guardrail — 이메일·전화번호 패턴 탐지
re.search(email/phone) → has_pii
→ tripwire_triggered=has_pii

- 빠름·결정적·추가 비용 없음
- 정의한 패턴만 탐지 — 의미는 이해 못 함

방식 2 — LLM 기반

llm_content_guardrail — 검증 전용 에이전트 활용
checker_agent(output_type=GuardrailCheckResult)
→ tripwire = not is_appropriate

- 문맥·의미 이해, 유연한 기준 적용
- LLM 호출 1회 추가 — 지연·비용 발생

07. Guardrails (가드레일)

가드레일(Guardrails) 은 에이전트와 함께 실행되며, 입력과 출력에 대해 검사와 유효성 검증을 수행할 수 있게 해줍니다. 조건을 만족하지 않으면 트립와이어 (tripwire) 를 발동시켜 실행을 조기에 중단할 수 있습니다.

이 노트북에서는 두 가지 가드레일을 순서대로 다룹니다.

1. **Input Guardrail** (`@input_guardrail`) — 에이전트 실행 전 사용자 입력을 검증
2. **Output Guardrail** (`@output_guardrail`) — 에이전트 실행 후 생성된 출력을 검증

Input vs Output Guardrail 비교

구분	Input Guardrail	Output Guardrail
검증 시점	에이전트 실행 전	에이전트 실행 후
검증 대상	사용자 입력	에이전트 출력
데코레이터	<code>@input_guardrail</code>	<code>@output_guardrail</code>
예외	<code>InputGuardrailTripwireTriggered</code>	<code>OutputGuardrailTripwireTriggered</code>
실행 모드	Parallel(병렬) 또는 Blocking(차단)	항상 Sequential(순차)

Tripwire (트립와이어)란?

가드레일이 문제를 감지했을 때 발생시키는 즉시 중단 신호입니다. `tripwire_triggered=True` 로 설정하면 즉시 예외가 발생하고 실행이 중단됩니다.

다양한 시각화 - draw_graph()

draw_graph() 사용법

단일 에이전트 + 도구

```
from agents.extensions.visualization import draw_graph
draw_graph(weather_agent) # Jupyter 인라인 표시
```

멀티에이전트 (핸드오프)

```
triage = Agent(name="분류", handoffs=[ko_agent, en_agent])
draw_graph(triage)
```

파일 저장 & 브라우저

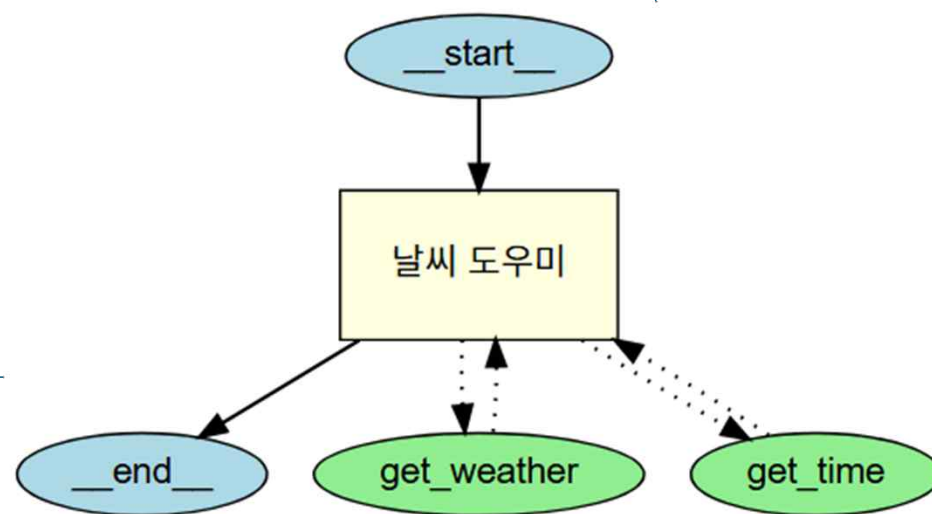
```
draw_graph(agent, filename="graph") # PDF 저장
draw_graph(agent).view()           # 뷰어에서 열기
```

08. Agent Visualization (에이전트 시각화)

OpenAI Agent SDK는 **Graphviz**를 사용하여 에이전트 구조를 시각적으로 표현하는 기능을 제공합니다.

시각화가 보여주는 것:

- 노란색 박스: 에이전트 (Agent)
- 녹색 타원: 도구 (Tool)
- 회색 박스: MCP 서버
- 실선 화살표: 핸드오프 연결
- 점선 화살표: 도구 연결
- 대시선 화살표: MCP 서버 연결



커스텀 트레이스 & 그룹핑

여러 실행을 하나로 그룹핑

커스텀 트레이스: 여러 실행을 하나로

```
from agents import trace

with trace("요약 및 평가 워크플로우"):
    summary = await Runner.run(summarizer, text)
    eval_result = await Runner.run(critic, summary.final_output)
```



group_id로 세션별 트레이스 관리

```
with trace("고객 요청 처리", group_id="customer-session-abc123"):
    result = await Runner.run(math_agent, "...")
# 같은 group_id → 대시보드에서 묶어서 조회
```



9. Tracing (트레이싱)

에이전트가 어떤 단계를 거쳐 응답을 생성했는지 추적(Trace) 할 수 있습니다. 트레이싱은 디버깅, 모니터링, 성능 분석에 필수적인 기능입니다.

핵심 개념

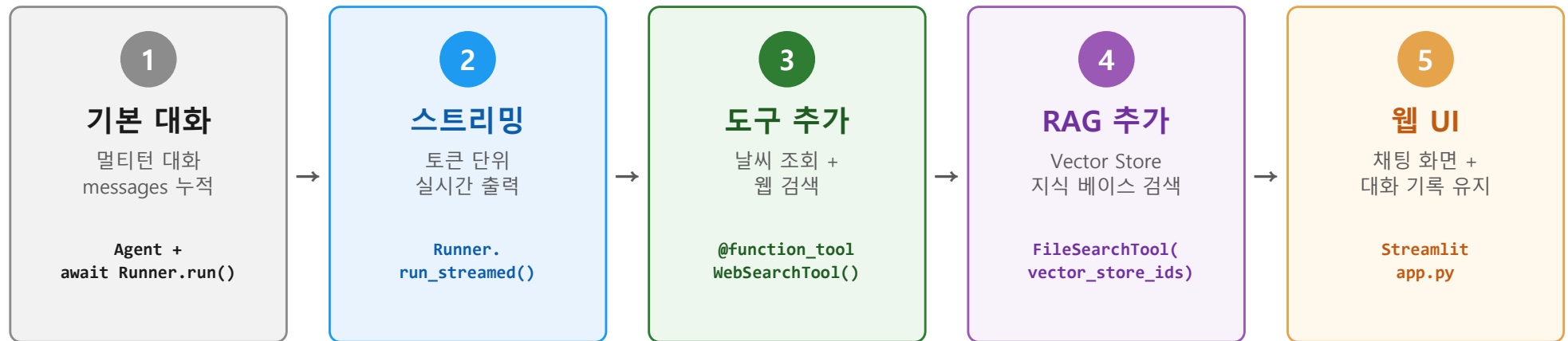
개념	설명
Trace	하나의 워크플로우 전체를 나타내는 단위
Span	Trace 내의 개별 작업 단위 (LLM 호출, 도구 실행 등)
자동 트레이싱	SDK가 자동으로 에이전트 실행, LLM 호출, 도구 호출을 추적
Custom Span	개발자가 직접 정의하는 추적 단위

자동으로 추적되는 항목

- `Runner.run()` 전체 실행 → **Trace**
- 에이전트 실행 → **Agent Span**
- LLM 호출 → **Generation Span**
- 도구 호출 → **Function Span**
- 가드레일 → **Guardrail Span**
- 핸드오프 → **Handoff Span**

Chatbot RAG Agent

도구 없는 단순 챗봇에서 시작해, 도구를 하나씩 붙여가며 RAG 에이전트를 완성



RAG란? : 검색 + 생성의 결합

검색 증강 생성(Retrieval-Augmented Generation) — 외부 지식 베이스를 검색해 LLM 응답의 **정확성과 최신성**을 보완하는 기술

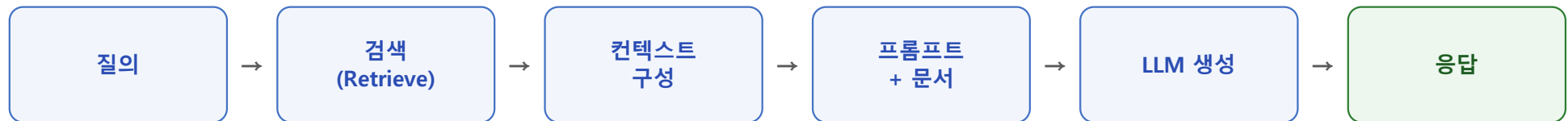
검색기 (Retriever)

사용자 입력과 관련된 정보를 외부 데이터베이스에서 검색

생성기 (Generator)

검색된 문서를 컨텍스트로 활용해 사용자에게 응답을 생성

실행 흐름



- 문서 로드 → 청킹 → 임베딩 → 저장 → 검색 파이프라인을 **OpenAI 관리형 Vector Store**가 자동 처리
- 파일 업로드와 **FileSearchTool** 연결만으로 RAG가 완성됩니다.

Vector Store란? — 의미로 검색하는 데이터베이스

텍스트를 **임베딩 벡터**(의미를 담은 숫자 배열)로 변환해 저장하고, **벡터 간 거리(유사도)**로 관련 문서를 찾는 데이터베이스

"환불 정책이 어떻게 되나요?"

질의 / 문서 텍스트



임베딩 모델

text-embedding-3 등



[0.21, -0.73, 0.55, ..., 1.42]

고차원 벡터 (수백~수천 차원)



유사도 검색

질의 벡터와 가까운 문서 벡터 탐색

벡터 공간 — 의미가 비슷하면 거리도 가깝다



유사도 상위 = 검색 결과

키워드 검색 — 단어 일치 기반

- "환불" ≠ "돈 돌려받기" — 단어가 다르면 못 찾음
- 동의어·표현 변형·오타에 취약

벡터(의미) 검색 — 임베딩 유사도 기반

- 표현이 달라도 의미가 같으면 매칭 (코사인 유사도 등)
- 전용 DB: Chroma·FAISS·Pinecone 등 — 이 과정은 OpenAI 관리형 사용

RAG 도구 추가 — OpenAI 관리형 Vector Store

초기 설정 (최초 1회): 같은 이름의 Vector Store가 있으면 재사용, 없으면 생성 후 일괄 업로드

```
VS_NAME = "notebook-faq-kb"
existing = [vs for vs in client.vector_stores.list() if vs.name == VS_NAME]

if existing:
    vs_id = existing[0].id # 기존 Vector Store 재사용
else:
    vs_id = client.vector_stores.create(name=VS_NAME).id
    batch = client.vector_stores.file_batches.upload_and_poll(
        vector_store_id=vs_id,
        files=[open(DATA_DIR / "faqs.json", "rb"),
               open(DATA_DIR / "knowledgeBase.json", "rb")]) # 업로드 + 인덱싱 완료 대기
```



에이전트 연결: `FileSearchTool(vector_store_ids=[vs_id])` 를 tools에 추가하면 RAG 완성

10. Chatbot RAG Agent 구현

OpenAI Agents SDK로 채팅 에이전트를 단계별로 확장합니다. 도구 없는 단순 챗봇에서 시작해 날씨/웹검색/RAG 도구를 하나씩 붙여가며 완성합니다.

- | | |
|-----------|----------------------------------|
| 1. 기본 대화 | Agent + Runner.run() |
| ↓ | |
| 2. 스트리밍 | Runner.run_streamed() + 토큰 단위 출력 |
| ↓ | |
| 3. 도구 추가 | + get_weather + WebSearchTool |
| ↓ | |
| 4. RAG 추가 | + FileSearchTool (Vector Store) |
| ↓ | |
| 5. 웹 UI | → Streamlit app.py |

단계	내용	핵심 API / 도구
1. 비동기 방식 호출	도구 없이 Agent + Runner로 대화 구현 (메시지 유지)	<code>await Runner.run()</code>
2. Streaming 방식 호출	동일 에이전트를 토큰 단위 실시간 출력으로 실행	<code>Runner.run_streamed()</code> , <code>ResponseTextDeltaEvent</code>
3. 도구를 추가한 Chatbot	날씨 조회 + 웹 검색 도구를 에이전트에 연결	<code>@function_tool</code> <code>get_weather</code> , <code>WebSearchTool()</code>
4. RAG 도구 추가	OpenAI Vector Store에 FAQ/KB 업로드 후 검색 도구 연결	<code>FileSearchTool(vector_store_ids=[...])</code>
5. 웹 UI 연결	동일 에이전트를 Streamlit 웹 앱으로 구현한 예제 안내	<code>10_chat-app-rag/app.py</code>

최종 에이전트 구성 + 웹 UI 연결

도구 3개를 통합한 최종 에이전트

```
agent = Agent(  
    model=Model,  
    name="여행·지식 베이스 에이전트",  
    instructions="날씨는 get_weather, 최신 정보는"  
        " 웹 검색, FAQ는 file_search로 답변",  
    tools=[  
        get_weather,                # ① 날씨  
        WebSearchTool(),            # ② 웹 검색  
        FileSearchTool(vector_store_ids=[vs_id]), # ③ RAG  
    ],  
)
```

여행·지식 베이스 에이전트

get_weather 날씨 조회 (open-meteo)

WebSearchTool 최신 정보 웹 검색

FileSearchTool FAQ 지식 베이스 검색 (RAG)

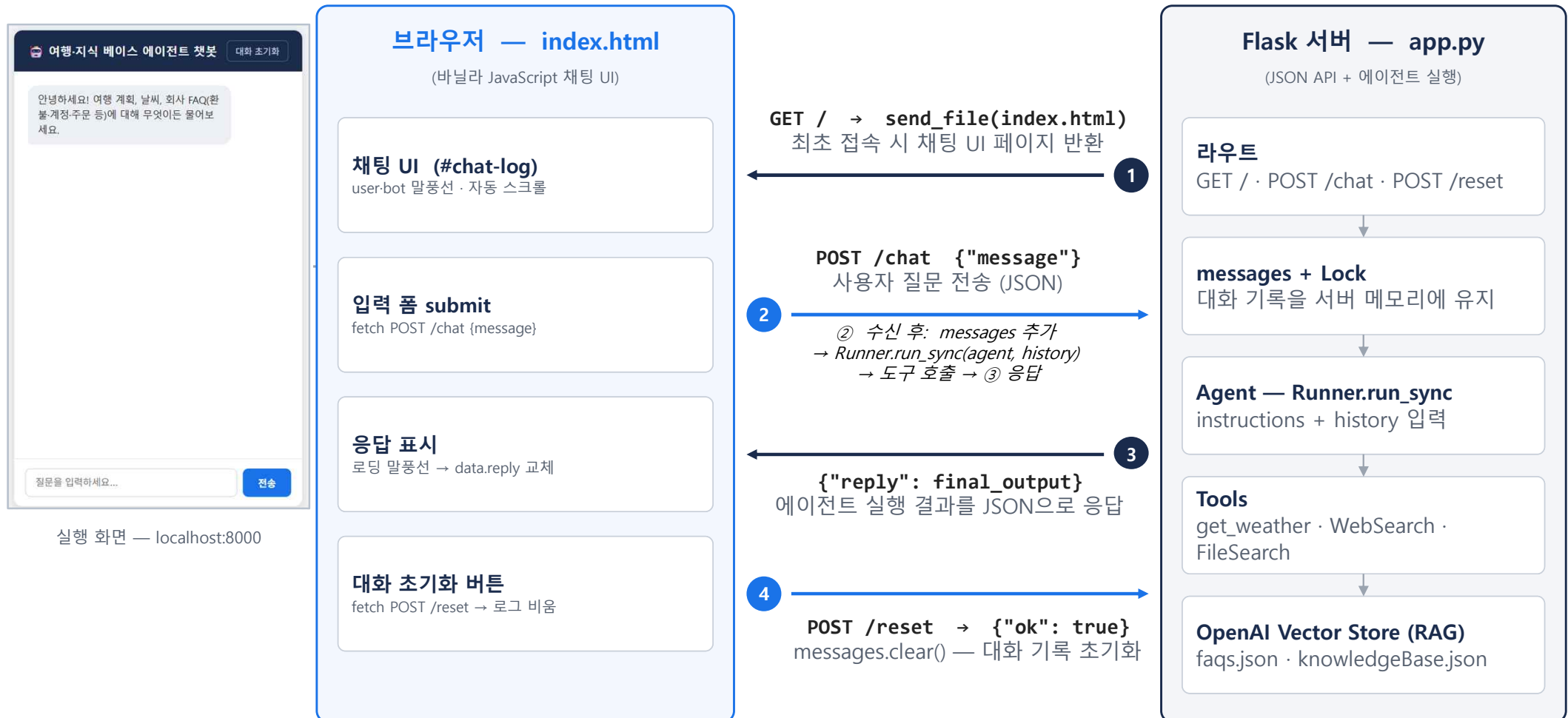
웹 UI 연결: 동일한 에이전트 구조를 Flask + HTML 앱으로 구현한 예제

— **11_rag_chat_flask/app.py** (실행: `python app.py` → `http://localhost:8000` 접속)

What is Flask ?

- Python으로 작성된 경량(micro) 웹 애플리케이션 프레임워크
 - 라우팅·요청·응답 등 웹 서버의 핵심 기능을 데코레이터(@app.route)로 간결하게 정의
 - JSON API를 쉽게 만들 수 있어 HTML/JavaScript 프론트엔드와 자유롭게 조합 가능
 - 템플릿(Jinja2), 세션, 확장 생태계(DB·인증 등)를 필요한 만큼만 선택해 사용
 - 코드 몇 줄로 서버 실행(app.run()) — 학습과 프로토타이핑에 적합
 - 소규모 API 서버부터 대규모 서비스까지 폭넓게 활용 (Django보다 가볍고 자유로움)
-

app.py ↔ index.html — Component 상호 작용 흐름



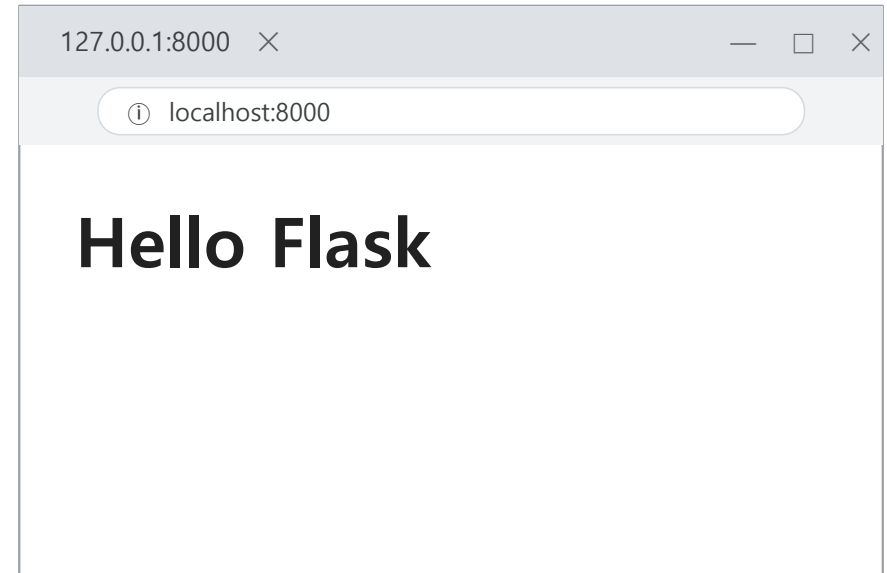
Flask 실행

- `python app.py`

```
app.py
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def home():
7      return "<h1>Hello Flask</h1>"
8
9  if __name__ == '__main__':
10     app.run(port=8000)
```



```
(base) C:\...\01_OpenAI_API_Basic\14_rag-chat-flask>python app.py
RAG 챗봇 서버 시작: http://localhost:8000 (종료: Ctrl+C)
* Serving Flask app 'app'
* Running on http://127.0.0.1:8000
```



실습 : 11_Rag_Chat_Flask_Agent — Flask + HTML

Hands-on

- **OpenAI Agents SDK** 에이전트를 웹으로 제공
- Flask가 JSON API를 담당하고, HTML(바닐라 JavaScript)이 채팅 UI를 담당

단계	내용	핵심 API / 도구
1. Vector Store 설정	서버 시작 시 1회 — 같은 이름(notebook-faq-kb) 있으면 재사용, 없으면 생성 후 FAQ/KB 업로드	<code>vector_stores.create()</code> <code>file_batches.upload_and_poll()</code>
2. 날씨 도구	위도·경도로 현재 기온 조회 (open-meteo API)	<code>@function_tool get_weather</code>
3. 에이전트 생성	서버 시작 시 1회 생성해 재사용 — 도구 3개 연결 (모델: gpt-5.4-mini)	<code>Agent, WebSearchTool(),</code> <code>FileSearchTool(vector_store_ids=[vs_id])</code>
4. 채팅 API	메시지를 대화 기록에 추가 후 전체 기록으로 에이전트 실행, 답변을 JSON으로 반환	<code>POST /chat, Runner.run_sync()</code>
5. 대화 상태 관리	서버 메모리 리스트 + Lock으로 보호 (st.session_state 대응) — /reset으로 초기화	<code>threading.Lock, POST /reset</code>

멀티에이전트 오케스트레이션 — 코드 기반 제어

LLM 기반(Handoffs) vs 코드 기반 오케스트레이션

구분	LLM 기반 (Handoffs)	코드 기반 (이 노트북)
제어 주체	LLM이 자율 판단	개발자가 코드로 제어
예측 가능성	낮음 (LLM 판단에 의존)	높음 (결정적 흐름)
비용 / 속도	변동적	예측 가능
적합한 경우	복잡한 대화, 분류가 어려운 경우	파이프라인, 배치 처리, 정해진 워크플로우

4가지 패턴의 Multi-Agent Orchestration

1. 순차 체이닝
A → B → C 파이프라인

2. 병렬 실행
asyncio.gather()

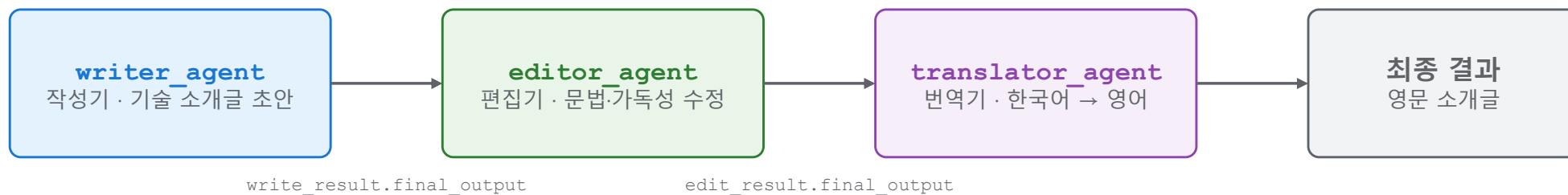
3. 분류 → 라우팅
Structured Output

4. 조합 패턴
병렬 + 순차 결합

패턴 1 — 순차 체이닝 (Sequential Chaining)

이전 단계의 `final_output`을 다음 에이전트의 입력으로 전달

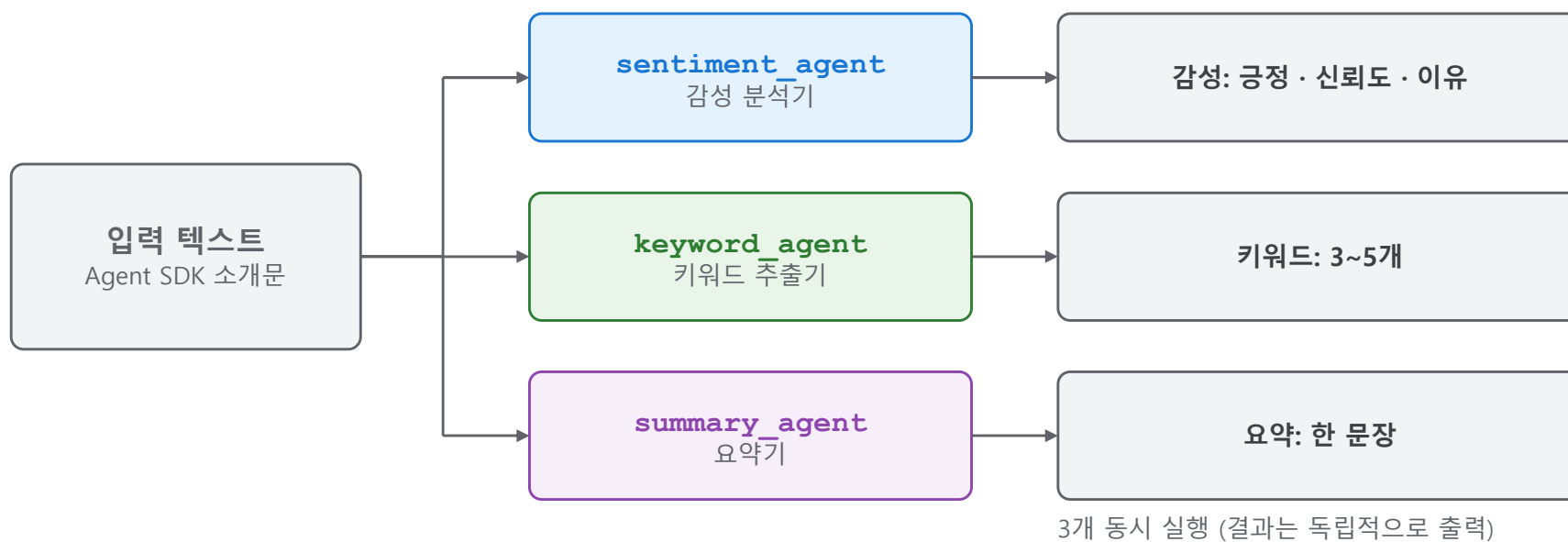
각 단계가 순서대로 실행 — `trace()`로 전체 파이프라인을 하나의 트레이스로 추적



패턴 2 — 병렬 실행 (Parallelization)

독립적인 에이전트 3개를 `asyncio.gather()`로 동시 실행

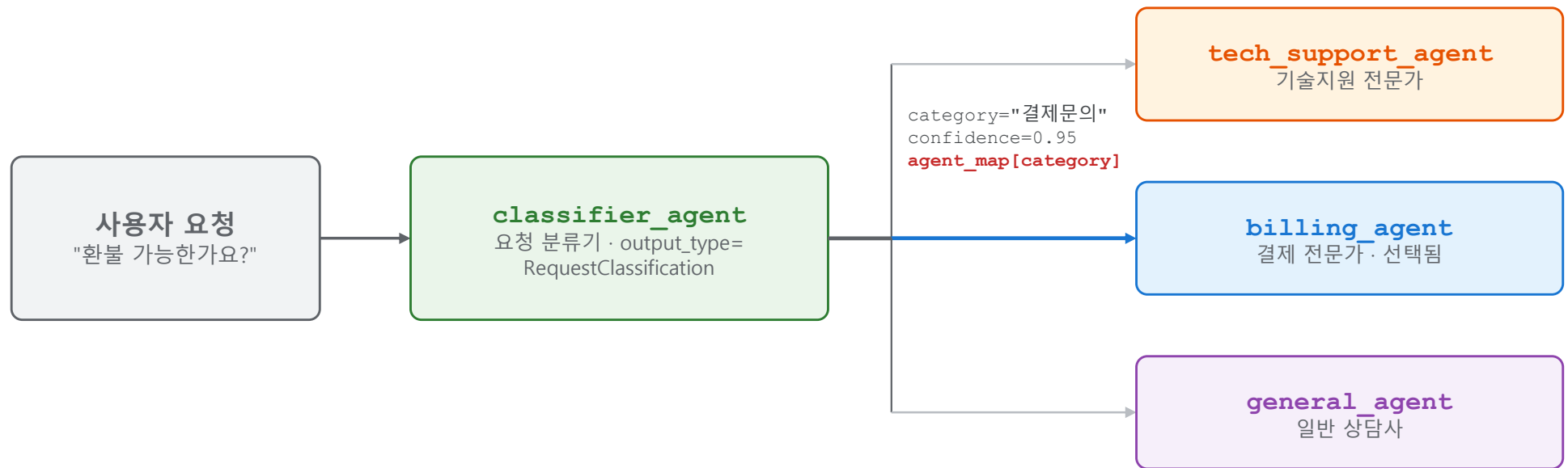
서로 의존성이 없는 분석 작업을 병렬로 실행해 전체 대기 시간을 단축



패턴 3 — 분류 + 라우팅 (Classification + Routing)

Structured Output으로 분류 → 코드(agent_map)가 라우팅을 결정

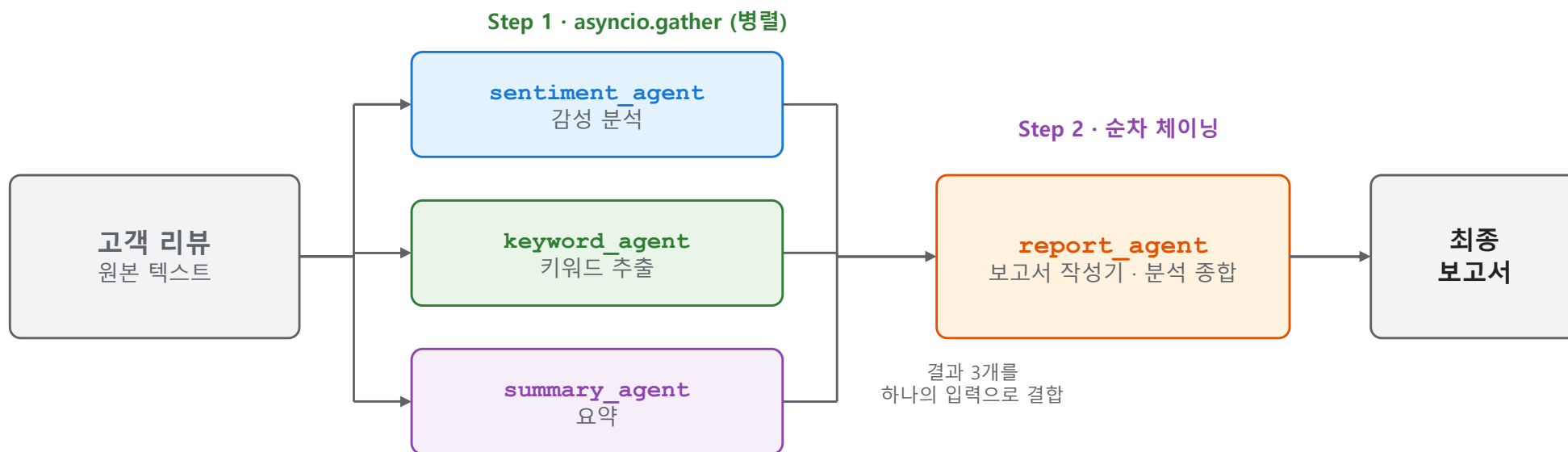
Handoffs와의 차이: LLM이 아니라 코드가 다음 에이전트를 선택하는 결정적(deterministic) 방식



패턴 4 : 병렬 분석 + 순차 종합

실전 워크플로우는 여러 패턴을 결합해 구성

고객 리뷰를 병렬 분석(Step 1)한 뒤, 결과를 모아 보고서 작성기로 순차 체이닝(Step 2)



12. Multi-Agent Orchestration (코드 기반 멀티에이전트 오케스트레이션)

04_Handoffs에서는 LLM이 자율적으로 핸드오프를 결정하는 LLM 기반 오케스트레이션을 배웠습니다. 이번에는 코드로 직접 에이전트 흐름을 제어하는 패턴을 학습합니다.

LLM 기반 vs 코드 기반 오케스트레이션

구분	LLM 기반 (Handoffs)	코드 기반
제어 주체	LLM이 자율 판단	개발자가 코드로 제어
예측 가능성	낮음 (LLM 판단에 의존)	높음 (결정적 흐름)
유연성	높음	보통
비용/속도	변동적	예측 가능
적합한 경우	복잡한 대화, 분류가 어려운 경우	파이프라인, 배치 처리, 정해진 워크플로우

주요 패턴

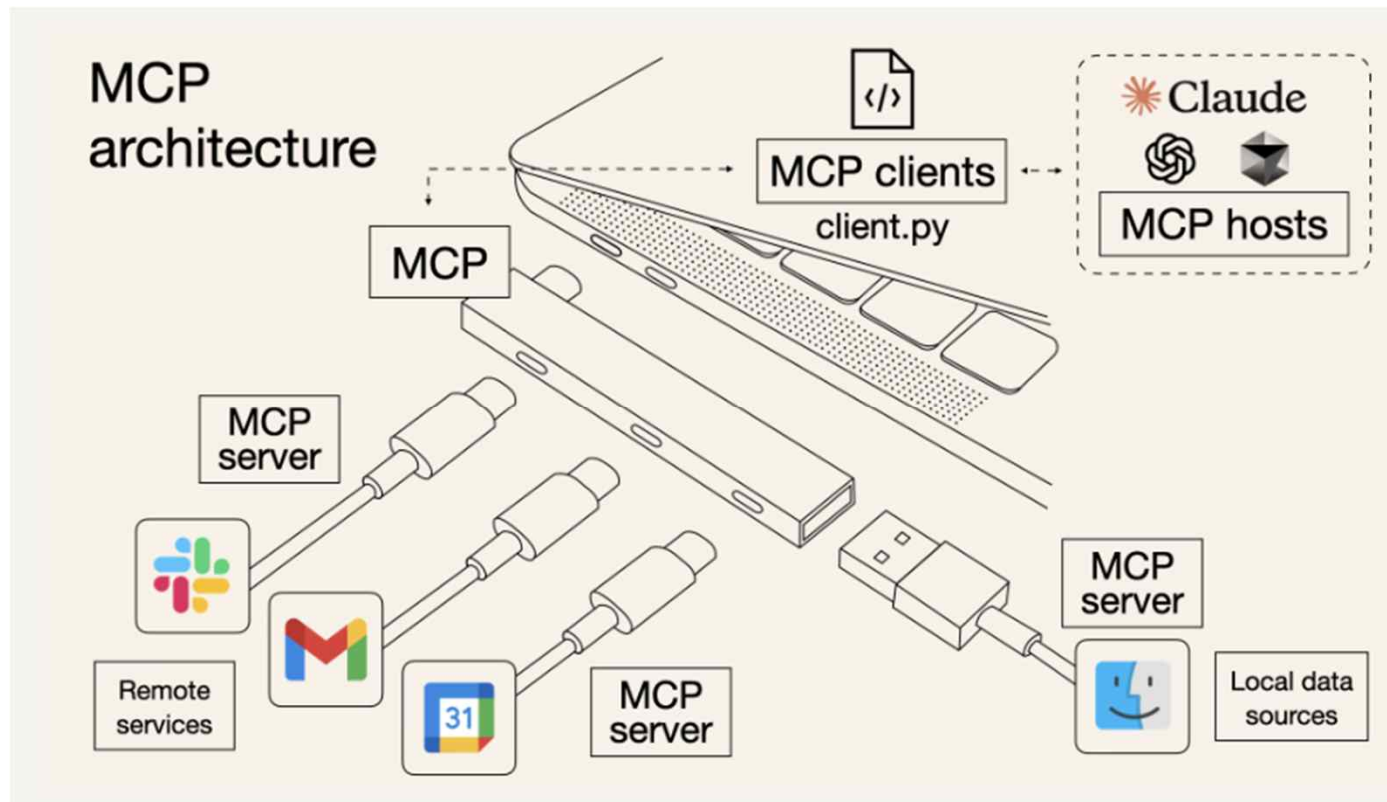
1. 순차 체이닝 (Sequential Chaining): $A \rightarrow B \rightarrow C$
2. 병렬 실행 (Parallelization): A, B, C 동시 실행
3. 분류 → 라우팅 (Classification + Routing): 분류 후 전문 에이전트에 전달

MCP

(Model Context Protocol)

MCP architecture

USB-C Type 같은 표준 규약



Cursor,
Claude Desktop 등

MCP 구현 구성 요소

- MCP server / MCP client
- Resources – AI 모델에게 필요한 정보와 컨텍스트를 제공
- Tools – AI 모델이 외부 시스템에서 작업을 수행할 수 있도록 도구 정의
- Prompts – AI 모델이 리소스와 도구를 활용하여 응답을 생성할 수 있도록 재사용 가능한 템플릿 제공

Resources (리소스)

- 역할: LLM(대형 언어 모델)에 데이터를 제공하는 인터페이스
→ REST API에서의 GET 엔드포인트와 유사
- AI 모델에게 필요한 정보와 컨텍스트를 제공 → 읽기 전용(read-only)
- 중대한 연산이나 부작용(side effect)이 없어야 함
- 유저 프로필 조회, 제품 목록 불러오기, 데이터베이스에서 값 읽기 등을 정의하는 안전한 조회용 API

Tools (툴)

- 역할: LLM이 행동을 수행하도록 하는 인터페이스
→ REST API에서의 POST/PUT/DELETE 엔드포인트와 유사
- AI 모델이 외부에서 작업을 수행할 수 있도록 도구 정의
- 부작용(side effect)이 발생할 수 있음
- 데이터 생성/수정, 이메일 발송, 결제 처리, 워크플로우 실행 등

Prompts (프롬프트)

- 역할: LLM이 리소스와 토크를 더 잘 활용하도록 돕는 재사용 가능한 템플릿
→ 가이드 문구, 매크로, 템플릿 등
- 자주 쓰이는 질문/명령을 표준화
- 일관성 있는 요청을 보장
- 예: “사용자 ID를 입력받아 주문 목록을 조회하는 표준 질문 템플릿”
→ 즉, LLM이 요청할 때 실수를 줄이고, 일관된 결과를 내도록 하는 미리 작성된 대화 가이드입니다.

MCP 표준 전송 유형

- Stdio (Standard In/Output)
서버와 클라이언트가 동일한 프로세스 내에서 작동하거나 모델을 local에서 실행할 때 사용
- SSE (Server-Sent Events)
 - 서버와 클라이언트 간 효율적인 단방향 통신(HTTP 연결)을 통해 간단한 실시간 데이터 업데이트를 구현할 때 사용.
 - 모델의 steaming 출력을 실시간으로 받아야 할 때 사용.

MCP 표준 전송 유형

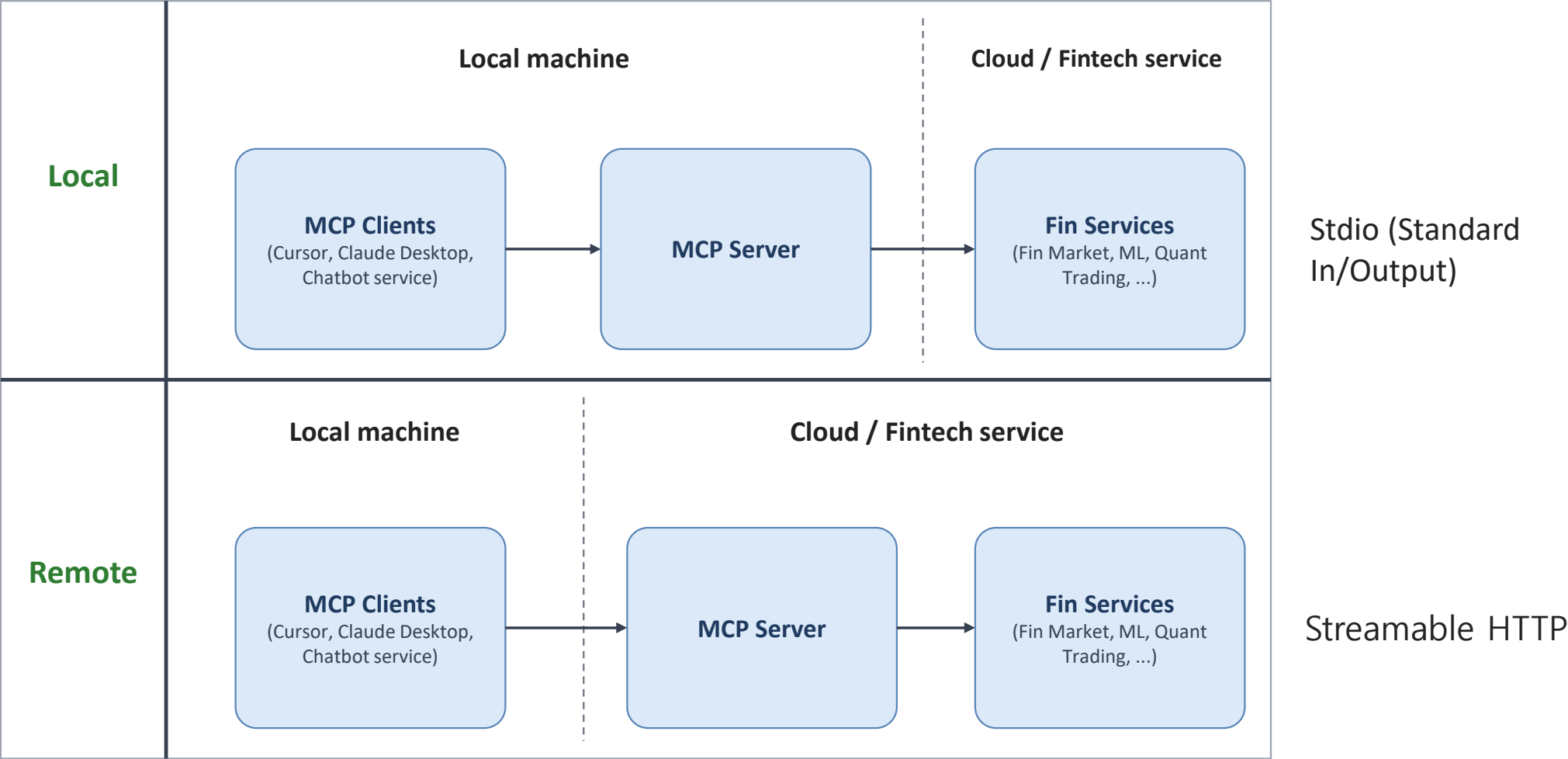
- **Stdio (Standard In/Output)**

- 클라이언트가 MCP 서버를 subprocess로 실행하고, 표준 입출력(stdin/stdout)으로 JSON-RPC 메시지를 교환
- 서버를 로컬 머신에서 실행할 때 사용 — 로컬 연동의 기본(권장) 방식

- **Streamable HTTP**

- 원격(Remote) MCP 서버용 표준 전송 — 단일 HTTP 엔드포인트(예: `https://example.com/mcp`)가 POST/GET을 모두 처리
- Mcp-Session-Id 헤더 기반 세션 관리 — 수평 확장, 서버리스 배포, OAuth 2.1 인증에 유리

Local/Remote MCP



MCP 파이썬 SDK

- SDK (Software Development Toolkit)
 - 개발자가 특정 플랫폼용 앱을 개발할 수 있도록 하는 도구 모음
- 모든 MCP 서버에 연결할 수 있는 MCP 클라이언트를 구축할 수 있음
- 리소스, 프롬프트 및 도구를 노출하는 MCP 서버를 만들 수 있음
- stdio, Streamable HTTP와 같은 표준 전송을 사용
- 모든 MCP 프로토콜 메시지 및 수명 주기 이벤트를 처리

MCP Python SDK 설치

- <https://github.com/modelcontextprotocol/python-sdk>
- `pip install "mcp[cli]"`
- `pip install langchain-mcp-adapters`

MCP Server

```
from mcp.server.fastmcp import FastMCP

# Create an MCP server
mcp = FastMCP("Demo")

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"
```

MCP server 지정

MCP 도구 지정

MCP가 가지고 있는 data, configuration 등 정의

MCP Client

```
client = MultiServerMCPClient(  
    {  
        # Chinook Database MCP 서버 (stdio 기반 로컬 실행)  
        "chinook": {  
            "transport": "stdio",      # 로컬 subprocess 통신  
            "command": "python",      # MCP 서버 실행 명령어  
            "args": ["/agent_server.py"], # MCP 서버 스크립트 경로  
        }  
    }  
)
```

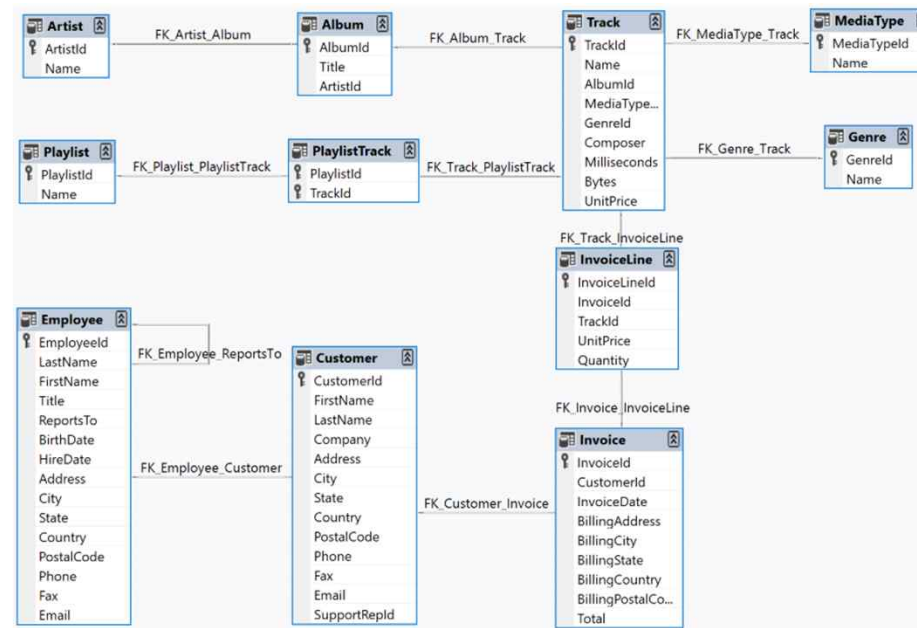
MultiServerMCPClient 설정

```
async def run():  
    print("===== MCP MultiServer 클라이언트 초기화 중... =====")  
  
    # MCP 서버 연결 및 도구 목록 가져오기  
    tools = await client.get_tools()  
  
    # 메모리 체크포인트 생성 (대화 히스토리 유지용)  
    memory = MemorySaver()  
  
    # LangChain ReAct 에이전트 생성  
    agent = create_agent(  
        model=model,  
        tools=tools,  
        checkpoint=memory  
    )
```

MCP 서버 연결

Chinook Database

- 관계형 DB를 학습하고 실습할 수 있도록 제공되는 샘플 DB
- 음악 스토어(Music Store)를 모델링한 데이터베이스



Chinook db 설치

- [Chinook_Sqlite.sql](#) 스크립트를 컴퓨터의 폴더/디렉토리에 저장
- Chinook DB 생성

(langchain) C:\Users\Wtrimu\OneDrive\LangChain_Streamlit_WebService>sqlite3 Chinook.db

- Chinook_Sqlite.sql 스크립트 실행

sqlite> .read Chinook_Sqlite.sql

sqlite> .read Chinook_Sqlite.sql

sqlite> SELECT * FROM Artist LIMIT 10;

```
(langchain) C:\Users\Wtrimu\OneDrive\LangChain_Streamlit_WebService>sqlite3 Chinook.db
SQLite version 3.45.3 2024-04-15 13:34:05 (UTF-16 console I/O)
Enter ".help" for usage hints.
sqlite> .read Chinook_Sqlite.sql
sqlite> SELECT * FROM Artist LIMIT 10;
1|AC/DC
2|Accept
3|Aerosmith
4|Alanis Morissette
5|Alice In Chains
6|Antônio Carlos Jobim
7|Apocalyptica
8|Audioslave
9|BackBeat
10|Billy Cobham
```

- 현재 등록된 모든 의류 목록을 보여줘.
- 신제품인 glove 를 5 달러로 새로 등록해줘.
- DB Resource 테스트 질문
 - **1. Database Info Resource:**
 - 데이터베이스 정보를 보여줘"
 - "Chinook 데이터베이스에 대해 알려줘"
 - "데이터베이스 기본 정보를 확인해줘"
 - **2. Table Info Resource:**
 - "Employee 테이블 정보를 보여줘"
 - "Customer 테이블의 구조를 알려줘"
 - "Album 테이블의 스키마를 확인해줘"
- DB Tools 테스트 질문
 - **1. SQL Query Execution Tool:**
 - "직원이 몇 명인가요?"
 - "고객 수를 알려줘"
 - "앨범이 총 몇 개인가요?"
 - "가장 많이 팔린 곡은 무엇인가요?"
 - "매니저는 몇 명인가요?"

- **2. Table Schema Tool:**
 - "Employee 테이블의 구조를 보여줘"
 - "Customer 테이블의 컬럼 정보를 알려줘"
 - "Track 테이블의 스키마를 확인해줘"
- **3. List Tables Tool:**
 - "데이터베이스에 어떤 테이블들이 있나요?"
 - "사용 가능한 테이블 목록을 보여줘"
 - "모든 테이블 이름을 알려줘"
- **연속 대화 테스트:**
 - 1단계: "직원이 몇 명인가요?"
 - 2단계: "그 중에서 매니저는 몇 명인가요?"
- **컨텍스트 참조 테스트:**
 - 1단계: "고객이 몇 명인가요?"
 - 2단계: "그 고객들 중에서 미국 고객은 몇 명인가요?"
 - 3단계: "앞서 말한 고객들의 주문 건수는?"

실습 1 : MCPServerStdio 클라이언트 (Flask)

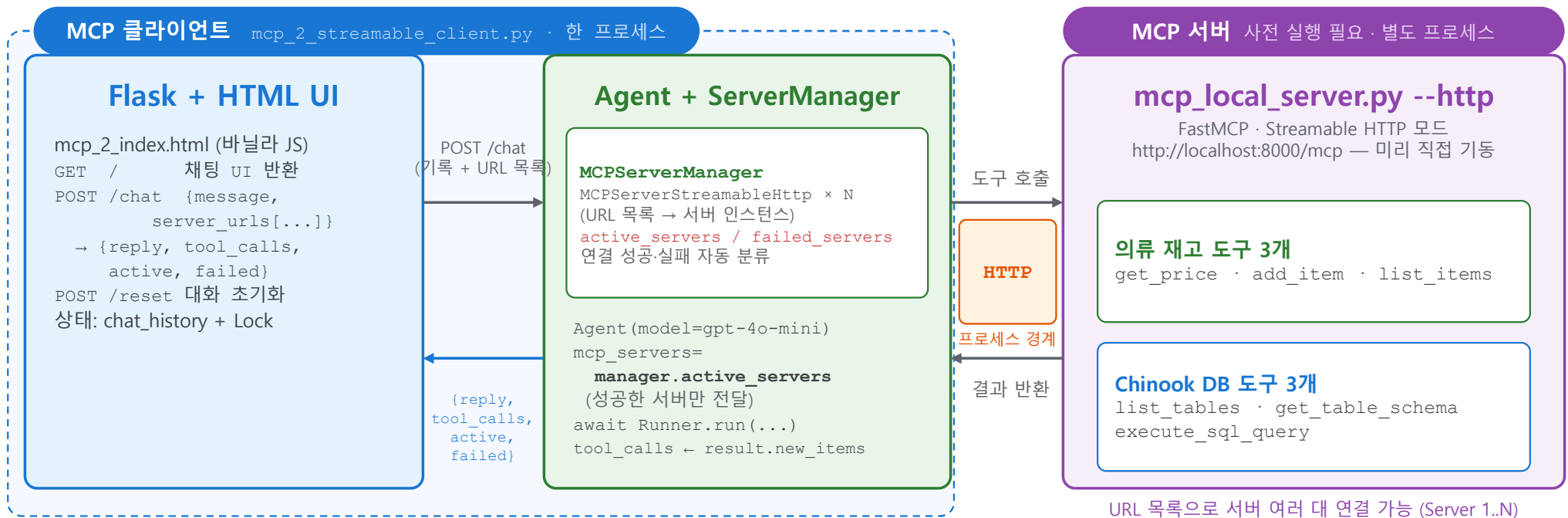
Hands-on



실행: `python mcp_1_stdio_client.py` → `http://localhost:8001` 접속

실습 2 : MCPServerStreamableHttp + MCPServerManager (Flask)

Hands-on



실행 순서: ① `python mcp_local_server.py --http (port 8000)` ② `python mcp_2_streamable_client.py` → `http://localhost:8002` 접속

서버 실행

```
(langchain) C:\Users\trimu\OneDrive\Agentic_AI_GPT_MCP\03_LangChain_Agent\400_DB_MCP_Agent> python mcp_local_server.py --http
MCP Server is running...
```

FASTMCP

FastMCP 2.14.5

📄 Server: ChinookDBAnalysis
🚀 Deploy free: <https://fastmcp.cloud>

🌟 FastMCP 3.0 is coming!

Pin `fastmcp < 3` in production, then upgrade when you're ready.

Chinook 데이터베이스 연결 성공

```
[02/13/26 14:41:50] INFO Starting MCP server 'ChinookDBAnalysis' server.py:2506
with transport 'stdio'
```


LangChain Agent

LangChain이란?

- 대규모 언어 모델(LLM)을 활용한 애플리케이션 개발 프레임워크
- LLM 애플리케이션의 전체 라이프사이클을 간소화
- Python 및 TypeScript 지원
- **LangChain**
 - 원하는 모델 제공자를 사용해 에이전트를 빠르고 손쉽게 구축
- **LangGraph**
 - 에이전트의 모든 단계를 세밀하게 제어하는 저수준 오케스트레이션, 메모리, Human-in-the-loop
- **DeepAgents** **NEW**
 - 계획(Planning), 서브에이전트, 파일시스템, 장기 메모리 내장 에이전트 하네스

LangChain이란?

에이전트를 설계·테스트·배포(design · test · ship)하기 위한 오픈소스 스택

빠른 시작 · 내장 기능 최대

세밀한 제어 · 저수준

Deep Agents NEW

복잡한 장기 실행 작업용 완성형 에이전트 하네스

- 계획 (Planning) 내장
- 서브에이전트 (Subagents)
- 가상 파일시스템
- 장기 메모리 (Long-term Memory)

LangChain

미니멀하고 구성 가능한 에이전트 프레임워크

- 필요한 것만 조합해 구성
- 모델 · 도구 · 프롬프트
- 미들웨어 (Middleware)
- 원하는 모델 제공자 사용

LangGraph

stateful · long-running 에이전트용 저수준 오케스트레이션

- Durable Execution (중단 후 재개)
- 스트리밍 (Streaming)
- 메모리 (Memory)
- Human-in-the-loop

선택 요령: 복잡한 장기 작업이거나 뭘 쓸지 모르겠다면 **Deep Agents** · 가볍고 단순한 커스텀 에이전트는 **LangChain** `create_agent` · 실행 흐름(상태·분기·중단/재개)을 코드로 직접 제어해야 하면 **LangGraph**

LangChain의 장점과 단점

- 장점

- 오픈 소스이므로 누구나 자유롭게 이용 가능하고 빠른 업데이트 가능
- 다양한 LLM 에 표준화된 인터페이스 지원 → LLM 공급자 전환 용이
- 모듈화된 설계: 프롬프트 템플릿, 문서 로더, 출력 파서 등 다양한 모듈 제공
- 관찰 가능성 및 평가 기능 제공 (LangSmith)

- 단점

- 언어 지원 제한: Python 및 TypeScript 이외의 언어 지원 부족
- 복잡한 초기 설정과 다양한 옵션으로 초보자에게 진입 장벽
- 복잡한 기능과 다양한 옵션으로 인해 학습 곡선이 가파를 수 있음
- 버전간 코드 차이가 크며 공식 문서가 최신 코드와 불일치 하는 경우 있음
- API 업데이트 반영 지연: OpenAI 등의 새로운 기능이 즉각 반영되지 않을 수 있음

LangChain 주요 개념 (Core Components)

1. **Agents (에이전트)** - 사용자 입력에 따라 도구를 선택하고 실행하는 자율적 시스템
2. **Models (모델)** - 채팅에 특화된 LLM (Chat Models), 다양한 제공자 통합 지원
3. **Messages (메시지)** - 채팅 모델의 입력과 출력 단위 (Human, AI, System, Tool)
4. **Tools (도구)** - 모델이 호출할 함수 정의 및 Tool Calling 기능
5. **Short-term memory (단기 메모리)** - 대화 기록을 메시지 시퀀스로 관리
6. **Streaming (스트리밍)** - 모델 응답을 토큰 단위로 실시간 전달
7. **Structured output (구조화된 출력)** - JSON 등 지정 스키마에 맞춰 응답

Messages & Context Engineering

메시지 구성 요소

Role (역할) → 메시지의 유형 식별,

예: system, user, assistant 등

Content (내용) → 메시지의 실제 내용 (텍스트, 이미지, 오디오 등)

Metadata (메타데이터) → 선택적(optional) 필드.
응답 정보(response info), 메시지 ID, 토큰
사용량(token usage) 등

메시지 유형 (Message types)

System message (시스템 메시지)

Human message (사용자 메시지)

AI message (AI 메시지)

Tool message (도구 메시지)

```
1 from langchain_core.messages import
   SystemMessage, HumanMessage
2
3 msgs = [
4     SystemMessage(content="너는 친절한 튜터야"),
5     HumanMessage(content="RAG를 3줄로 설명해
   줘"),
6 ]
7 resp = llm.invoke(msgs)
8 print(resp.content)
9
```

Runnables: invoke · batch · stream

Runnable이 주는 이점

- 동일한 인터페이스로 실행(invoke/batch/stream)
- 구성 요소를 "레고처럼" 연결
- 캐싱/로깅/추적을 붙이기 쉬움

```
1 chain = prompt | llm
2
3 # 1) invoke
4 chain.invoke({"q": "LangGraph가 뭐야?"})
5
6 # 2) batch
7 chain.batch([{"q": "A"}, {"q": "B"}])
8
9 # 3) stream
10 for chunk in chain.stream({"q": "짧게 요약"}):
11     print(chunk)
```

Langgraph 란?

Langgraph 란?

- LLM을 활용해 상태 기반의 다중 에이전트 애플리케이션을 구축
- 그래프 구조를 통해 복잡한 워크플로우를 효율적으로 설계 및 관리
- 상태를 지속적으로 유지해 반복 작업과 복잡한 상태 관리를 지원
- 인간의 개입이 필요한 워크플로우와 메모리 기능을 유연하게 구현

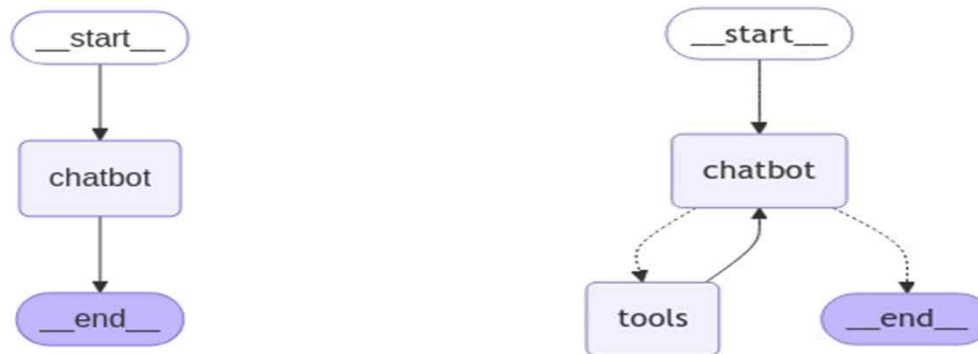
주요 특징

- 상태 지속성
- 워크플로우 최적화
- 복잡한 작업 처리
- **활용 사례** - 고객 서비스 챗봇, 복잡한 데이터 분석, 교육용 튜터링 시스템 등

Graph 핵심 요소 – state machine

Graph 핵심 요소 – state machine

- State – 애플리케이션의 현재 스냅샷을 나타내는 공유 데이터 구조. TypedDict 나 Pydantic BaseModel로 구성.
- Nodes – Agent 의 logic을 가진 Python 함수. 현재 값을 State 입력으로 받고, 업데이트된 값을 반환
- Edges – 현재 State에 따라 다음에 실행할 Node 를 결정하는 Python 함수. 조건부 분기 또는 고정 전환 가능.



State 구성

- State는 여러 키를 가진 딕셔너리(TypedDict)
- 노드 와 노드 간에 정보를 전달하는 역할
- 각 키는 독립적인 Reducer(병합 규칙)를 가집니다.
- Reducer(add_messages) : 자동으로 list 에 메시지를 추가해 주는 기능
- Reducer를 명시하지 않으면 기본값은 덮어쓰기(override)입니다.

Node 구성

Node 구성

- 파이썬 함수(동기 또는 비동기)이며, 첫 번째 위치 인수는 state
- (선택적으로) 두 번째 위치 인수는 선택적 구성 가능한 매개변수
 - 예: thread_id를 포함하는 "config"
- **START node** – 사용자 입력을 그래프로 전송하는 특수 노드
- **END node** – 완료를 나타내는 특수 노드

Edge 구성

- 노드와 노드 간의 연결
- **일반 에지**: 한 노드에서 다음 노드로 직접 이동
- **조건부 에지**: 다음에 어느 노드로 이동할지 결정하는 함수를 호출
- **진입점**: 사용자 입력이 도착했을 때 가장 먼저 호출할 노드 결정
 - `graph.add_edge(START, "node_a")`
- **조건부 진입점**: 사용자 입력이 도착했을 때 먼저 호출할 노드를 결정하는 함수를 호출
 - `graph.add_conditional_edges(START, routing_function, {True: "node_b", False: "node_c"})`

LangGraph Checkpointer Memory

LangGraph Checkpointer Memory

Checkpoint이란?

- LangGraph 실행 상태(state)를 저장·복구하는 메커니즘
- 각 노드간 실행결과를 추적하기 위한 메모리(대화에 대한 기록과 유사 개념)

동작 방식

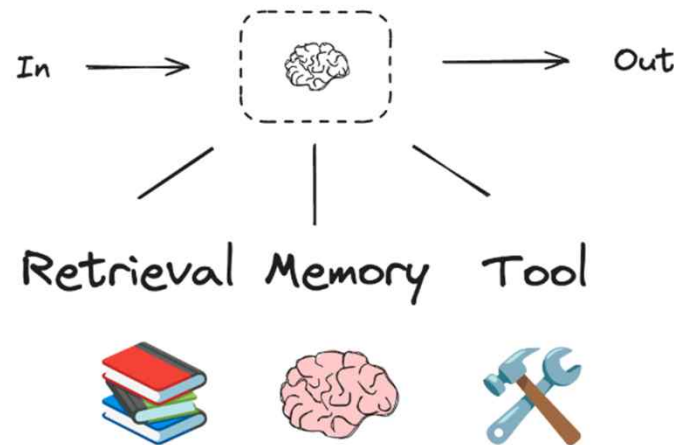
- 노드 실행 전/후의 상태(state)를 저장 (messages, 변수 값, edge 위치 등)
- 실행이 중단되면 마지막 체크포인트에서 재개
- 체크포인트를 활용하여 특정 시점(Snapshot)으로 되돌리기 기능도 가능

활용 시나리오

- 대화형 에이전트(Chatbot)에서 사용자 맥락 유지
- 복잡한 워크플로우 실행 중단 후 재개
- 장기 세션 관리 및 상태 기반 로깅

Agent의 핵심 기술 스택

- LLM (예: GPT, Claude): 언어 이해 및 생성
- LangChain / LangGraph: 에이전트 워크플로우 및 상태 관리
- Vector Database (예: Chroma, Pinecone): 정보 저장 및 검색
- Memory: 상태 유지 및 맥락 관리
- API 통합 (예: Tavily, SerpAPI): 외부 도구와 연결



도구 속성 커스터마이징 & Pydantic 스키마

도구 속성 재정의

- name, description 커스터마이징으로 LLM 이해도 향상
- args_schema로 입력 파라미터 타입/제약 명시
- return_direct=True로 에이전트 루프 즉시 종료 가능

Pydantic 스키마 활용

- BaseModel 상속으로 도구 입력 구조 정의
- Field(description=...) 으로 각 필드 설명 추가
- 타입 검증 자동화 → LLM 호출 오류 감소

핵심 패턴

- @tool 데코레이터: 함수→도구 변환
- StructuredTool: 클래스 기반 정의
- bind_tools(): 모델에 도구 바인딩

코드 예시

```
class SearchInput(BaseModel):  
    query: str = Field(  
        description="search term"  
    )
```

ReAct Agent 생성 & 상태 관리

create_agent()

- model + tools → 즉시 실행 가능한 에이전트
- system prompt로 에이전트 역할 지정
- invoke() / stream() 두 가지 실행 방식

에이전트 상태(AgentState)

- messages: 대화 기록 자동 누적
- thread_id로 대화 세션 분리
- checkpointer 연결 시 상태 영속화

ReAct 실행 흐름

1. 사용자 입력 수신
2. LLM 추론 (Reason)
3. 도구 호출 (Action)
4. 결과 관찰 (Observe)
5. 완료 판단 → 응답 반환

상태 관리 포인트

- config={"thread_id": "abc"}
- get_state()로 현재 상태 조회

도구 속성 커스터마이징 & Pydantic 스키마

도구 속성 재정의

- name, description 커스터마이징으로 LLM 이해도 향상
- args_schema로 입력 파라미터 타입/제약 명시

Pydantic 스키마 활용

- BaseModel 상속으로 도구 입력 구조 정의
- Field(description=...) 으로 각 필드 설명 추가
- 타입 검증 자동화 → LLM 호출 오류 감소

핵심 패턴

- @tool 데코레이터: 함수→도구 변환
- bind_tools(): 모델에 도구 바인딩

코드 예시

```
class SearchInput(BaseModel):  
    query: str = Field(  
        description="search term"  
    )
```

구조화된 출력 (Structured Output)

- 모델이 특정한 출력 구조를 따르도록 지시 받는 것
- 모델의 응답이 데이터베이스 스키마나 다른 정해진 형식에 부합하도록 보장
→ 모델 출력의 일관성 유지
- 구조화된 출력 구현 방법 → 출력 구조를 스키마로 정의
- 스키마는 JSON과 같은 단순한 형식으로 표현
- Pydantic과 같은 도구를 사용하여 타입 힌트와 유효성 검사를 포함한 스키마 정의
- LangChain은 with_structured_output() 메서드를 제공하여 스키마를 모델에 바인딩하고, 모델이 구조화된 출력을 생성하도록 지원

```
1 from pydantic import BaseModel, Field
2
3 class Ticket(BaseModel):
4     category: str =
5         Field(description="billing")
6     priority: int = Field(ge=1, le=5)
7     summary: str
8
9 classifier =
10     llm.with_structured_output(Ticket)
11
12 out = classifier.invoke("결제가 두 번 됐어
13     요")
14
15 print(out)
```

실습 : 101_Tools & Agents

Tools & Agents 기초

이 노트북에서는 LangChain의 도구(Tools)와 에이전트(Agents) 개념을 학습합니다. 도구는 모델이 외부 세계와 상호작용할 수 있게 해주며, 에이전트는 도구를 사용하여 복잡한 작업을 수행합니다.

참고:

- [LangChain 공식 문서 - Tools](#)
- [LangChain 공식 문서 - Agents](#)

Memory: “대화”를 상태로 다루기

단기 메모리(Short-term Memory)

- AI가 이전 상호작용의 정보를 기억하는 시스템
- 에이전트가 여러 번의 사용자 상호작용을 포함한 복잡한 작업을 수행가능
- 에이전트는 AgentState 를 사용하여 단기 메모리(short-term memory) 를 관리합니다. 특히, 대화 이력(conversation history)은 messages 키를 통해 저장

장기 메모리(Long-term))

- 사용자 선호/프로필/지식 조각을 JSON 형태로 별도 Store에 보관
- retrieval로 필요할 때만 불러오기
- 개인정보/보안 정책이 핵심

InMemorySaver + SqliteSaver

InMemorySaver (단기 메모리)

- Python 딕셔너리 기반 인메모리 저장
- 프로세스 종료 시 데이터 소실
- 개발/테스트 환경에 적합

SqliteSaver (영속 메모리)

- SQLite DB 파일 기반 영구 저장
- 서버 재시작 후에도 대화 복원
- async with 컨텍스트 매니저 사용
- 프로덕션 환경에 적합

비교표

InMemorySaver

- 저장: RAM | 영속: X | 속도: 빠름
- 용도: 개발/테스트

SqliteSaver

- 저장: 파일 | 영속: O | 속도: 중간
- 용도: 프로덕션

PostgresSaver

- 저장: DB서버 | 영속: O | 속도: 중간
- 용도: 대규모 프로덕션

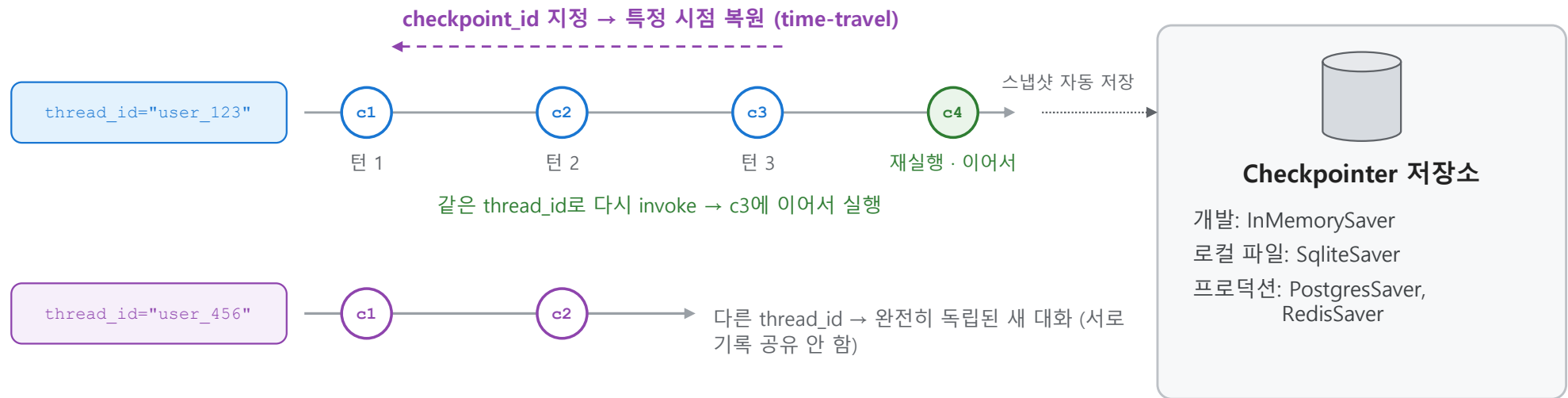
Persistence 상세 — thread_id / checkpoint_id 동작 원리

Thread = 대화 격리 단위

- thread_id: 각 실행 흐름(대화)을 식별하는 고유 ID
- 같은 thread_id → 이전 상태 이어서 실행
- 다른 thread_id → 완전히 독립된 새 대화
- **멀티유저 = 사용자별 thread_id 할당**

Checkpoint = 상태 스냅샷

- 매 super-step마다 자동 저장
- checkpoint_id로 특정 시점 상태 복원 가능
- 개발: InMemorySaver · 로컬: SqliteSaver
- 프로덕션: PostgresSaver / RedisSaver



실습 : 102_Memory Concepts

Memory Concepts (메모리 개념)

메모리(Memory) 는 AI 에이전트가 대화 이력과 상태를 저장하고 관리하는 시스템입니다.

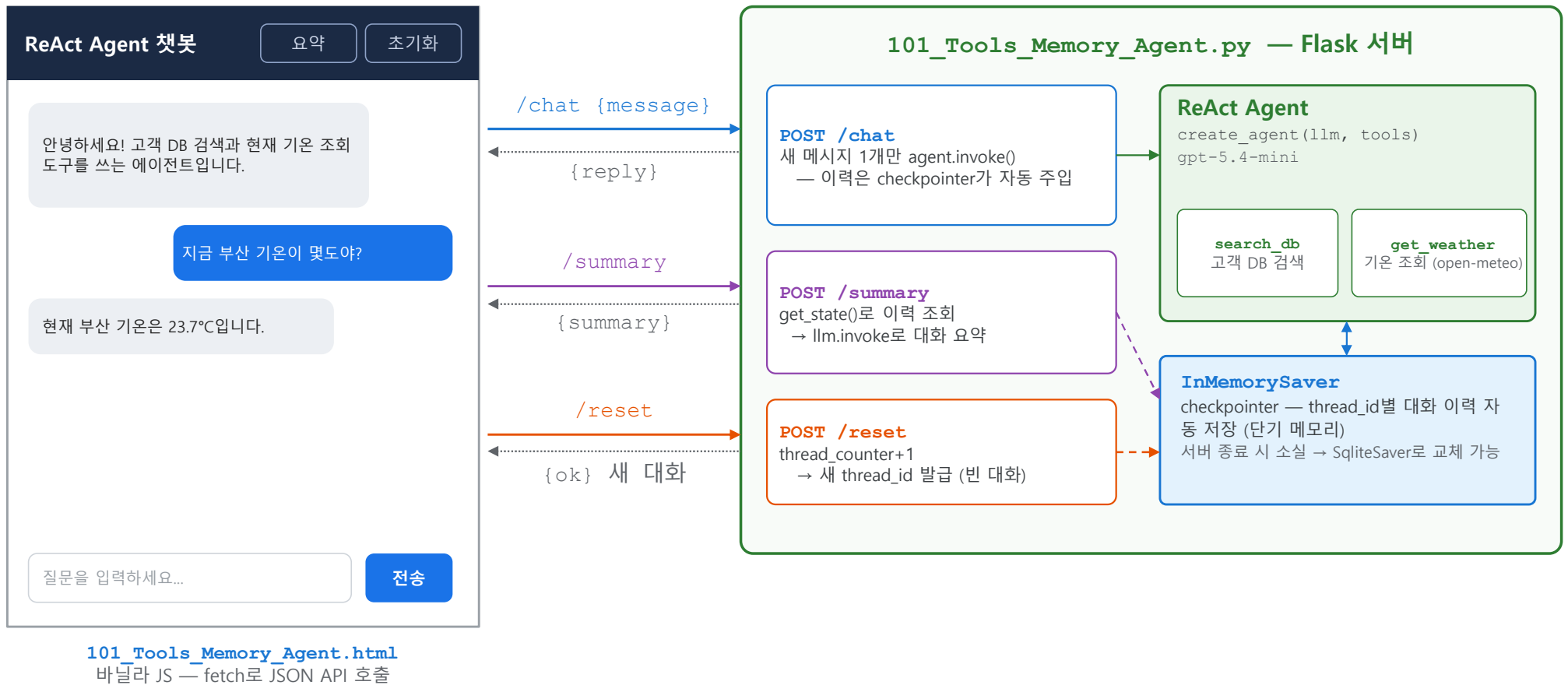
메모리는 두 가지 주요 유형으로 분류됩니다:

- 단기 메모리(Short-term Memory): 하나의 대화 세션(thread) 내에서 이전 상호작용을 기억
- 장기 메모리(Long-term Memory): 여러 대화 세션에 걸쳐 정보를 유지

이 노트북에서는 메모리의 기본 개념과 사용법을 다룹니다.

참고: [LangChain 공식 문서 - Memory](#)

실습 : Flask_WebUI / 101_Tools_Memory_Agent.py



실행: `python 101_Tools_Memory_Agent.py` → `http://localhost:8000`

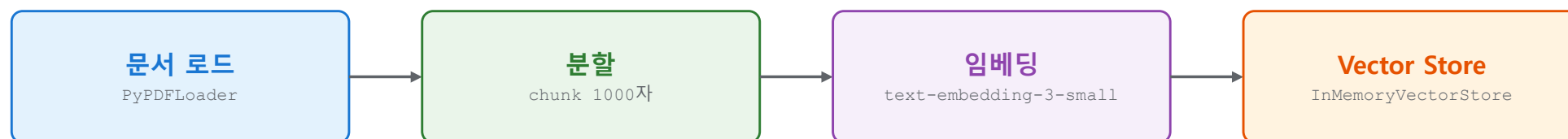
핵심: 매 요청에 새 메시지 1개만 전달 — 이력은 checkpoint가 자동 관리

Semantic Search → RAG 파이프라인

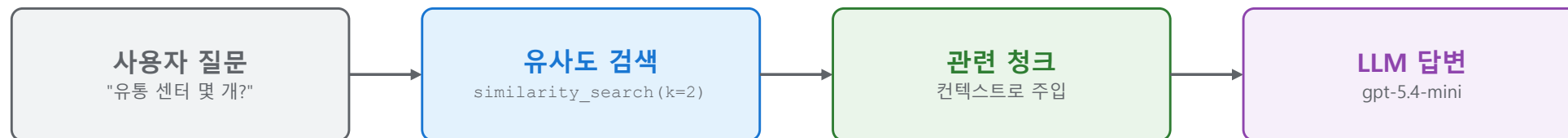
의미(임베딩 유사도) 기반 검색 — 키워드 매칭이 아닌 벡터 거리로 관련 문서를 찾는다

예) 나이키 10-K 보고서 PDF를 지식 베이스로 만들어 에이전트가 검색하게 한다.

① 인덱싱 (최초 1회)



② 검색 + 생성 (질문마다)



검색 품질 3요소 ① 분할 — 의미 단위 청크 ② Retriever 전략 — similarity/MMR-k값 ③ 프롬프트 — 검색 결과 사용법 지시

문서 로드와 분할 (Loaders & Splitters)

페이지 단위는 검색에 너무 크다 — 의미 단위 청크로 분할해야 정확히 검색된다

문서 로드 — 소스를 Document 객체로

PDF · 웹페이지 · CSV · DB 등 소스별 로더 제공

```
loader = PyPDFLoader("nke-10k-2023_korean.pdf")
docs = loader.load() # 페이지당 Document 1개
docs[0].metadata     # {source, page, ...}
```

텍스트 분할 — RecursiveCharacterTextSplitter

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, # 청크 최대 1000자
    chunk_overlap=200, # 200자 중첩
    add_start_index=True)
```

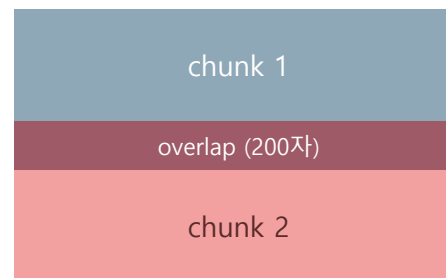
분할 전략 3가지

길이 기준 고정 크기 — 간단·일관

텍스트 구조 단락·문장 단위 — 의미 일관성 (실습 방식)

문서 구조 HTML·Markdown 구조 보존

Chunk Overlap — 경계에서 문맥이 끊기지 않도록 중첩



경계에 걸린 문장도 두 청크 중 하나에는 온전히 포함 → 검색 누락 방지

임베딩과 Vector Store

텍스트 → 벡터 변환 후 저장 — 의미가 비슷하면 벡터 공간에서 가깝다

```
embeddings = OpenAIEmbeddings(  
    model="text-embedding-3-small")  
vs = InMemoryVectorStore(embeddings)  
vs.add_documents(all_splits) # 임베딩+저장  
  
results = vs.similarity_search(query, k=2)
```

어떤 Vector Store를 쓸까?

	InMemoryVectorStore	Chroma
용도	실습·소규모 — 이 노트북	영구 저장·대규모
저장	메모리 — 종료 시 소실	persist_directory로 디스크 저장
설치	불필요 — 코어에 내장	chromadb + langchain-chroma

교체는 한 줄: `Chroma(embedding_function=embeddings,
persist_directory="./db")`

호스팅형(Pinecone) · 독립형(FAISS, Chroma) · 인메모리형 — 규모와 영속성 요구로 선택

검색을 에이전트에 연결 — Agentic vs Chain RAG

벡터 스토어 검색을 도구(tool)로 감싸면 에이전트가 필요할 때 스스로 호출한다

```
@tool(response_format="content_and_artifact")
def retrieve_context(query: str):
    """질문 관련 정보를 검색합니다."""
    docs = vs.similarity_search(query, k=2)
    return serialized, docs

agent = create_agent(model, [retrieve_context],
                    system_prompt="먼저 검색 후 답변")
```

`content_and_artifact` — LLM에게 줄 문자열과 원본 Document 객체를 함께 반환

Agentic RAG (이 실습)

- 에이전트가 필요할 때만 검색 도구 호출
- 유연함 — 검색 불필요한 질문은 바로 답
- 모델 호출 여러 번 → 지연 시간 증가

Chain 기반 RAG

- 항상 검색 먼저 실행 → 컨텍스트로 주입
- 단일 모델 호출 — 빠르고 예측 가능
- 모든 질문에 검색 → 유연성 낮음

선택 기준: 대화형·질문 다양 → Agentic / FAQ처럼 항상 검색이 필요 → Chain

Streaming: UX를 바꾸는 기능

스트리밍이 중요한 이유

- 응답 지연이 길수록 체감 품질 급락 — 완성 전에 점진적으로 표시
- 토큰/단계 단위로 "진행 중"을 보여주면 신뢰 상승

stream_mode — 무엇을 스트리밍할지 선택

values

매 스텝 후 전체 상태 스냅샷 — 실습 기본값

updates

각 스텝의 변경분만 — 노드별 진행 상황

messages

LLM 토큰 단위 — 실시간 타이핑 효과

실시간 챗봇

조합 가능: stream_mode=["updates", "messages"]

토큰 스트리밍 + 상태 추적 동시 수신

```
for event in agent.stream(
    {"messages": [{"role": "user",
                    "content": "질문"}]},
    config, # thread_id
    stream_mode="values",
):
    event["messages"][-1].pretty_print()
```

실습 : 103_RAG_Agent 작성

RAG Agent (검색 증강 생성 에이전트)

RAG (Retrieval-Augmented Generation) 는 외부 지식 소스에서 정보를 검색하여 LLM의 응답을 향상시키는 기술입니다.

RAG 에이전트는 검색 도구를 사용하여 관련 문서를 찾고, 이를 컨텍스트로 활용하여 답변을 생성합니다.

참고: [LangChain 공식 문서 - RAG Agent](#)

RAG 워크플로우

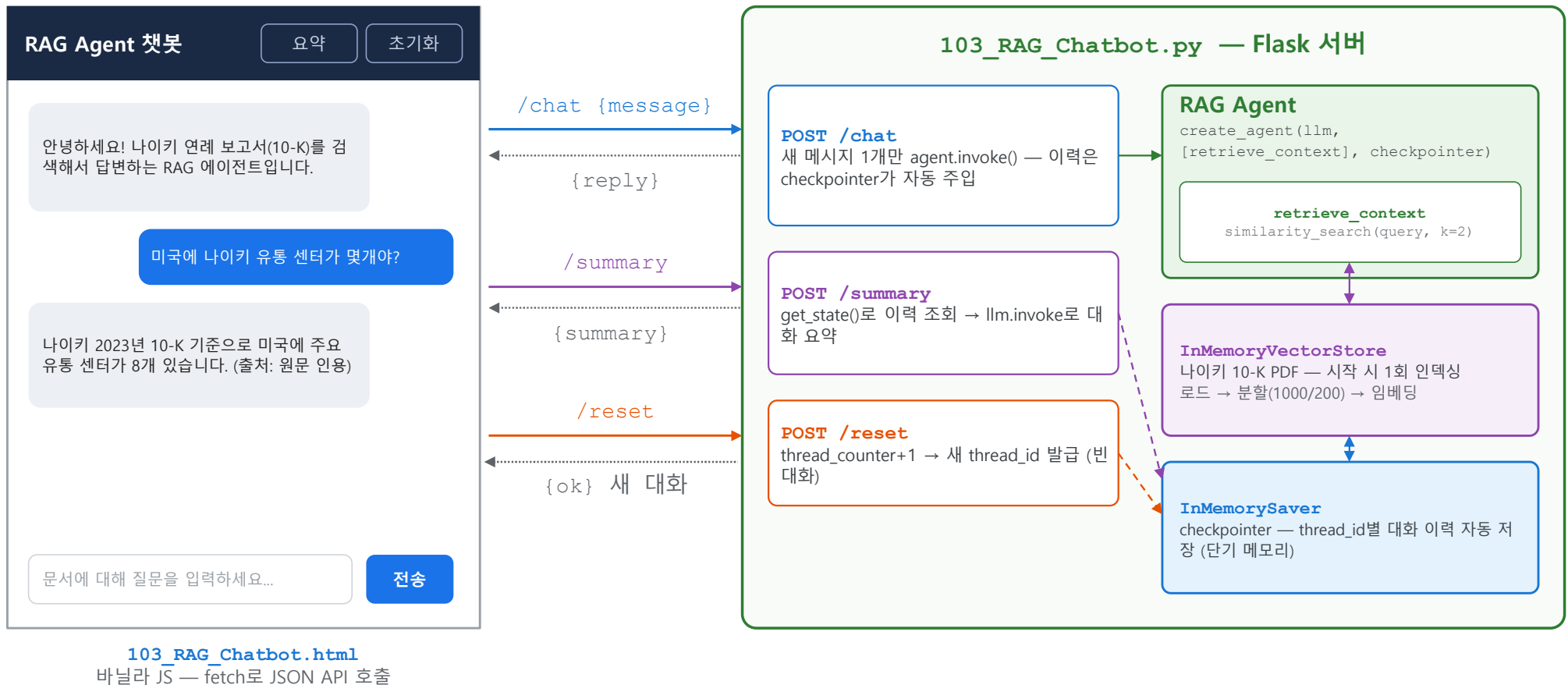
1. 인덱싱 (Indexing): 문서를 로드, 분할, 임베딩하여 벡터 스토어에 저장
2. 검색 및 생성 (Retrieval & Generation): 사용자 질문에 대해 관련 문서를 검색하고 답변 생성

RAG 패턴 이해

RAG는 두 가지 주요 접근 방식이 있습니다. 이 노트북에서는 에이전트 기반 RAG를 중심으로 다루고, 체인 기반 RAG는 마지막에 참고로 비교합니다.

실습 : Flask_WebUI / 103_RAG_Chatbot.py

Hands-on

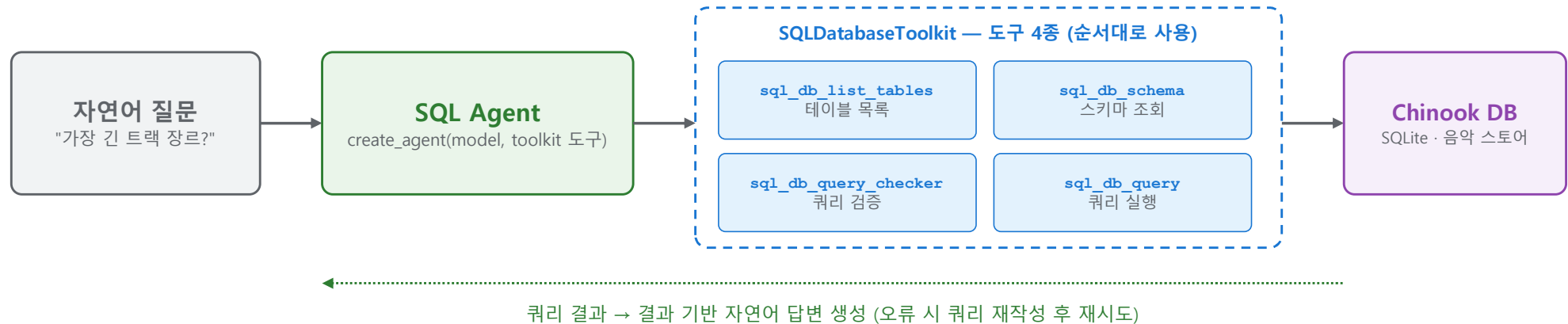


실행: `python 103_RAG_Chatbot.py` → `http://localhost:8000`

SQL Agent & HITL (Human-in-the-Loop)

LangChain Learn

자연어 → SQL 변환 → 실행 → 답변 — Chinook DB(음악 스토어)로 실습



시스템 프롬프트에 내장된 안전 규칙 (노트북)

- DML(INSERT/UPDATE/DELETE/DROP) 절대 실행 금지
- 결과는 최대 top_k=5개로 제한 (LIMIT 강제)
- 테이블 목록 → 스키마 확인을 반드시 먼저
- 실행 전 구문 재확인, 오류 시 재작성

그래도 부족하다 — 왜 HITL인가?

- 프롬프트 규칙은 LLM이 어길 수 있음 — 잘못된 SQL은 비용·보안 사고로 직결
- 운영 권장: 읽기 전용 DB 계정 + SELECT 화이트리스트
- 실행 직전 사람이 SQL을 보고 승인 → 다음 슬라이드

Human-in-the-Loop — interrupt & resume 흐름

LangChain Learn

sql_db_query 실행 직전에 일시 정지 → 사람이 SQL 검토·승인 → 중단 지점부터 재개



장점

- 안전성 — 민감 작업(결제·데이터 삭제) 실행 전 승인
- 품질 — 사람이 중간 결과 검토 후 피드백 → 응답 개선
- 투명성 — 개입 지점이 기록으로 남음 → 감사/로그 관리

활용 시나리오

- 도구 호출 — 외부 API 요청 전에 사람 승인 (이 실습)
- 콘텐츠 생성 — 보고서·코드 자동 작성 후 검토·수정
- 안전 민감 작업 — 보안 설정 변경, 재무 트랜잭션 등

실습 : 104_SQL Agent

SQL Agent (SQL 데이터베이스 Q&A 에이전트)

이 튜토리얼에서는 LangChain을 사용하여 SQL 데이터베이스에 대한 자연어 질의응답 시스템을 구축합니다. 에이전트는 자연어 질문을 SQL 쿼리로 변환하고, 결과를 바탕으로 답변을 생성합니다.

참고: [LangChain 공식 문서 - SQL Agent](#)

학습 내용

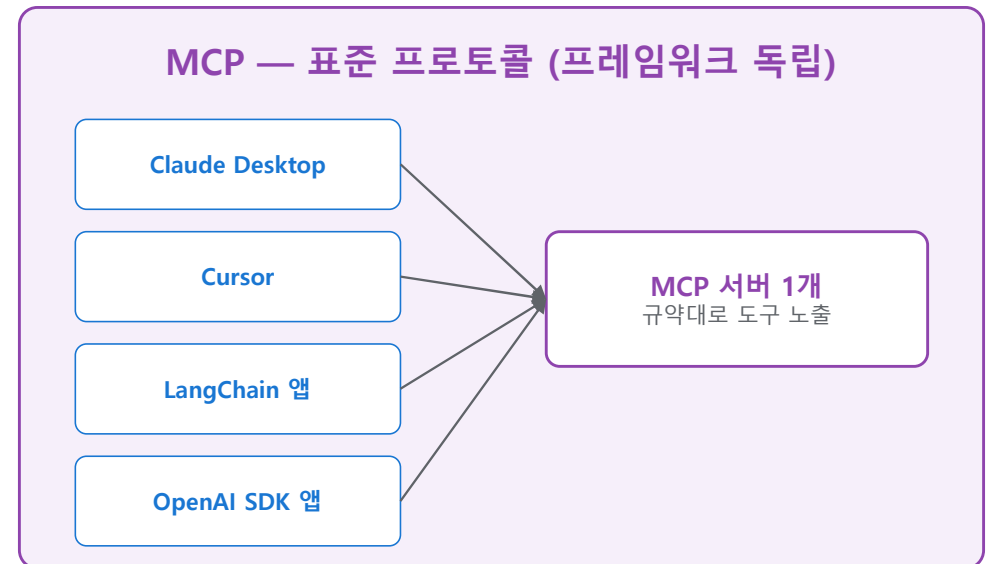
- SQL 데이터베이스 연결
- SQL 도구 생성
- SQL 에이전트 구축
- Human-in-the-loop 검토

테스트 질문 예시

- "고객 수는?"
- "가장 많이 팔린 트랙은?"
- "국가별 매출 Top 5"
- "매니저가 몇 명인가요?"

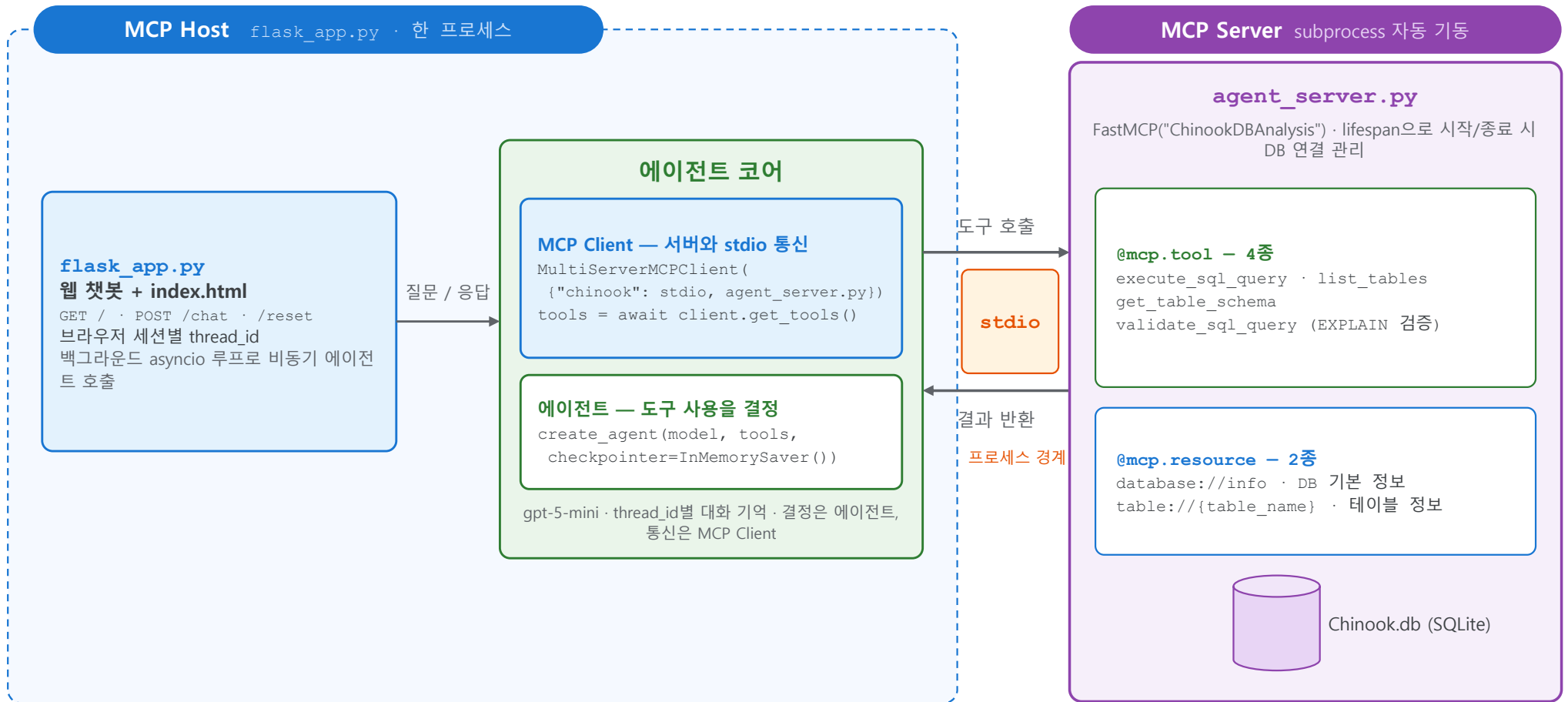
MCP (Model Context Protocol)

LangChain Tools는 프레임워크 안에서만 — MCP는 표준 규약만 지키면 어떤 클라이언트든 사용



langchain-mcp-adapters : MultiServerMCPClient.get_tools()가 MCP 서버의 도구를 LangChain 도구로 변환 → create_agent에 그대로 전달

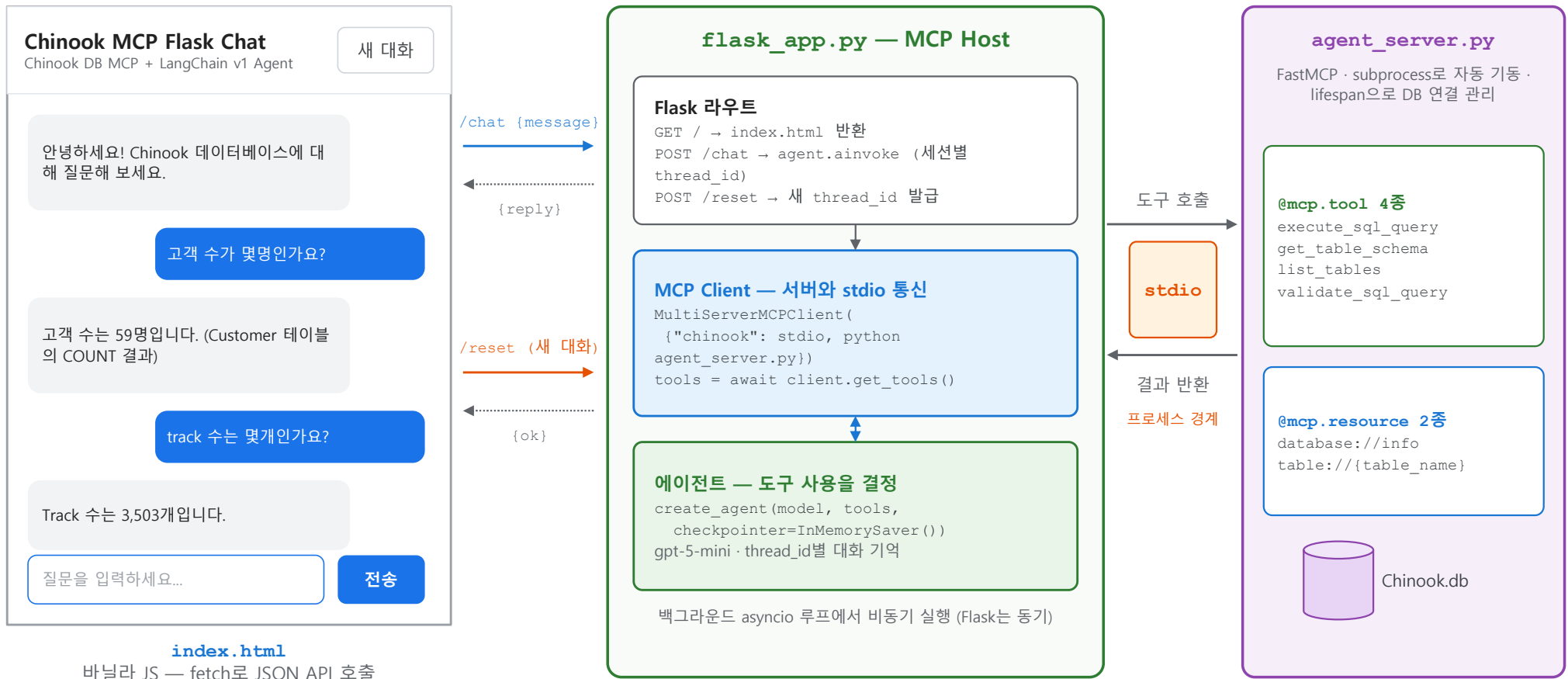
MCP_Server — 프로그램 구성과 관계



실행: python flask_app.py → http://localhost:5000 접속 — MCP 서버(agent_server.py)는 클라이언트가 subprocess로 자동 기동

실습 : 105_DB_MCP_Agent

Hands-on



실행: `python flask_app.py` → `http://localhost:5000`

핵심: 에이전트가 결정 → MCP Client가 stdio로 전달 → 서버가 Chinook.db에서 실행

Resource 테스트 질문

- DB Resource 테스트 질문

- **1. Database Info Resource:**

- 데이터베이스 정보를 보여줘"
 - "Chinook 데이터베이스에 대해 알려줘"
 - "데이터베이스 기본 정보를 확인해줘"

- **2. Table Info Resource:**

- "Employee 테이블 정보를 보여줘"
 - "Customer 테이블의 구조를 알려줘"
 - "Album 테이블의 스키마를 확인해줘"

- DB Tools 테스트 질문

- **1. SQL Query Execution Tool:**

- "직원이 몇 명인가요?"
 - "고객 수를 알려줘"
 - "앨범이 총 몇 개인가요?"
 - "가장 많이 팔린 곡은 무엇인가요?"
 - "매니저는 몇 명인가요?"

- **2. Table Schema Tool:**

- "Employee 테이블의 구조를 보여줘"
 - "Customer 테이블의 컬럼 정보를 알려줘"
 - "Track 테이블의 스키마를 확인해줘"

- **3. List Tables Tool:**

- "데이터베이스에 어떤 테이블들이 있나요?"
 - "사용 가능한 테이블 목록을 보여줘"
 - "모든 테이블 이름을 알려줘"

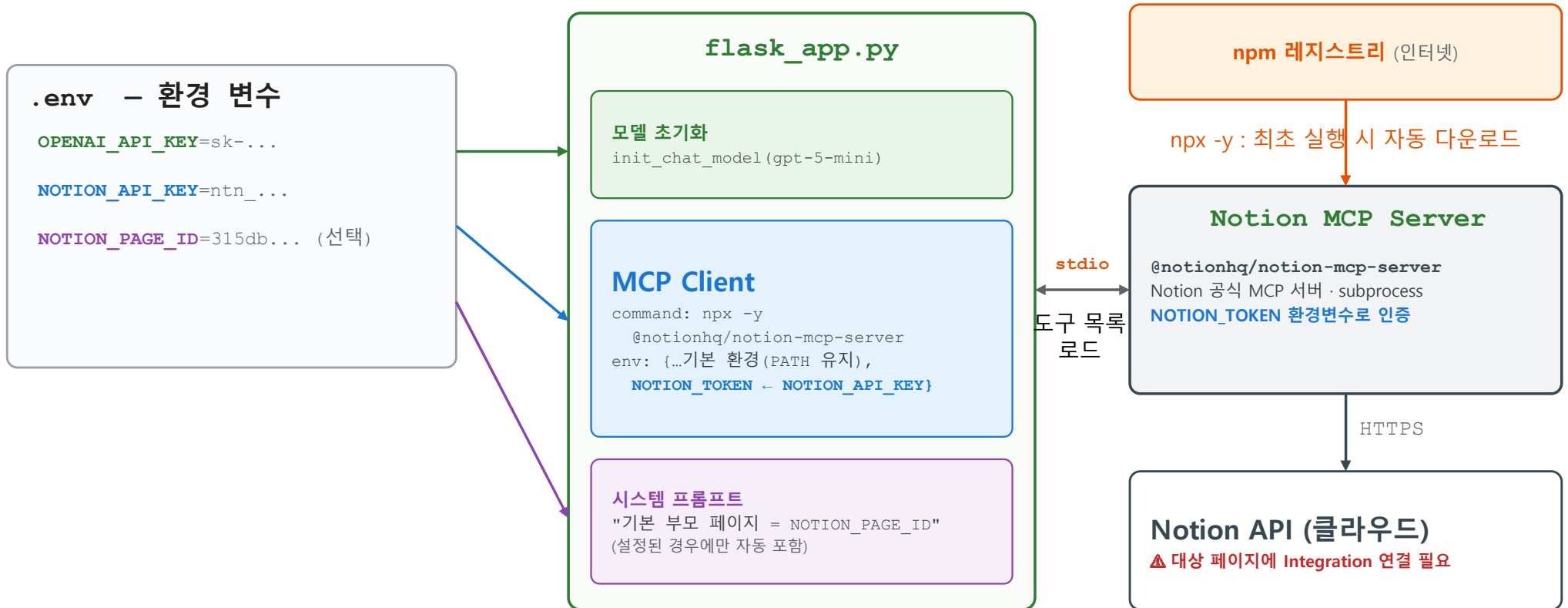
- **연속 대화 테스트:**

- 1단계: "직원이 몇 명인가요?"
 - 2단계: "그 중에서 매니저는 몇 명인가요?"

- **컨텍스트 참조 테스트:**

- 1단계: "고객이 몇 명인가요?"
 - 2단계: "그 고객들 중에서 미국 고객은 몇 명인가요?"
 - 3단계: "앞서 말한 고객들의 주문 건수는?"

Notion Page 연결 방법



Node.js 미설치 시: <https://nodejs.org/ko/download> → Windows 설치 프로그램(.msi) · 최초 npx 실행은 패키지 다운로드로 수십 초 걸릴 수 있음 (정상)

Notion API key 생성 - Notion의 개발자 설정(Integrations) 메뉴

Notion 개발자

Notion API와 Worker 런타임을 활용해 Notion의 기능을 확장하고, 시스템을 연결하며, 업무 흐름을 조율하세요



Worker 구축하기 베타

어떤 데이터든 Notion 데이터베이스로 동기화하세요. Notion 에이전트가 항상 따르는 스크립트를 생성하세요. 이 모든 기능을 호스팅 런타임에서 만나보세요.

1 Notion CLI 설치

`curl -fsSL https://ntn.dev | b...`

2 새 작업자 스캐폴딩하기

`ntn workers new`



연결 만들기

연결은 공개 및 비공개 통합으로, 자주 사용하는 외부 앱, 도구, 에이전트가 Notion API와 연동할 수 있게 해줍니다.

+ 신규 연결

신규 연결

Notion으로 앱과 에이전트, 워크플로를 인증하기 위한 연결을 생성하세요. 단일 워크스페이스 범위 또는 다중 워크스페이스 범위를 선택하세요.

연결 이름

kosa-test-2026-2

인증 방법

☒ 액세스 토큰

협업 참여자 간에 공유되는 워크스페이스 범위 내 정적 API 토큰으로, 자체 사용 권한 설정을 갖추고 있습니다. 워크스페이스 1개로 제한되며, 마켓플레이스에 등록할 수 없습니다.

☐ OAuth

사용자 범위 OAuth 2.0이며, 사용 권한은 로그인한 사용자로 제한됩니다. 여러 워크스페이스를 지원하며, 마켓플레이스에 등록할 수 있습니다.

설치 가능 워크스페이스:

YoungJea Oh의 워크스페이스 ▾

이 토큰은 선택한 워크스페이스에 적용됩니다.

연결을 생성하면 Notion의 개발자 약관에 동의하는 것으로 간주됩니다.

연결 생성하기

Notion으로 돌아가기

개발자

시작하기

Worker 신규

API 액세스 토큰

연결

Notion 개발자

Notion API와 Worker 런타임을 활용해 Notion의 기능을 확장하고, 시스템을 연결하며, 업무 흐름을 조율하세요

Worker 구축하기 베타

어떤 데이터든 Notion 데이터베이스로 동기화하세요. Notion 에이전트가 항상 따르는 스크립트를 생성하세요. 이 모든 기능을 호스팅 런타임에서 만나보세요.

1 Notion CLI 설치

```
curl -fsSL https://ntn.dev | b...
```

연결 만들기

연결은 공개 및 비공개 Notion API와 연동합니다.

+ 신규 연결

연결

Notion의 오픈 및 프라이빗 API 연결을 만들고 관리하세요. [자세히 알아보기](#)

연결

설치 가능 워크스페이스:

인증 유형

생성 시점

K	kosa-test-2026-2 콘텐츠 읽기, 가능 여부 업데이트 실패 및 삽입 및 사용자 정보 읽기	Y YoungJea Oh의 워크스페이스	액세스 토큰	2026년 7월 5일
C	ChatGPT연결 콘텐츠 읽기, 가능 여부 업데이트 실패 및 삽입 및 사용자 정보 읽기	Y YoungJea Oh의 워크스페이스	액세스 토큰	2026년 3월 26일
K	kosa-mar-1 콘텐츠 읽기, 가능 여부 업데이트 실패 및 삽입 및 사용자 정보 읽기	Y YoungJea Oh의 워크스페이스	액세스 토큰	2026년 3월 6일
K	KOSA-TEST-2026 콘텐츠 읽기, 가능 여부 업데이트 실패 및 삽입 및 사용자 정보 읽기	Y YoungJea Oh의 워크스페이스	액세스 토큰	2026년 3월 1일

```
# .env 파일
OPENAI_API_KEY=

# Notion API 설정
NOTION_API_KEY=
NOTION_PAGE_ID=
```

NOTION_API_KEY

NOTION_API_KEY=ntn_588632592525nUx*****EezC2PFiMcGeT0Eyz6xJ

연결 > 연결 관리하기

KOSA-TEST-2026

구성 콘텐츠 사용 권한 템플릿 소유자

정보 표시

K

연결 이름

KOSA-TEST-2026

관리하기

API 통합 토큰

이 토큰을 사용하여 이 워크스페이스에 대한 API 요청을 인증하세요. 토큰을 가진 모든 사용자가 연결을 대신하여 조치를 취할 수 있으므로, 토큰을 비공개로 유지하세요. 내부 연결에 대해 자세히 알아보세요.

설치 가능 워크스페이스:

Y YoungJea Oh의 워크스페이스

액세스 토큰

.....

Notion Page 생성

NOTION_PAGE_ID=315db4860e7480eebcbbdd05af92c01e

page-id

- <https://www.notion.so/nobody-project-315db4860e7480eebcbbdd05af92c01e>

notion.so/nobody-project-26cdb4860e7480468071d68f79141934

YoungJea Oh의 ... nobody 팀의 project 보고서

nobody 팀의 project 보고서

안녕하세요

자바 스프링 프레임워크의 장점

- 의존성 주입(DI): 객체 간의 결합도를 낮춰 유지보수
- 제어의 역전(IOC): 객체 생성과 생명주기 관리 자동화
- AOP(관점 지향 프로그래밍): 횡단 관심사 분리로 코
- 풍부한 생태계: Spring Boot, Spring Security, Spr
- 테스트 지원: 단위 테스트와 통합 테스트를 위한 강
- 확장성: 마이크로서비스 아키텍처 구축에 적합
- 커뮤니티 지원: 활발한 오픈소스 커뮤니티와 풍부한

연결 검색

Hightouch

IFTTT

K KOSA-TEST-2026
생성자: YoungJea Oh

Make

연결 관리

API 통합 개발

페이지 내용 복사하기

복제 Ctrl+D

움기기 Ctrl+⌘+P

휴지통으로 이동

A↓ 작은 텍스트

↔ 전체 너비

페이지 사용자 지정

페이지 잠금

편집 제안

번역

실행 취소 Ctrl+Z

가져오기

내보내기

위키로 전환

업데이트와 애널리틱스

버전 기록

알림 받기 댓글 >

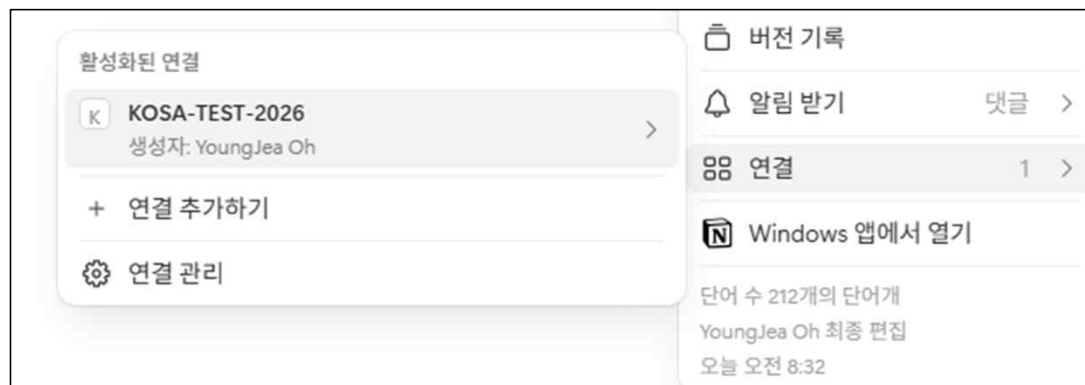
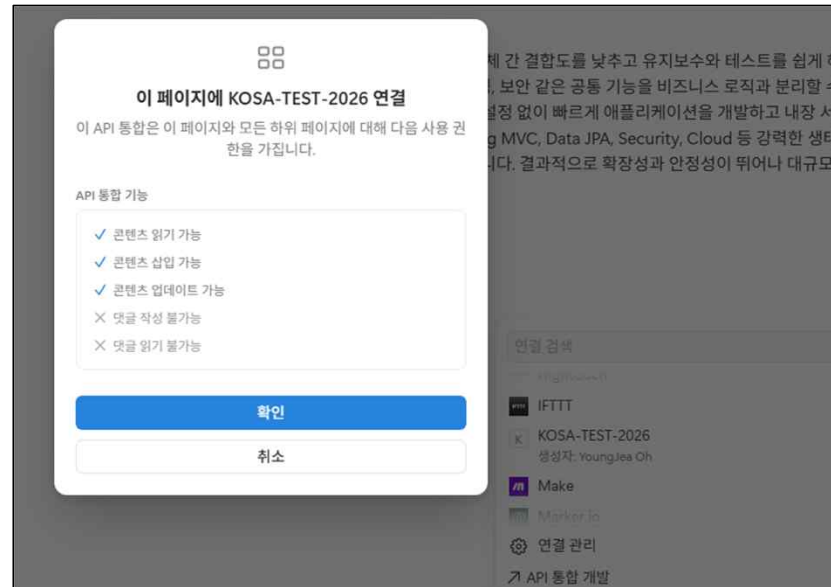
연결 없음 >

Windows 앱에서 열기

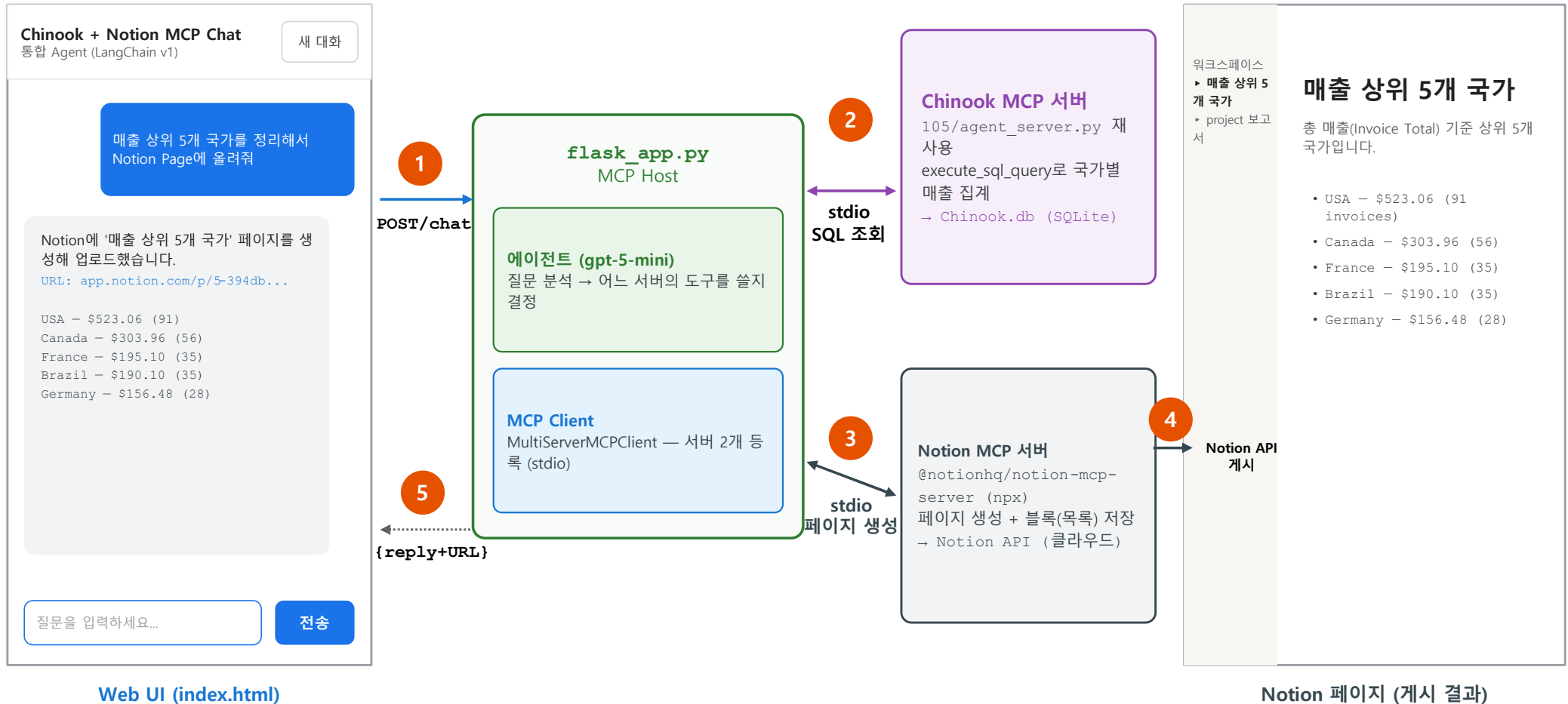
단어 수 212개의 단어개

YoungJea Oh 최종 편집

오늘 오전 8:24



실습 : 106_Notion_DB_MCP 구현



흐름: ① 질문(POST /chat) → ② Chinook 서버로 SQL 조회 → ③ 결과를 Notion 서버로 페이지 생성 → ④ Notion API 게시 → ⑤ URL 포함 답변 반환

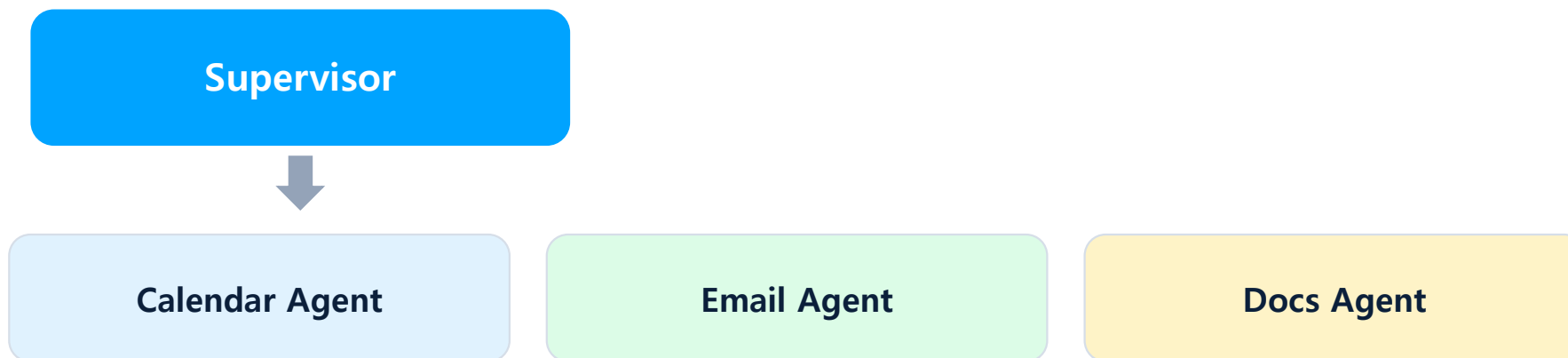
왜 멀티에이전트인가?

- 한 에이전트에 도구/지식이 너무 많으면 품질이 흔들림
→ 컨텍스트 창을 과부하시키지 않고 전문 지식 제공
- 역할을 나누면 프롬프트/도구/컨텍스트가 깔끔
→ 여러 팀이 독립적으로 기능을 개발하고 유지 관리
- 조율자는 “라우팅/핸드오프/검증”에 집중

- Subagents - 메인 에이전트는 하위 에이전트들을 도구로 활용하여 조정
- Handoffs - 동작은 상태에 따라 동적으로 변경
- Router - 입력을 분류하고 하나 이상의 전문 에이전트로 전달
- Skills - 필요에 따라 특화된 프롬프트와 지식이 로드

설계 체크리스트

- 각 에이전트의 “책임 범위”를 한 문장으로 적는다
- 입력/출력 스키마를 정한다(Structured Output 권장)
- 공유 상태는 최소화(필요한 정보만 전달)
- 실패 시 fallback 경로를 만든다



Supervisor는 요청을 분해하고, 적절한 Worker에 위임(필요 시 순차 실행).

핵심 구현 포인트

- Worker별 Toolset/Prompt 분리
- Supervisor는 "계획/라우팅"에 집중
- 결과는 공통 스키마로 합성(예: ActionResult)

실습 미션

- 사용자: "다음 주 화요일 2시에 회의 잡고 리마인더 이메일 보내줘"
- Supervisor: `schedule_event` → `send_email` 순서로 실행
- Streamlit UI에서 "실행 계획"을 먼저 보여주기

에이전트→도구 래핑 + 3계층 아키텍처

Subagent = Tool 래핑 패턴

- 하위 에이전트를 도구로 변환 (as_tool())
- 상위 에이전트가 하위를 '도구 호출'로 위임
- 각 하위 에이전트는 독립적 도구 집합 보유

3계층 아키텍처

- L1: Orchestrator (라우팅/위임 결정)
- L2: Specialist Agents (전문 에이전트)
- L3: Tools (실제 작업 수행)

as_tool() 변환

```
research = create_react_agent(
    model, [tavily_search]
)

research_tool = research.as_tool(
    name="researcher",
    description="Research"
)
```

장점

- 하위 에이전트 독립 개발/테스트
- 상위 에이전트 로직 단순화

실습 : 310_Sub-Agents Pattern (하위 에이전트 위임 패턴)

Subagents Pattern (하위 에이전트 위임 패턴)

슈퍼바이저 패턴(Supervisor Pattern) 은 중앙 감독 에이전트가 여러 전문 하위 에이전트를 조율하는 멀티 에이전트 아키텍처입니다.

이 패턴은 서로 다른 전문 지식이 필요한 작업을 수행할 때 특히 효과적입니다. 각 도메인별로 전문 에이전트를 만들고, 슈퍼바이저가 전체 워크플로우를 이해하고 관리합니다.

참고: [LangChain 공식 문서 - Subagents Personal Assistant](#)

학습 내용

- 하위 에이전트(Sub-agents) 생성
- 에이전트를 도구로 래핑
- 슈퍼바이저 에이전트 구성
- 복합 다중 도메인 요청 처리

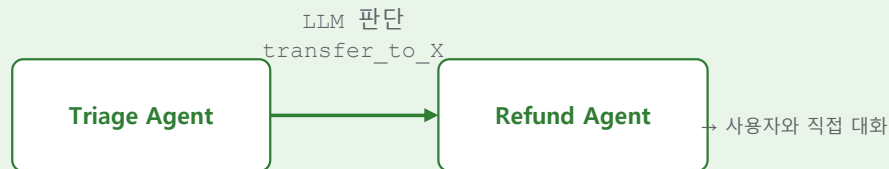
Handoff — OpenAI SDK vs LangChain 개념 차이

핵심 아이디어(제어권을 넘긴다)는 같다 — 누가 전환을 결정하고, 어느 층위에서 구현되는지가 다르다

OpenAI Agents SDK

SDK 내장 기능 · LLM 기반 오케스트레이션

- handoffs=[refund_agent] 한 줄 → transfer_to_refund_agent 도구 자동 생성
- LLM이 위임 시점을 자율 판단 (도구 호출로 표현)
- 제어권 완전 이전 — 대상 에이전트가 사용자와 직접 대화
- 대화 히스토리 자동 인계 (input_filter로 가공)



LangChain/LangGraph

상태 머신으로 직접 구성 · 코드 기반 오케스트레이션

- 내장 기능 아님 — State + Command(goto)로 구성
- 도구가 상태를 기록(record_issue_type) → 그래프가 다음 단계 결정
- '에이전트 위임'보다 '단계(state) 전환'에 가까움
- 결정적 흐름 — 상태별 정책 분리, 감사/로깅 용이



Command(goto=...) · 상태 기반 전환 (+ ESCALATE → HITL)

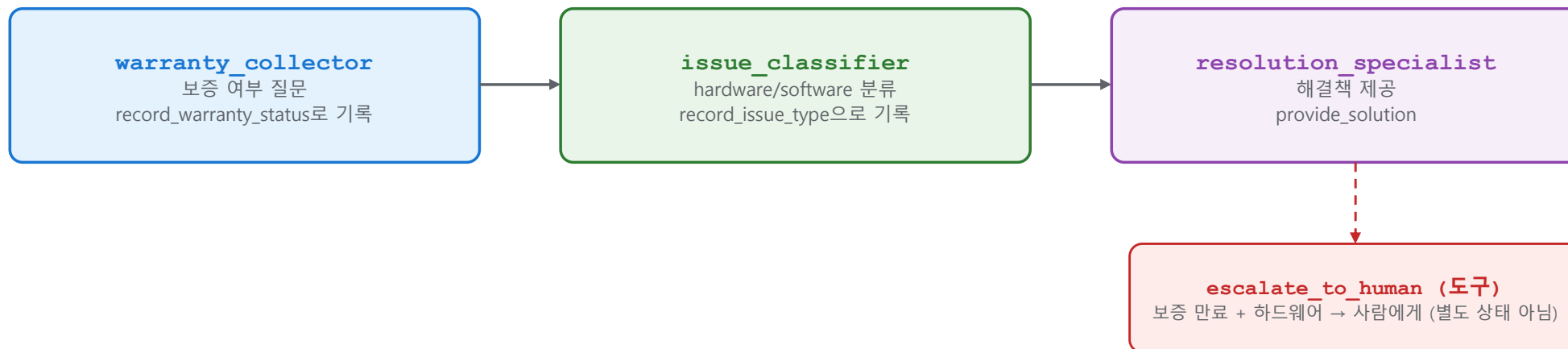
OpenAI Handoff는 LLM 기반, LangChain 패턴은 코드 기반으로 개념 구현

Pattern 2 — Handoffs: 하나의 에이전트가 “상태(state)”에 따라 다른 단계로 전환

예) 병원 접수처

접수 → 진료과 배정 → 진료 → (필요하면 큰 병원 이송) → 수납. 환자 차트(상태)에 기록이 쌓이고, 기록을 보고 다음 창구가 정해진다

고객지원 워크플로 — 한 에이전트가 3단계를 이동



핵심 아이디어 요약:

- ① 대화하면서 알게 된 정보를 상태(state)에 기록한다
- ② 다음 단계는 상태를 보고 정해진다 (LLM 혼자 정하지 않음)
- ③ 단계마다 쓸 수 있는 도구·정책이 다르다 → 안전하고 추적 쉬움

Handoffs 패턴 — 예시로 보는 동작과 구현

Multi-agent

예시: "폰이 고장났어요" — 대화가 진행될수록 상태가 쌓인다

턴 1 "폰이 고장났어요"

"보증 기간인가요?" → 답 듣고 기록
`record_warranty_status("in_warranty")`

Workflow State
보증: O

턴 2~3 "화면이 깨졌어요"

문제 유형을 기록
`record_issue_type("hardware")`

Workflow State
유형: hardware

턴 4 (상태 2개 확보 후)

상태를 보고 해결책 제시 또는 사람 연결
`provide_solution / escalate_to_human`

Workflow State
→ 해결 단계로

→ LLM의 기억력에 맡기지 않고, 중요한 사실을 상태에 명시적으로 저장

구현 핵심 (LangGraph)

공유 상태 (SupportState)

`current_step · warranty_status · issue_type`

전환 — 도구가 Command 반환

`Command(update={warranty_status, current_step: "issue_classifier"})`
기록과 단계 전환을 한 번에

단계별 주입 — `wrap_model_call`

STEP_CONFIG에서 현재 단계의 프롬프트·도구만
`request.override(...)`로 적용 + `requires` 진입 조건 검증

Subagents 패턴과의 차이

- Subagent = 호출-복귀: 결과가 상사에게 돌아옴
- Handoff = 전진: 제어가 다음 단계로 넘어가고 안 돌아옴
- 한 에이전트가 단계를 이동 또는 단계마다 별도 에이전트를 둘 수도 있음(next_agent)

실습 : 320_Handoffs Pattern (에이전트 간 핸드오프 패턴)

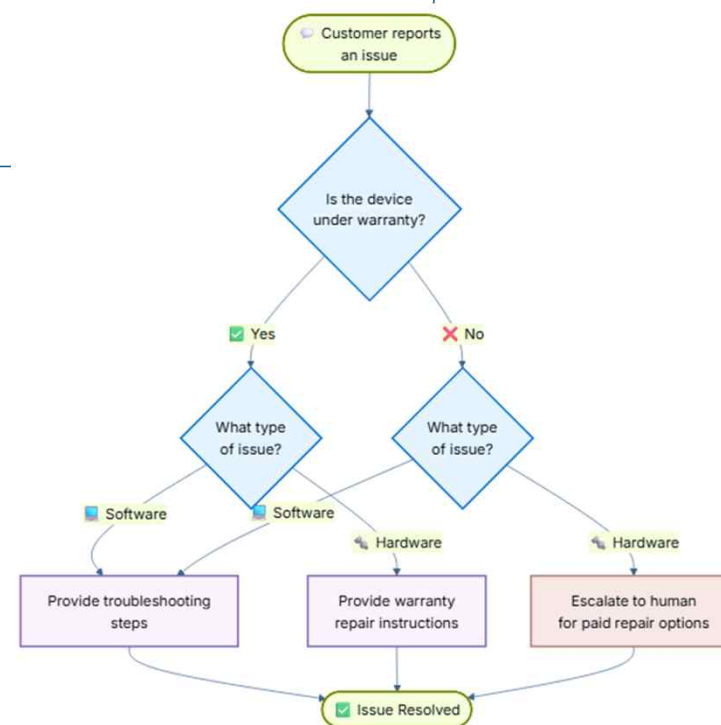
Handoffs Pattern (핸드오프 패턴: 한 에이전트가 세 전문가 역할을 순서대로 수행)"

핸드오프 패턴(**Handoffs Pattern**) 은 단일 에이전트가 여러 상태(state)를 거치며 작업을 수행하는 패턴입니다. 각 상태마다 다른 프롬프트와 도구를 사용하여, 복잡한 워크플로우를 단일 에이전트로 구현할 수 있습니다.

이 튜토리얼에서는 고객 지원(Customer Support) 시나리오를 구현합니다:

- 보증 상태 확인 → 이슈 분류 → 해결책 제공

참고: LangChain 공식 문서 - Handoffs Customer Support



Pattern 3 - Router Pattern

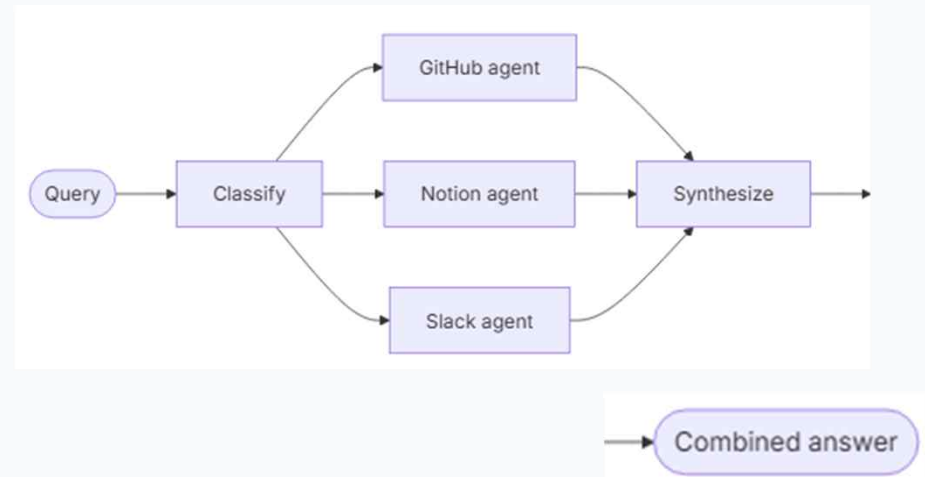
개요

- 라우팅 단계가 사용자 질문을 보고 "어떤 소스를 조회할지" 결정
- 각 소스는 전문 에이전트가 맡고, 필요한 에이전트들을 병렬 실행한 뒤, 결과를 합쳐서 답변

핵심 키워드

- 선택적 라우팅
- 병렬 실행
- 전문화된 에이전트
- 결과 합성

다이어그램



- 선택적 라우팅 — 질문에 맞는 소스 자동 선택
- 병렬 실행 — 지연시간 감소
- 전문화된 에이전트 — 소스별 최적화
- 결과 합성 — 자연스러운 답변 통합

Router의 역할

- 질문을 보고 "어떤 지식/도구"로 갈지 결정
- 소스별 전용 Retriever/Agent를 둬

예시

- 사내 위키 RAG
- 제품 매뉴얼 RAG
- DB 질의(SQL)
- 웹 검색(선택)

구현 팁

- 라우팅은 Structured Output으로
(예: route=wiki|manual|db)
- 불확실하면 2개 경로 병렬 실행 후 합치기
- 각 소스는 메타데이터/출처 정책을 통일

LangGraph 워크플로우 + 결과 통합

Router 워크플로우 구성

- conditional_edge로 쿼리 유형별 분기
- 각 브랜치가 전문 에이전트/도구 호출
- 결과를 공유 상태에 병합

결과 통합 전략

- Aggregator 노드: 분산 결과 종합
- ReduceAction으로 상태 리스트 병합
- 최종 응답 생성 노드에서 요약

Router 분기 예시

- 질문 유형 → "기술문서" → RAG Agent
- 질문 유형 → "데이터" → SQL Agent
- 질문 유형 → "일반" → Chat Agent

Send API (병렬 실행)

- Send()로 여러 노드 동시 실행
- Map-Reduce 패턴 구현 가능

실습 : 330_Router_Pattern (라우터 패턴)

Router Pattern (라우터 패턴)

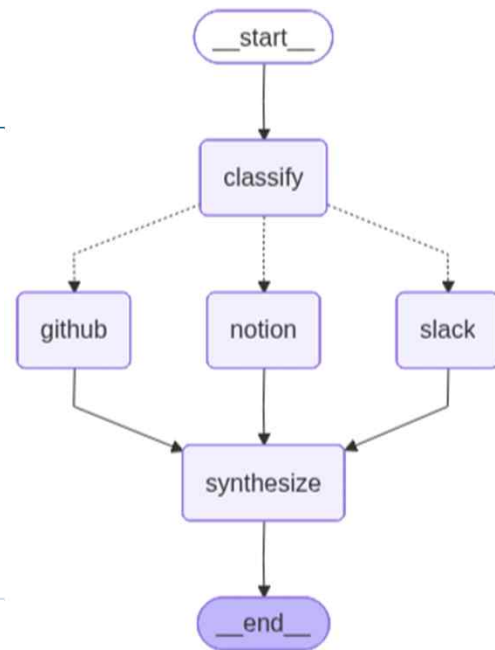
라우터 패턴(Router Pattern) 은 라우팅 단계에서 입력을 분류하고, 전문 에이전트로 라우팅하며, 결과를 통합하여 결합된 응답을 생성하는 멀티 에이전트 아키텍처입니다.

이 패턴은 조직의 지식이 서로 다른 수직 영역(verticals) 에 분산되어 있을 때 특히 효과적입니다. 각 영역은 고유한 도구와 프롬프트를 가진 전문 에이전트가 필요합니다.

이 튜토리얼에서는 GitHub, Notion, Slack 세 가지 지식 소스를 통합하는 라우터를 구축합니다.

참고: [LangChain 공식 문서 - Router Knowledge Base](#)

그래프 구성 (병렬 라우팅)



Pattern 4 — Skills: 온디맨드 컨텍스트 로딩

Multi-agent

Skills의 아이디어 — Progressive Disclosure

- 모든 지식을 한 번에 프롬프트에 넣지 않는다
- 설명(description)만 먼저 노출 → 상세 내용은 "필요할 때만" load_skill로 로드

Skill의 3요소 (TypedDict)

name 스킬 식별자 — load_skill 호출에 사용

description 가벼운 메타 정보

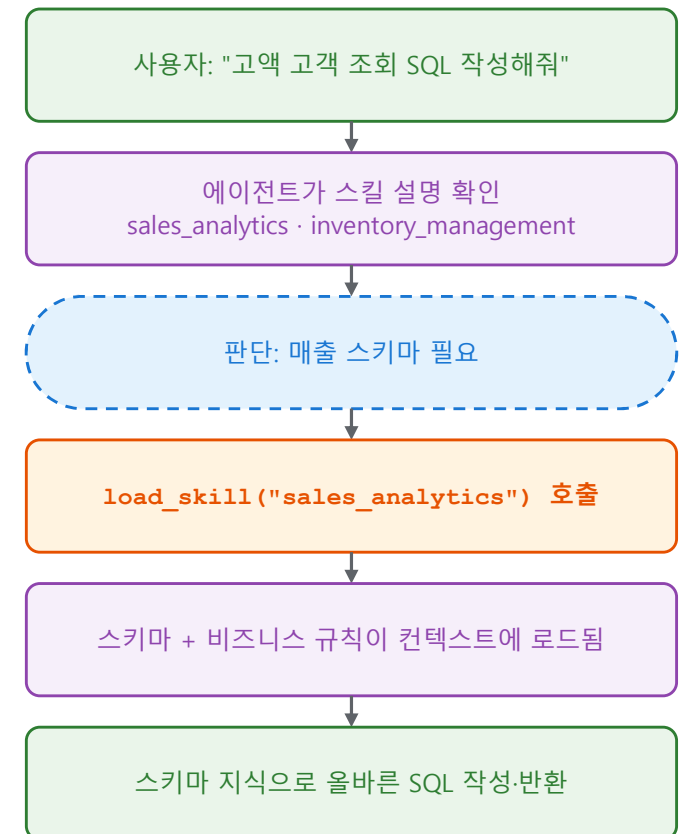
항상 프롬프트에

content 스키마·규칙·예시 등 상세 지식

필요할 때만

SkillMiddleware가 wrap_model_call 시점에 설명 주입 + load_skill 도구 자동 등록
(tools=[load_skill])

실행 흐름 — SQL assistant 예시



실습 : 340_Skills Pattern (스킬 기반 패턴)

Skills Pattern (스킬 기반 패턴 - Progressive Disclosure)

Progressive Disclosure는 에이전트가 필요한 정보를 필요할 때만 로드하는 방식입니다. 시스템 프롬프트에는 스킬의 간단한 설명만 노출하고, 상세 스키마-비즈니스 로직은 `load_skill` 도구로 불러옵니다.

주요 포인트 정리

1. **Progressive Disclosure**: 스킬 설명만 먼저 노출, 상세 내용은 `load_skill`로 필요 시 로드
2. **SkillMiddleware**: 시스템 프롬프트에 스킬 설명 추가, `load_skill` 도구 등록
3. **컨텍스트 절약**: 필요한 1~2개 스킬만 로드해 토큰 사용 최소화
4. **팀 확장성**: 도메인별 스킬을 독립적으로 추가-유지보수 가능
5. **트레이드오프**:
 - 스킬을 필요할 때 로드하므로 첫 요청에 도구 호출이 추가되어 지연(latency) 이 늘어남
 - 기본 구현은 프롬프트로 `load_skill` 사용을 유도할 뿐 강제하지 못함 → 아래 실습문제에서 상태 기반 제약으로 보완

참고: LangChain 공식 문서 - [Build a SQL assistant with on-demand skills](#)

멀티 에이전트 패턴 종합 비교

패턴	핵심 개념	제어 흐름	핵심 구현 (API)	적합한 작업	실습
Subagents	하위 에이전트를 도구로 래핑 — 상위 에이전트가 필요할 때 호출	호출-복귀 (결과가 부모로 돌아옴)	<code>agent.as_tool()</code> L1 오케스트레이터 → L2 전문 → L3 도구	계층적 업무 분담 (개인 비서: 캘린더·이메일·노트)	310
Handoffs	상태(state)에 따라 단계 전환 — 단일 에이전트가 역할을 바꿔가며 수행	전진 (다음 단계로, 안 돌아옴)	<code>Command(update={...})</code> <code>wrap_model_call</code> + <code>STEP_CONFIG</code>	단계형 프로세스 (고객 지원: 보증→분류→해결)	320
Router	질문 유형을 분류해 전문 에이전트로 분기 — 필요 시 병렬 실행 후 통합	분기-병합 (선택 라우팅 + 병렬)	Structured Output 분류 <code>conditional_edges</code> + <code>Aggregator</code>	멀티 소스 지식베이스 (위키·매뉴얼·DB·웹 검색)	330
Skills	지식을 필요할 때만 로드 (Progressive Disclosure) — 설명만 상시 노출	온디맨드 로드 (도구 호출로 컨텍스트 주입)	<code>SkillMiddleware</code> (tools 등록) <code>load_skill · description/content</code> 분리	도메인 지식이 방대할 때 (SQL assistant · 토큰 절약)	340

선택 가이드: 위에서 지시하고 결과만 종합 → Subagents · 정해진 단계를 밟는 프로세스 → Handoffs
질문마다 갈 곳이 다름 → Router · 지식이 많아 토큰이 부담 → Skills

LangChain Deep Agent

Deep Agents Overview

- LLM 기반 에이전트를 가장 쉽게 시작하는 방법
- 계획 수립, 서브에이전트, 파일 시스템을 활용한 복잡한 작업 수행
- **에이전트 하네스(Agent Harness)란? → LLM: 엔진, Harness: 자동차**
 - 다른 에이전트 프레임워크와 동일한 도구 호출 루프
 - 단, 도구와 기능이 내장(built-in)되어 즉시 사용 가능
 - LangChain 빌딩 블록 + LangGraph 런타임 위에 구축된 독립 라이브러리
- **내장 기능**
 - 작업 계획(Task Planning)
 - 컨텍스트 관리를 위한 파일 시스템
 - 서브에이전트 생성(Subagent Spawning)
 - 장기 메모리(Long-term Memory)
- **deepagents 라이브러리 구성**
 - **SDK**: 모든 작업을 처리하는 에이전트 구축 패키지
 - **CLI**: SDK 위에 구축된 터미널 코딩 에이전트

일반 Agent vs Deep Agent 비교

항목	일반 Agent — create_agent()	Deep Agent — create_deep_agent()
본질	미니멀 도구 호출 루프 — 필요한 것만 조합	같은 루프 + 하네스 미들웨어 내장 (결국 create_agent에 미들웨어를 얹은 것)
계획	없음 — 직접 구현	write_todos 내장 — 작업 분해·상태 추적
서브에이전트	agent.as_tool()로 직접 구성	task 도구 내장 — 격리 컨텍스트에서 실행, 결과만 압축 반환
파일 처리	파일 도구를 직접 등록해야 함	가상 파일시스템 내장 (ls-read-write-edit-glob-grep) — 기본 인메모리, 디스크/샌드박스 로 교체 가능
코드 실행	도구로 직접 구성	옵션 — 샌드박스 백엔드(execute 셸) 또는 QuickJS 인터프리터(eval). 기본 구성엔 없음
컨텍스트 관리	수동 (직접 요약·트리밍)	자동 요약 + 대용량 결과 오프로딩 + 프롬프트 캐싱
장기 메모리	직접 구현 (Store 등)	AGENTS.md 메모리 내장 — 세션 간 지식 유지·갱신
멀티턴 대화	checkpointer + thread_id	동일 (같은 LangGraph 런타임) — 차이 없음
적합 작업	대화·Q&A·단발 도구 호출	복잡한 장기 작업 — 분석·코딩·보고서 생성

주의: Deep Agent는 클라우드 실행·내장 브라우저·자동 패키지 설치가 아니다 — 같은 프로세스에서 도는 라이브러리이며, 샌드박스·코드 실행은 옵션 백엔드다.

Deep Agent vs. LangChain vs. LangGraph — 역할과 스택

Framework

LangChain

- Models · Tools · Runnables 추상화
- 프로바이더 통합 (700+)
- create_agent로 빠른 시작

언제 사용?

- 단순한 에이전트 구축
- 팀 표준화 · 통합 인터페이스

Runtime

LangGraph

- Durable execution
- Streaming & Persistence
- HITL & Checkpointing

언제 사용?

- 상태 관리가 필요한 워크플로
- 세밀한 흐름 제어가 필요할 때

Harness

Deep Agents

- Planning (write_todos)
- Subagent spawning
- 파일시스템 · 장기 메모리 · 스킬

언제 사용?

- 복잡한 multi-step 작업
- 대규모 컨텍스트 관리 필요 시

세 가지는 대체재가 아니라 한 스택 — Deep Agent를 쓰면 아래 층을 그대로 물려받는다

Deep Agents — 완성형 하네스 `create_deep_agent()`

LangChain — 빌딩블록·추상화 `create_agent · tools · middleware`

LangGraph — 런타임 엔진 `checkpointer · HITL · streaming`

create_deep_agent 내부 구조

create_deep_agent()는 결국 middleware가 추가된 create_agent()를 반환

```
def create_deep_agent(model, tools, ...):  
    deepagent_middleware = [  
        TodoMiddleware(),  
        FilesystemMiddleware(backend),  
        SubAgentMiddleware(subagents),  
        ContextMiddleware(),  
        ...  
    ]  
  
    return create_agent(  
        model,  
        tools=tools,  
        middleware=deepagent_middleware,  
        ...  
    )
```



Deep Agents가 추가하는 Middleware

TodoMiddleware — 작업 계획/추적
FilesystemMiddleware — 파일 I/O (절대경로)
SubAgentMiddleware — 서브에이전트
ContextMiddleware — 컨텍스트 관리
HITLMiddleware — 사람 승인 (CLI)

반환값

create_agent() = CompiledStateGraph

💡 핵심

동일한 도구 호출 루프에
하네스 기능(middleware)만 추가한 것

<https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/deepagents/graph.py>

<https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/deepagents/middleware/filesystem.py>

Harness Capabilities (1/2)

1. Planning

write_todos 도구로 작업을 단계별 분해

상태 추적: pending → in_progress → completed

2. Virtual Filesystem

ls, read_file, write_file, edit_file 도구 제공

→ **Backend**라는 코드 실행 미들웨어를 통해 파일시스템에 직접 접근

교체 가능한 백엔드 (State / Filesystem / Sandbox)

3. Subagents

task()로 임시 서브에이전트를 생성하여 작업 위임

컨텍스트 격리, 토큰 효율성, 병렬 실행 가능

4. Context Management

대용량 도구 결과(20K+ 토큰) 자동 파일 오프로드

컨텍스트 윈도우 한계 시 히스토리 자동 요약

Harness Capabilities (2/2)

5. Code execution

셸 명령 실행, 패키지 설치, 스크립트 실행 → 에이전트가 코드를 직접 실행해야 할 때 사용
샌드박스 없는 에이전트는 코드를 파일로 저장 까지만 가능하고, 실행은 못 함

6. Human-in-the-Loop

interrupt_on 파라미터로 특정 도구 실행 전 일시정지
사람이 승인 / 수정 / 거부 후 재개

7. Skills

SKILL.md로 전문 워크플로우 정의
필요할 때만 전체 로드

8. Memory

AGENTS.md 파일 기반의 영속적 컨텍스트
대화 간 유지 — 코딩 스타일, 선호도, 가이드라인 저장

Deep Agent 생성/실행 & 멀티턴

Deep Agent 생성

- create_deep_agent() 팩토리 함수
- 주요 파라미터: model, tools, backend,
- system_prompt, subagents, memory 등
- backend로 실행 환경 교체 가능 (기본: 인메모리)

멀티턴 대화

- thread 기반 상태 유지
- 이전 실행 결과 참조하여 연속 작업
- 파일 생성→수정→분석 연속 워크플로우

내장 도구 (기본 제공)

- write_todos: 작업 계획/추적
- ls, read/write/edit_file
- task (서브에이전트 호출)
- execute (Sandbox 시 코드 실행)

확장 도구 (tools로 추가)

- 웹 검색 (Tavily 등)
- MCP 어댑터, 커스텀 도구

실습 : 400_Deep Agent

Deep Agent (딥 에이전트)

Deep Agent는 데이터 분석처럼 계획(planning), 코드 실행(code execution), 아티팩트(스크립트, 리포트, 시각화 등) 생성이 필요한 복잡한 작업을 처리하기 위해 설계된 에이전트입니다.

이 노트북에서는 Deep Agent를 사용하여:

1. CSV 파일을 분석하고
2. 커스텀 도구를 통해 결과를 공유하는

데이터 분석 에이전트를 구축합니다.

참고:

- [LangChain 공식 문서 - Deep Agents](#)
- [LangChain 공식 문서 - Data Analysis](#)
- [LangChain 공식 문서 - Backends](#)

핵심 개념:

- **Backend**: 샌드박스 환경에서 코드를 실행하는 백엔드 시스템
- **Custom Tools**: 외부 서비스(Slack, 이메일 등)와 연동하는 커스텀 도구
- **Checkpointers**: 멀티턴 대화를 지원하는 체크포인트

실습 : 401_Deep Agent_Advanced

Deep Agent 고급 기능 (Human-in-the-Loop · Skills · Memory)

400_Deep_Agent.ipynb에서 배운 Deep Agent의 **harness** 고급 기능 3가지를 실습합니다.

기능	파라미터	역할
Human-in-the-Loop	<code>interrupt_on</code>	특정 도구 실행 전 일시정지 → 사람이 승인/수정/거부 후 재개
Skills	<code>skills</code>	SKILL.md로 전문 워크플로우 정의, 필요할 때만 전체 로드
Memory	<code>memory</code>	AGENTS.md 파일 기반의 영속적 컨텍스트 (대화 간 유지)

참고:

- [LangChain 공식 문서 - Human-in-the-Loop](#)
- [LangChain 공식 문서 - Skills](#)
- [LangChain 공식 문서 - Memory](#)

감사합니다.